Lecture3a

September 14, 2025

1 CS5489 - Machine Learning

2 Lecture 3a - Discriminative Classifiers

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

1. Discriminative linear classifiers
2. Logistic regression
3. SVM & Kernel SVM
4. Summary

[1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
```

4 Classification with Generative Model

- Steps to build a classifier
  1. Collect training data (features $\mathbf{x}$ and class labels y)
  2. Learn class-conditional distribution (CCD), $p(\mathbf{x}|y)$.
  3. Use Bayes' rule to calculate class probability, $p(y|\mathbf{x})$.
- **Note:** the data is used to learn the CCD – the classifier is secondary.
  - Density estimation is an "ill-posed" problem – which density to use? how much data is needed?
- Advice from Vladimir Vapnik (inventor of SVM): > When solving a problem, try to avoid solving a more general problem as an intermediate step.
- **Discriminative solution**
  - Solve for the classifier $p(y|\mathbf{x})$ directly!

- Terminology
  - **"Discriminative"** - learn to directly discriminate the classes apart using the features.
  - **"Generative"** - learn model of how the features are generated from different classes.

5 Revisit the Naive Bayes Gaussian Classifier

- CCDs: assume the same variance for all Gaussians:
  - $p(\mathbf{x}|y=1) = \prod_{i=1}^{D} N(x_i|\mu_i, \sigma^2)$
  - $p(\mathbf{x}|y=2) = \prod_{i=1}^{D} N(x_i|\nu_i, \sigma^2)$
- prior:
  - $p(y=1) = \pi_1$, $p(y=2) = \pi_2$.
- look at the log-ratio of CCDs,

$$\begin{aligned}
\log \frac{p(\mathbf{x}|y=1)}{p(\mathbf{x}|y=2)} &= \log \frac{\prod_{i=1}^{D} N(x_i|\mu_i, \sigma^2)}{\prod_{i=1}^{D} N(x_i|\nu_i, \sigma^2)} \\
&= \sum_{i=1}^{D} \log N(x_i|\mu_i, \sigma^2) - \log N(x_i|\nu_i, \sigma^2) \\
&= \sum_{i=1}^{D} -\frac{1}{2\sigma^2}(x_i - \mu_i)^2 + \frac{1}{2\sigma^2}(x_i - \nu_i)^2 \\
&= \frac{1}{2\sigma^2} \sum_{i=1}^{D} (2x_i\mu_i - \mu_i^2 - 2x_i\nu_i + \nu_i^2) \\
&= \frac{1}{2\sigma^2} \sum_{i=1}^{D} 2(\mu_i - \nu_i)x_i - \mu_i^2 + \nu_i^2
\end{aligned}$$

- Thus

$$\log \frac{p(\mathbf{x}|y=1)}{p(\mathbf{x}|y=2)} = \frac{1}{\sigma^2} \sum_{i=1}^{D} (\mu_i - \nu_i)x_i + \frac{1}{2\sigma^2} \sum_{i=1}^{D} (\nu_i^2 - \mu_i^2)$$

- **Bayes decision rule:** Compute the posterior probability of each class $p(y=j|\mathbf{x})$
  - select class 1 when:

$$\begin{aligned}
\log p(y=1|\mathbf{x}) &> \log p(y=2|\mathbf{x}) \\
\log p(\mathbf{x}|y=1) + \log p(y=1) &> \log p(\mathbf{x}|y=2) + \log p(y=2) \\
\log \frac{p(\mathbf{x}|y=1)}{p(\mathbf{x}|y=2)} + \log \frac{p(y=1)}{p(y=2)} &> 0
\end{aligned}$$

- substituting for the CCDs and priors, the BDR is:
  - select class $y=1$ when:

$$\sum_{i=1}^{D} \frac{1}{\sigma^2}(\mu_i - \nu_i)x_i + \frac{1}{2\sigma^2} \sum_{i=1}^{D} (\nu_i^2 - \mu_i^2) + \log \frac{\pi_1}{\pi_2} > 0$$

[2]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris2.csv', delimiter=',', skiprows=1)
```

Lecture3b

September 14, 2025

1 CS5489 - Machine Learning

2 Lecture 3b - Support Vector Machines

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

1. Discriminative linear classifiers
2. Logistic regression
3. SVM & Kernel SVM
4. Summary

4 Support vector machines

- With logistic regression we used a maximum-likelihood framework to learn the separating hyperplane.
- Let's consider a purely geometric approach...

[1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
import warnings
#warnings.filterwarnings("ignore")
```

5 Linearly-Separable Data

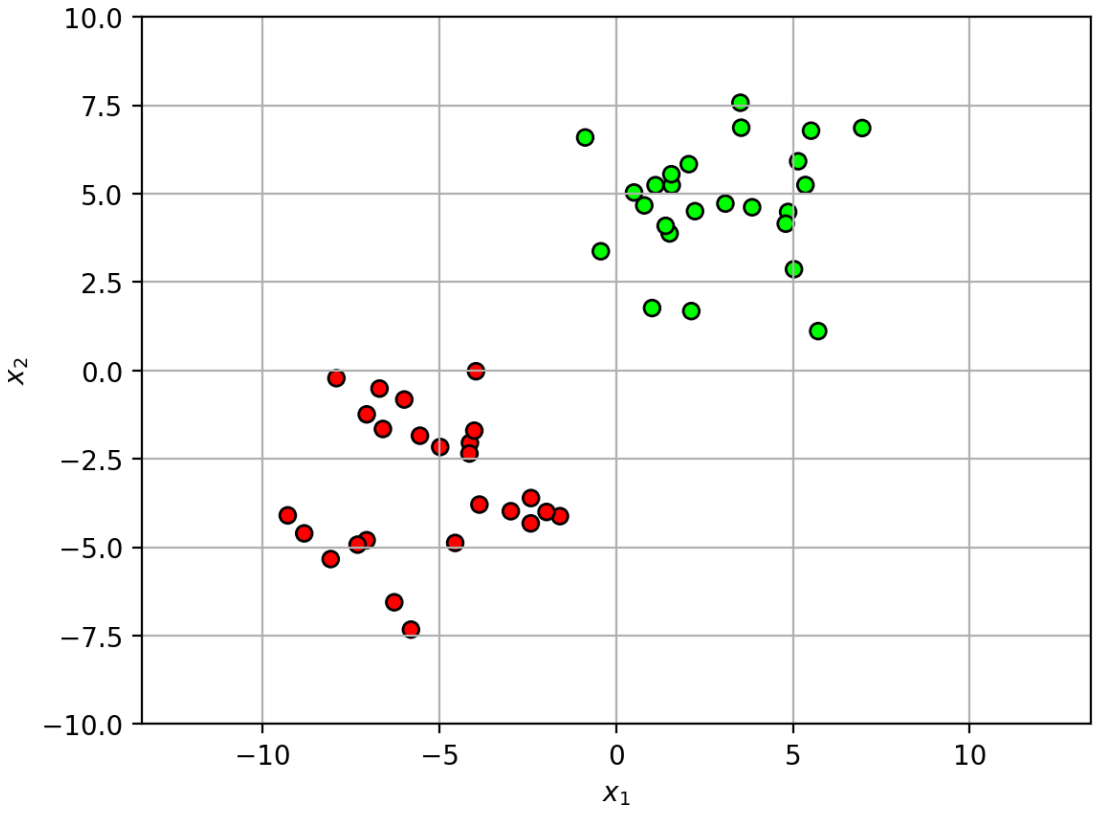
- For now, assume the training data is *linearly separable*
  - the two classes in the training data can be separated by a line (hyperplane)

[2]:
```
# generate random data
X,Y = datasets.make_blobs(n_samples=50,
         centers=2, cluster_std=2, n_features=2,
         center_box=(-5,5), random_state=4487)
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#FFFFFF", "#00FF00"])

lsdatafig = plt.figure()
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1$'); plt.ylabel('$x_2$')
axbox = [-10,10,-10,10]
plt.axis('equal'); plt.axis(axbox); plt.grid(True);
plt.close()
```

[3]:
```
lsdatafig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[3]:



6 Which is the best separating line?

- there are many possible solutions...

[4]:
```
def drawplane(w, b=None, c=None, wlabel=None, poscol=None, negcol=None, lw=2,␣
↪ls='k-', label=None):
    # w^Tx + b = 0
    # w0 x0 + w1 x1 + b = 0
    # x1 = -w0/w1 x0 - b / w1

    # OR
    # w^T (x-c) = 0 = w^Tx - w^Tc  --> b = -w^Tc
    if c is not None:
        b = -sum(w*c)

    # the line
    if (abs(w[0])>abs(w[1])):   # vertical line
        x0 = array([-30,30])
        x1 = -w[0]/w[1] * x0 - b / w[1]
    else:                       # horizontal line
        x1 = array([-30,30])
        x0 = -w[1]/w[0] * x1 - b / w[0]

    # fill positive half-space or neg space
    #if (poscol):
    #    polyx = [x0[0], x0[-1], x0[-1], x0[0]]
    #    polyy = [x1[0], x1[-1], x1[0], x1[0]]
    #    plt.fill(polyx, polyy, poscol, alpha=0.2)
    #
    #if (negcol):
    #    polyx = [x0[0], x0[-1], x0[0], x0[0]]
    #    polyy = [x1[0], x1[-1], x1[-1], x1[0]]
    #    plt.fill(polyx, polyy, negcol, alpha=0.2)

    if (poscol) or (negcol):
        polyx = [x0[0], x0[-1], x0[-1], x0[0]]
        polyy = [x1[0], x1[-1], x1[0], x1[0]]
        f = sum(w*[polyx[2], polyy[2]])+b
        if (f>0) and (poscol):
            plt.fill(polyx, polyy, poscol, alpha=0.2)
        if (f<0) and (negcol):
            plt.fill(polyx, polyy, negcol, alpha=0.2)

        polyx = [x0[0], x0[-1], x0[0], x0[0]]
        polyy = [x1[0], x1[-1], x1[-1], x1[0]]
        f = sum(w*[polyx[2], polyy[2]])+b
        if (f>0) and (poscol):
```

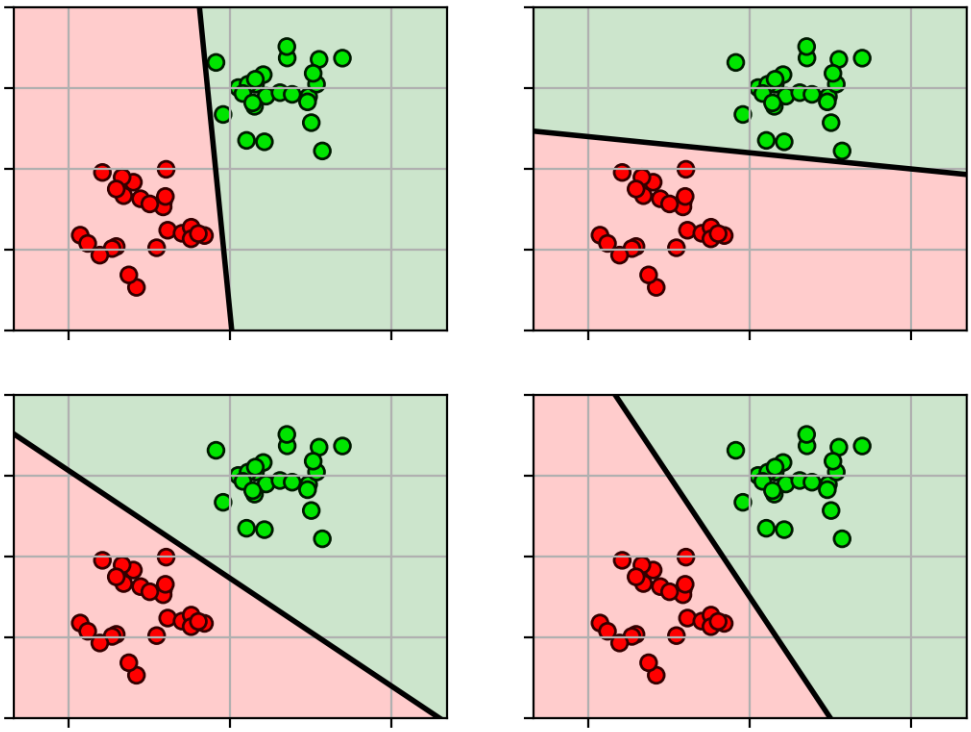
```
            plt.fill(polyx, polyy, poscol, alpha=0.2)
        if (f<0) and (negcol):
            plt.fill(polyx, polyy, negcol, alpha=0.2)

    # plot line
    plt.plot(x0, x1, ls, lw=lw, label=label)

    # the w
    if (wlabel):
        xp = array([0, -b/w[1]])
        xpw = xp+w
        plt.arrow(xp[0], xp[1], w[0], w[1], width=0.01)
        plt.text(xpw[0]-0.5, xpw[1], wlabel)
```

[5]:
```
seplinefig = plt.figure()
ws = array([[1,0.1], [0.1,1], [1,1.5], [1.5,1]])
bs = array([0.9, -1, 2, 2.5])

for i in range(len(bs)):
    plt.subplot(2,2,i+1)
    plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
    drawplane(ws[i,:], bs[i], poscol='g', negcol='r')
    plt.axis('equal'); plt.axis(axbox); plt.grid(True)
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])

plt.close()
```

[6]:
```
seplinefig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[6]:



7 Maximum margin

- Define the space between the separating line and the closest point as the *margin*.
  - think of this space as the "amount of wiggle room" for accommodating errors in estimating **w**.

[7]:
```
def drawmargin(w,b,xmarg=None, label=None):
    wnorm = w / sqrt(sum(w**2))
    if xmarg is None:
        # calculate a point on the margin
        dm = 1 / sqrt(sum(w**2))
        xpt  = array([1,-w[0]/w[1]-b/w[1]])
        # then find the margin (assuming learned with SVM)
        xmarg = xpt + dm*wnorm
        xmarg2 = xpt - dm*wnorm
    else:
        # find the distance to the margin
        dm = (sum(w*xmarg)+b) / sqrt(sum(w**2))
        # move to the other side of the decision plane
        xmarg2 = xmarg - 2*dm*wnorm

    drawplane(w, c=xmarg, ls='k--', lw=1, label=label)
```

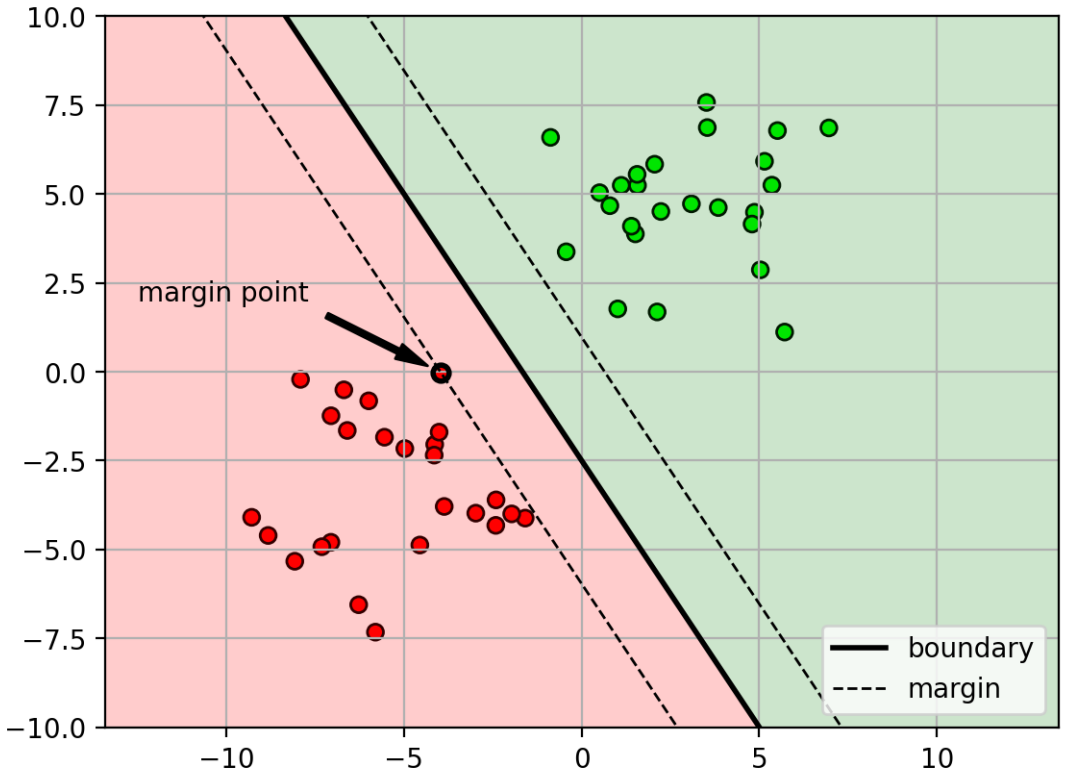
```
    drawplane(w, c=xmarg2, ls='k--', lw=1)
```

[8]:
```
w = array([1.5,1])
b = 2.5

# calculate distance to margin
d = abs(dot(X,w.T) + b)
mmi = argmin(d)
x0 = X[mmi,:]

margfig = plt.figure()
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
drawplane(w, b, poscol='g', negcol='r', label='boundary')
plt.plot(x0[0], x0[1], 'ko', fillstyle='none', markeredgewidth=2)
drawmargin(w,b,x0, label='margin')
plt.axis('equal'); plt.axis(axbox); plt.grid(True)
leg = plt.legend(loc='lower right' ,fontsize=10)
leg.get_frame().set_facecolor('white')
plt.annotate(text='margin point', xy=x0, xytext=(-12.5,2.0),
             arrowprops=dict(facecolor='black', width=2, shrink=0.1,␣
 ↪headwidth=6))
plt.close()
```

[9]:
```
margfig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[9]:



- **Idea:** the best separating line is the one that *maximizes the margin*.
  - i.e., puts the most distance between the closest points and the decision boundary.

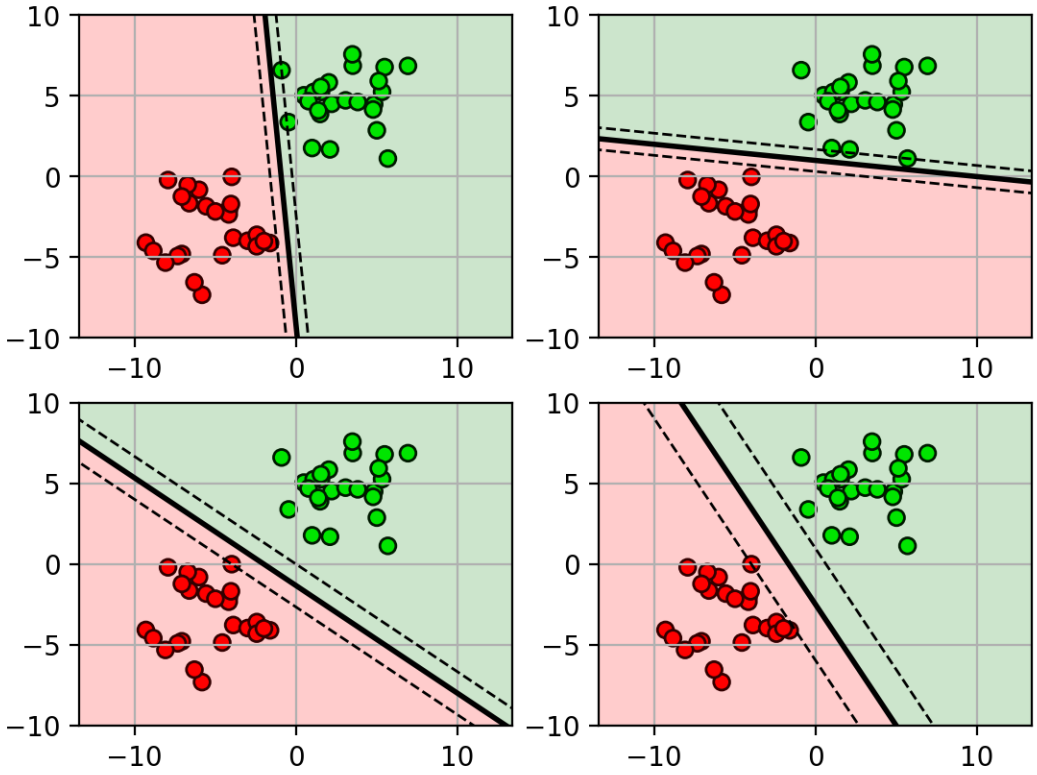
[10]:
```
margfigs = plt.figure()
for i in range(len(bs)):
    w = ws[i,:]
    b = bs[i]
    d = abs(dot(X,w.T) + b)
    mmi = argmin(d)
    x0 = X[mmi,:]

    plt.subplot(2,2,i+1)
    plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
    drawplane(w, b, poscol='g', negcol='r')
    drawmargin(w, b, x0)
    plt.axis('equal'); plt.axis(axbox); plt.grid(True)

plt.close()
```

[11]:
```
margfigs
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[11]:



- *the solution...*
  - by symmetry, there should be at least one margin point on each side of the boundary
    - if there is only one margin point, we could increase the margin distance by moving the boundary away from the margin point.
  - the points on the margins are called the **support vectors**
    - the points support (define) the margin

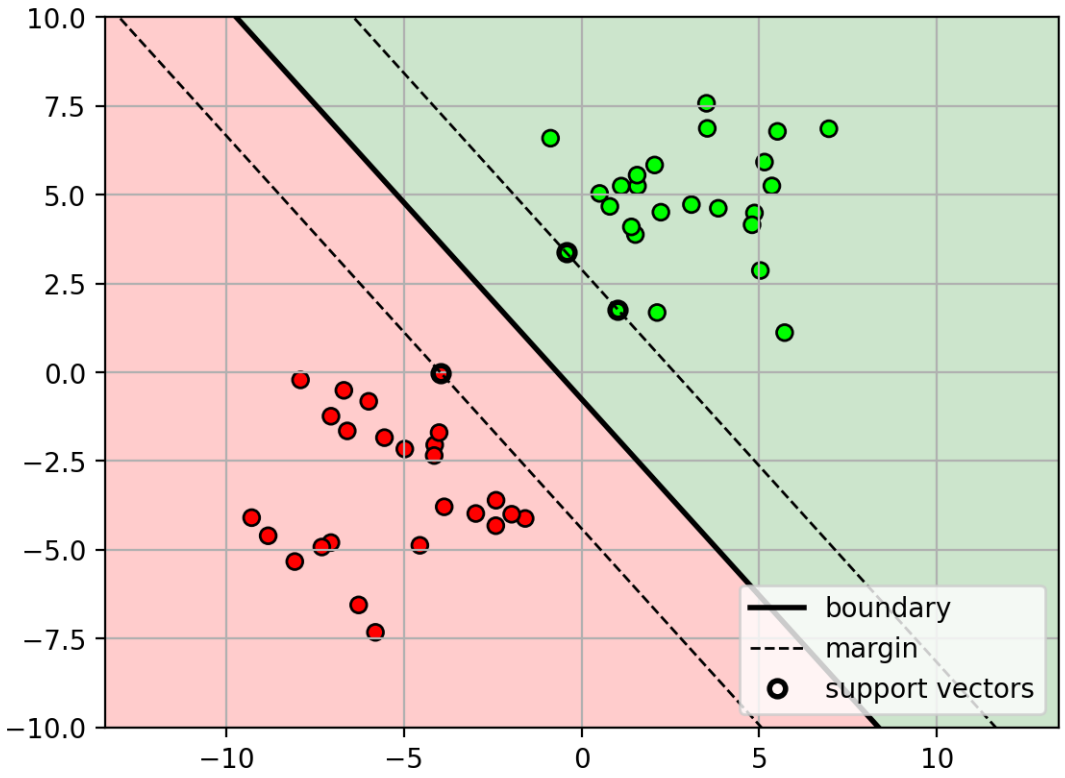
[12]:
```
# fit SVM (kernel is the type of decision surface...more in the next lecture)
clf = svm.SVC(kernel='linear', C=inf)
clf.fit(X, Y)

def plot_svm(clf, axbox, mycmap, showleg=True):
    # get line parameters
    w = clf.coef_[0]
    b = clf.intercept_[0]
```

```
    drawplane(w, b, poscol='g', negcol='r', label='boundary')
    drawmargin(w, b, label='margin')
    plt.plot(clf.support_vectors_[:,0], clf.support_vectors_[:,1],
         'ko',fillstyle='none', markeredgewidth=2, label='support vectors')
    plt.grid(True); plt.axis('equal'); plt.axis(axbox);
    if showleg:
        leg = plt.legend(loc='lower right', fontsize=10)
        leg.get_frame().set_facecolor('white')


maxmfig = plt.figure()
plot_svm(clf, axbox, mycmap)
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.close()
```

[13]:
```
maxmfig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[13]:



8 Why is maximizing the margin good?

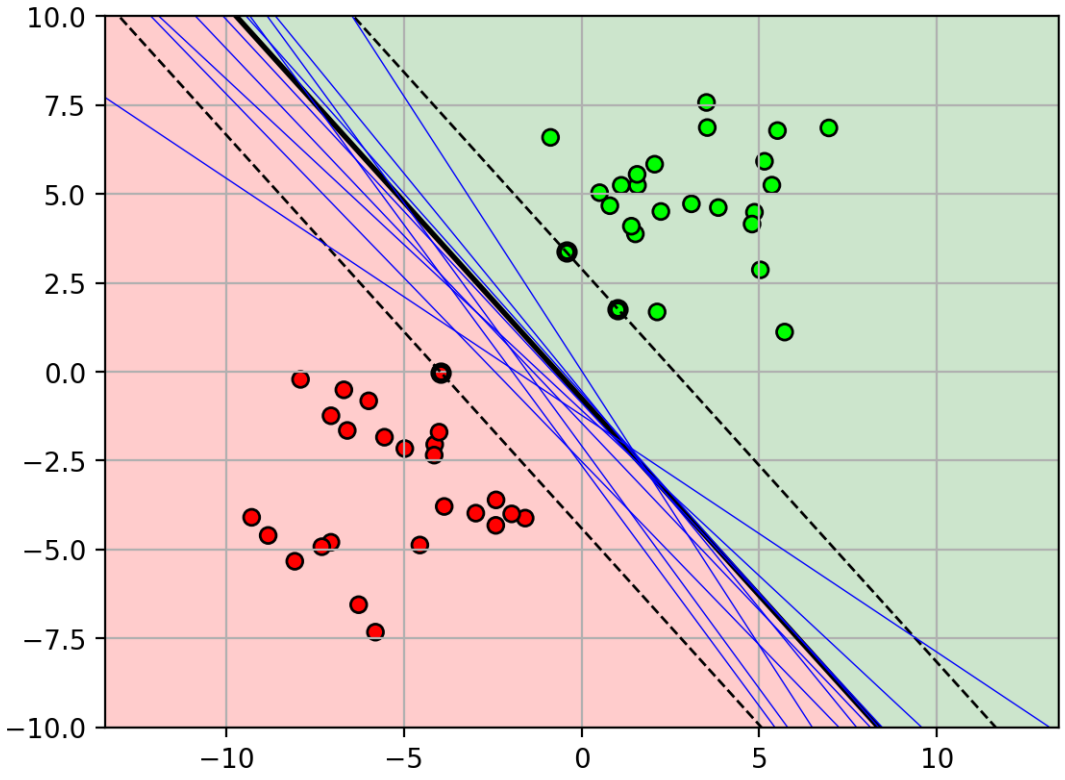
- the true **w** is uncertain
  - maximizing the margin allows the most uncertainty (wiggle room) for **w**, while keeping all the points correctly classified.

[14]:
```
maxmfigw = plt.figure()
plot_svm(clf, axbox, mycmap, showleg=False)
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')

w = clf.coef_[0]
b = clf.intercept_[0]
for i in range(10):
    w2 = w + 0.05*random.standard_normal(w.shape)
    b2 = b + 0.2*random.standard_normal(b.shape)
    drawplane(w2, b2, lw=0.5, ls='b-')
plt.close()
```

[15]:
```
maxmfigw
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[15]:



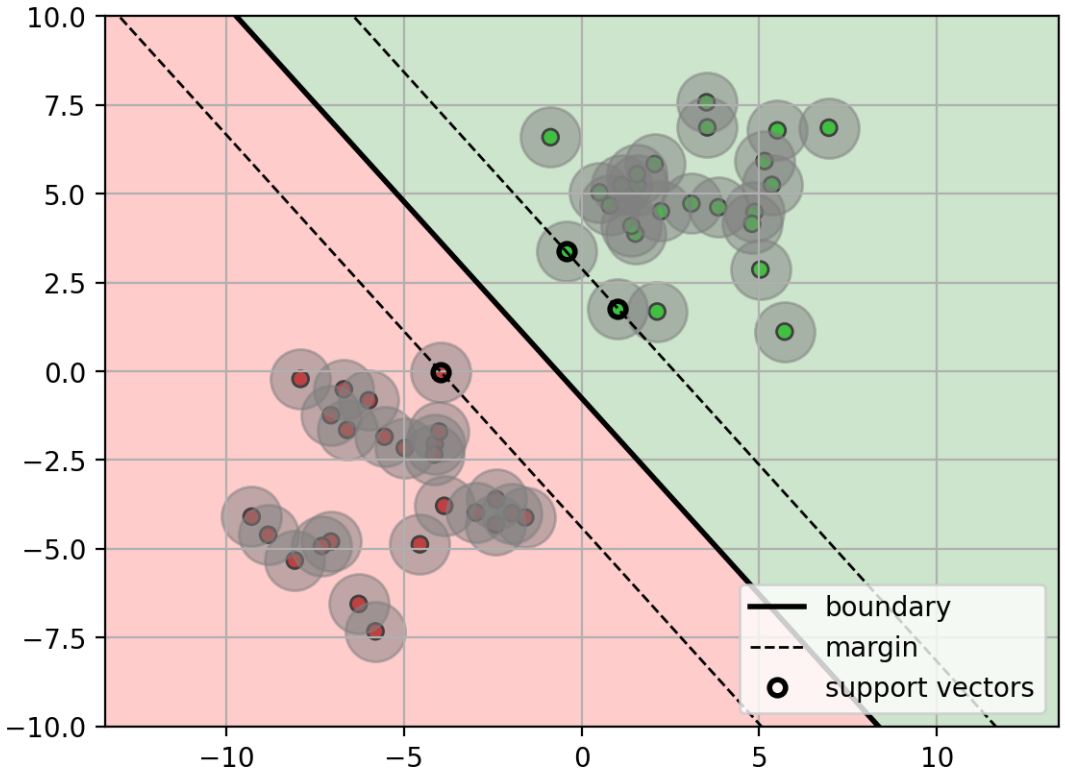
- the data points are uncertain
  - maximizing the margin allows the most wiggle of the points, while keeping all the points correctly classified.

[16]:
```
maxmfigp = plt.figure()
plot_svm(clf, axbox, mycmap)
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.scatter(X[:,0], X[:,1], s=500, alpha=0.5, color='gray')
plt.close()
```

[17]:
```
maxmfigp
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[17]:



9 SVM Training

- Given a training set $\{\mathbf{x}_i, y_i\}_{i=1}^N$.
- First define the margin distance:
  - Distance from a point $\mathbf{x}_i$ to hyperplane $\mathbf{w}$:

$$d_i = \frac{|f(\mathbf{x}_i)|}{||\mathbf{w}||}$$

– Margin distance is the minimum distance among all points:

$$\gamma = \min_i \frac{|f(\mathbf{x}_i)|}{||\mathbf{w}||}$$

[18]:
```
def draw_dist(X, w, b, justmargin=False):
    wu = sqrt(w@w)
    d = (X@w + b) / wu
    Xo = w*d[:,newaxis] / wu
    Xp = X-Xo
    #plt.scatter(Xp[:,0], Xp[:,1])
    if not justmargin:
        for xp,x in zip(Xp,X):
            plt.plot([xp[0], x[0]], [xp[1], x[1]], ':b', lw=0.5)
    mm = amin(abs(d))
    for i,myd in enumerate(abs(d)):
        if abs(myd-mm) <= .001*mm:
            plt.plot([Xp[i,0], X[i,0]], [Xp[i,1],X[i,1]], 'b-', label="margin␣
 ↪distance $\gamma$")
    #mmd = plt.plot([Xp[mmi,0], X[mmi,0]], [Xp[mmi,1],X[mmi,1]], 'b-',␣
 ↪label="margin distance $\gamma$")
```

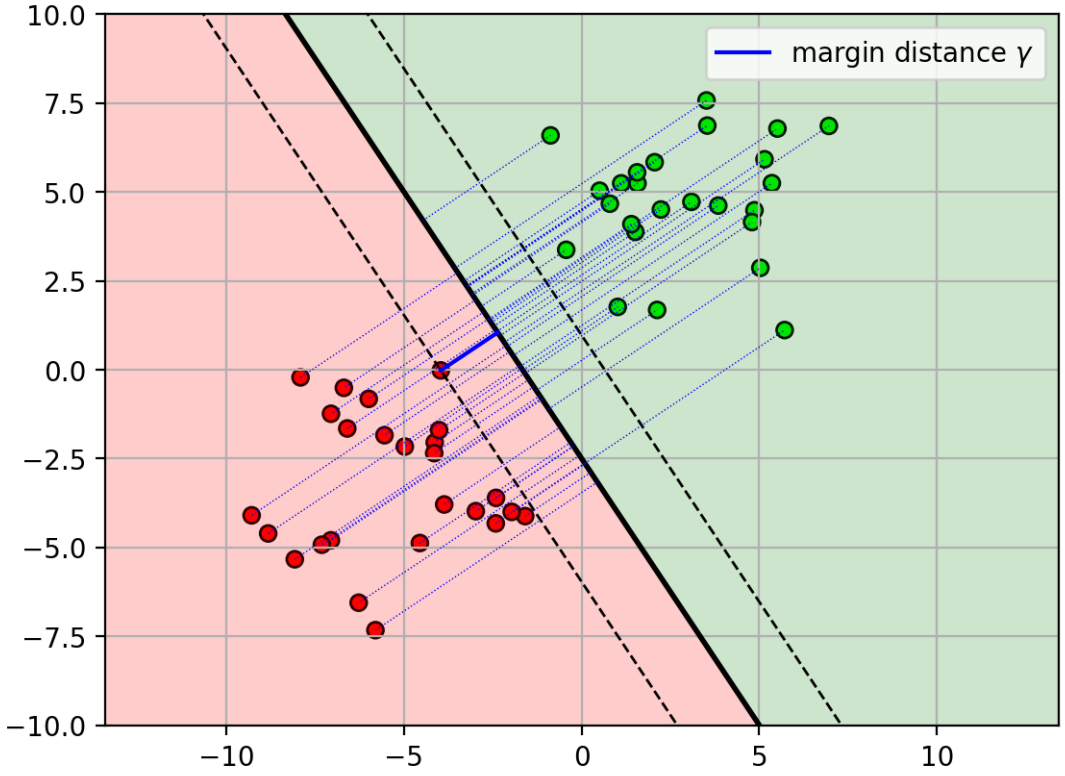
```
<>:13: SyntaxWarning: invalid escape sequence '\g'
<>:13: SyntaxWarning: invalid escape sequence '\g'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/3284564162.py:1
3: SyntaxWarning: invalid escape sequence '\g'
  plt.plot([Xp[i,0], X[i,0]], [Xp[i,1],X[i,1]], 'b-', label="margin distance
$\gamma$")
```

[19]:
```
dfig = plt.figure()
w = ws[3,:]
b = bs[3]
d = abs(dot(X,w.T) + b)
mmi = argmin(d)
x0 = X[mmi,:]

plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
drawplane(w, b, poscol='g', negcol='r')
drawmargin(w, b, x0)
draw_dist(X, w, b)
plt.grid(True)
plt.axis('equal')
plt.axis(axbox);
plt.legend()

plt.close()
```

[20]:
```
dfig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[20]:



- The hyperplane $\mathbf{w}$ appears in both numerator and denominator!
  - $\gamma = \min_i \frac{|f(\mathbf{x}_i)|}{||\mathbf{w}||} = \min_i \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{||\mathbf{w}||}$
- Changing the length of $\mathbf{w}$ won't affect the margin distance.
  - $\hat{\mathbf{w}} = a\mathbf{w}$, $\hat{b} = ab$
  - $\frac{|\hat{\mathbf{w}}^T\mathbf{x}_i + \hat{b}|}{||\hat{\mathbf{w}}||} = \frac{|a\mathbf{w}^T\mathbf{x}_i + ab|}{||a\mathbf{w}||} = \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{||\mathbf{w}||}$
- Margin distance is determined by the direction of $\mathbf{w}$, not the length.

10 Normalization

- Since the length of $\mathbf{w}$ doesn't matter, we can assume a normalization for $\mathbf{w}$.
- Two possibilities:
  1. set denominator to 1: $||\mathbf{w}|| = 1$
  - $||\mathbf{w}||$ is a unit-norm vector
  2. set numerator to 1: $\min_i |f(\mathbf{x}_i)| = 1$
  - the point $\mathbf{x}_i$ on the margin has $f(\mathbf{x}_i) = 1$.
- Which is better?

11 SVM Optimization Problem

- Choose the 2nd option.
  - constraint: $\min_i |f(\mathbf{x}_i)| = 1$
  - margin: $\gamma = \frac{1}{||\mathbf{w}||}$
- Maximize the margin:

$$(\hat{\mathbf{w}}, b) = \underset{\mathbf{w},b}{\text{argmax}} \frac{1}{||\mathbf{w}||} \quad \text{s.t. } \min_i |f(\mathbf{x}_i)| = 1$$

- Invert the objective to turn into a minimization problem

$$(\hat{\mathbf{w}}, b) = \underset{\mathbf{w},b}{\text{argmin}} \frac{1}{2}||\mathbf{w}||^2 \quad \text{s.t. } \min_i |f(\mathbf{x}_i)| = 1$$

- Note: if the points are correctly classified…
  - for points in class $y_i = 1$, then $f(\mathbf{x}_i) > 0$.
  - for points in class $y_i = -1$, then $f(\mathbf{x}_i) < 0$.
- Thus, the constraint: $\min_i |f(\mathbf{x}_i)| = 1$,
  - can be rewritten: $\min_i y_i f(\mathbf{x}_i) = 1$
- Closer look:
  - original constraint: $\min_i y_i f(\mathbf{x}_i) = 1$
    - the minimum over all i is 1.
  - equivalent constraint: $y_i f(\mathbf{x}_i) \geq 1, \forall i$
    - suppose $y_i f(\mathbf{x}_i) = y_i(\mathbf{w}^T\mathbf{x}_i + b) > 1, \forall i$.
    - since we are minimizing $||\mathbf{w}||$, $\mathbf{w}$ will shrink so that at least one $\mathbf{x}_i$ will have $y_i f(\mathbf{x}_i) = 1$.
      - $(\mathbf{w}, b) \Rightarrow (\delta\mathbf{w}, \delta b)$, where $y_i(\delta\mathbf{w}^T\mathbf{x}_i + \delta b) \geq 1, \forall i$

12 SVM Training Objective

- given a training set $\{\mathbf{x}_i, y_i\}_{i=1}^N$, optimize:

$$\underset{\mathbf{w},b}{\text{argmin}} \frac{1}{2}\mathbf{w}^T\mathbf{w}$$
$$\text{s.t. } y_i f(\mathbf{x}_i) \geq 1, \quad \forall i$$

  - the objective minimizes the inverse of the margin distance, i.e., maximizes the margin.
  - the inequality constraints ensure that all points are either on or outside of the margin.
    - a point on the margin has $y_i f(\mathbf{x}_i) = 1$.
    - also written as $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$.

13 Review of Constrained Optimization

- Consider an optimization problem with inequality constraints:

$$\min_{\mathbf{x}} f(\mathbf{x}) \text{ s.t. } g(\mathbf{x}) \geq 0$$

  - $f(\mathbf{x})$ is the objective function (for SVM, it's the inverse margin).

  - $g(\mathbf{x})$ is the constraint function (for SVM, it's the margin constraint).

- Use Langrange multipliers to solve this problem:
  - introduce Lagrange multiplier: $\lambda \geq 0$
  - form the Lagrangian: $L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda g(\mathbf{x})$
  - find the stationary point $(\mathbf{x}^*, \lambda^*)$ of the Lagrangian
    - find solution of $\frac{dL}{d\mathbf{x}} = 0$ and $\frac{dL}{d\lambda} = 0$.
  - at the solution, the Langrange multiplier indicates the mode of the inequality constraint
    - when $\lambda^* > 0$, then $g(\mathbf{x}^*) = 0$ (called "active equality").
    - when $\lambda^* = 0$, then $g(\mathbf{x}^*) > 0$ (called "inactive").

14 Duality

- We can rewrite the original (primal) problem into its dual form:
  - dual function: $q(\lambda) = \min_{\mathbf{x}} L(\mathbf{x}, \lambda)$
  - dual problem: $\max_{\lambda \geq 0} q(\lambda)$
- Solve for λ, rather than original variable $\mathbf{x}$.
  - can recover the value of $\mathbf{x}$ from λ.

- If the optimization problem is convex…
  - solving the dual is equivalent to solving the **primal**
  - $\min_{\mathbf{x}, g(\mathbf{x}) \geq 0} f(\mathbf{x}) = \max_{\lambda \geq 0} q(\lambda)$.
- **Note:** The SVM problem is convex, so we can obtain an equivalent dual problem.

15 Lagrange multipliers & SVM

- introduce a Langrange multiplier α_i for each constrain

$$L(\mathbf{w}, \alpha) = \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_i \alpha_i [y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1]$$

- Lagrange multiplier tells us which points are on the margin:
  - If $\alpha_i = 0$, then $y_i(\mathbf{w}^T\mathbf{x}_i + b) > 1$ ($\mathbf{x}_i$ is beyond margin).
  - If $\alpha_i > 0$, then $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ ($\mathbf{x}_i$ is on the margin).
    - i.e., the point is a *support vector*.

16 SVM Dual Problem

- The SVM problem can be rewritten as a *dual* problem:

$$\underset{\alpha}{\text{argmax}} \sum_i \alpha_i - \frac{1}{2}\sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad \text{s.t.} \sum_{i=1}^N \alpha_i y_i = 0, \quad \alpha_i \geq 0$$

- The new variable α_i corresponds to each training sample $(\mathbf{x}_i, y_i)$.
  - instead of solving for $\mathbf{w}$, we now solve for

- Recover the hyperplane $\mathbf{w}$ using α:
  - weighted combination of (transformed) data points.
  - $\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$

- Classify a new point $\mathbf{x}_*$,

$$y_* = \text{sign}(\mathbf{w}^T\mathbf{x}_* + b) \tag{1}$$

$$= \text{sign}(\sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i^T \mathbf{x}_* + b) \tag{2}$$

- Interpretation of α_i
  - $\alpha_i = 0$ when the sample $\mathbf{x}_i$ is not on the margin.
  - $\alpha_i > 0$ when the sample $\mathbf{x}_i$ is on the margin (or violates).
    - i.e., the sample $\mathbf{x}_i$ is a support vector.
    - $y_i\alpha_i > 0$ margin sample for positive class
    - $y_i\alpha_i < 0$ margin sample for negative class.

17 SVM Prediction

- given a new data point $\mathbf{x}_*$, use sign of linear function to predict class
  - $y_* = \text{sign}\, f(\mathbf{x}_*) = \text{sign}(\mathbf{w}^T\mathbf{x}_* + b)$

18 Example

- Regions defined by $f(\mathbf{x})$:
  - $f(\mathbf{x}) = 0$ – decision boundary
  - $f(\mathbf{x}) = \pm 1$ – positive and negative margins
  - $f(\mathbf{x}) > 1$ – points correctly classified as class 1
  - $f(\mathbf{x}) < -1$ – points correctly classified as class -1

[21]:
```
# fit SVM (kernel is the type of decision surface...more in the next lecture)
clf = svm.SVC(kernel='linear', C=inf)
clf.fit(X, Y)

egsvmfig = plt.figure()
plot_svm(clf, axbox, mycmap)
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
draw_dist(X, clf.coef_[0], clf.intercept_[0], justmargin=True)
plt.text(-9.5,7.5, '$f(\mathbf{x})=0$', fontsize=9,␣
 ↪bbox=dict(facecolor='white', alpha=0.8, edgecolor='white'))
plt.text(-5,7.5, '$f(\mathbf{x})=1$', fontsize=9, bbox=dict(facecolor='white',␣
 ↪alpha=0.8, edgecolor='white'))
plt.text(-12,5, '$f(\mathbf{x})=-1$', fontsize=9, bbox=dict(facecolor='white',␣
 ↪alpha=0.8, edgecolor='white'))
plt.text(7,5, '$f(\mathbf{x})>1$', fontsize=9, bbox=dict(facecolor='white',␣
 ↪alpha=0.8, edgecolor='white'))
plt.text(-12,-2.5, '$f(\mathbf{x})<-1$', fontsize=9,␣
 ↪bbox=dict(facecolor='white', alpha=0.8, edgecolor='white'))
plt.annotate(text='$\gamma = 1/||\mathbf{w}||$', xy=(0,1), xytext=(2.5,-2.0),
             backgroundcolor='white',
```

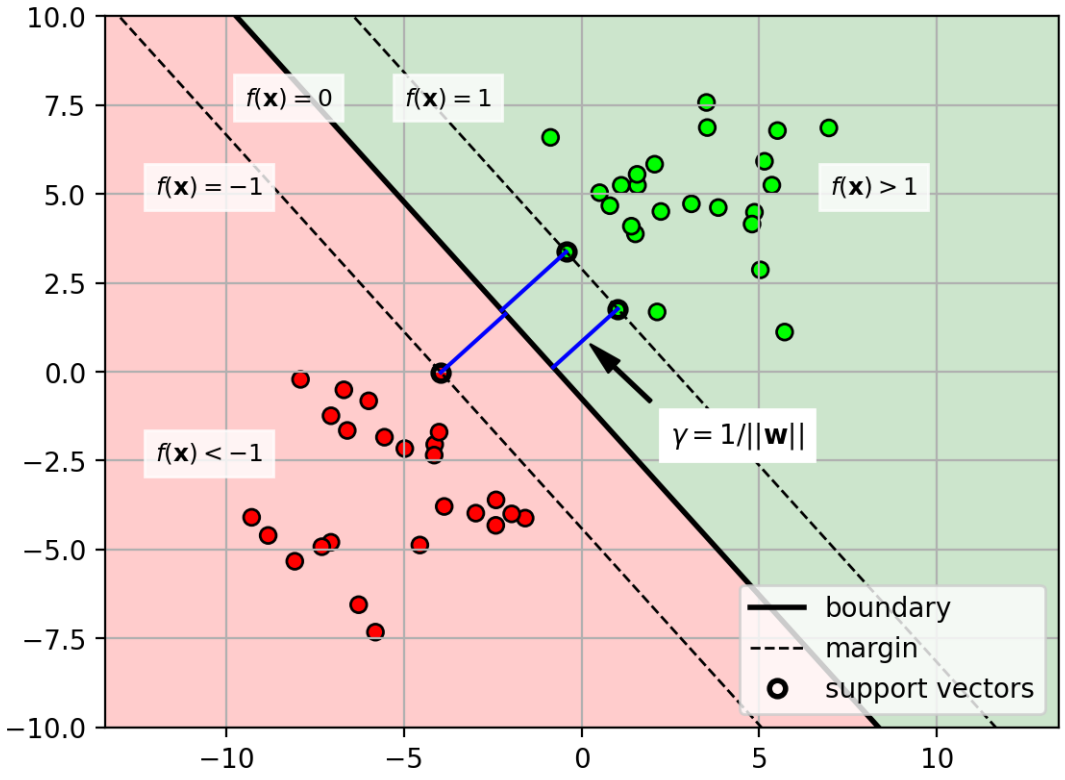
```
            arrowprops=dict(facecolor='black', width=1, shrink=0.1,␣
 ↪headwidth=6))
plt.close()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\m'
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:11: SyntaxWarning: invalid escape sequence '\m'
<>:12: SyntaxWarning: invalid escape sequence '\m'
<>:13: SyntaxWarning: invalid escape sequence '\m'
<>:14: SyntaxWarning: invalid escape sequence '\g'
<>:9: SyntaxWarning: invalid escape sequence '\m'
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:11: SyntaxWarning: invalid escape sequence '\m'
<>:12: SyntaxWarning: invalid escape sequence '\m'
<>:13: SyntaxWarning: invalid escape sequence '\m'
<>:14: SyntaxWarning: invalid escape sequence '\g'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/2869223478.py:9
: SyntaxWarning: invalid escape sequence '\m'
  plt.text(-9.5,7.5, '$f(\mathbf{x})=0$', fontsize=9,
bbox=dict(facecolor='white', alpha=0.8, edgecolor='white'))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/2869223478.py:1
0: SyntaxWarning: invalid escape sequence '\m'
  plt.text(-5,7.5, '$f(\mathbf{x})=1$', fontsize=9, bbox=dict(facecolor='white',
alpha=0.8, edgecolor='white'))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/2869223478.py:1
1: SyntaxWarning: invalid escape sequence '\m'
  plt.text(-12,5, '$f(\mathbf{x})=-1$', fontsize=9, bbox=dict(facecolor='white',
alpha=0.8, edgecolor='white'))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/2869223478.py:1
2: SyntaxWarning: invalid escape sequence '\m'
  plt.text(7,5, '$f(\mathbf{x})>1$', fontsize=9, bbox=dict(facecolor='white',
alpha=0.8, edgecolor='white'))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/2869223478.py:1
3: SyntaxWarning: invalid escape sequence '\m'
  plt.text(-12,-2.5, '$f(\mathbf{x})<-1$', fontsize=9,
bbox=dict(facecolor='white', alpha=0.8, edgecolor='white'))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/2869223478.py:1
4: SyntaxWarning: invalid escape sequence '\g'
  plt.annotate(text='$\gamma = 1/||\mathbf{w}||$', xy=(0,1), xytext=(2.5,-2.0),
```

[22]:
```
egsvmfig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[22]:



19 What about non-separable data?

- use the same linear classifier
  - allow some training samples to **violate** the margin
    - i.e., are inside the margin (or even mis-classified)
- Define "slack" variable $\xi_i \geq 0$
  - $\xi_i = 0$ means sample is outside of margin area (no slack)
  - $\xi_i > 0$ means sample is inside of margin area (slack)

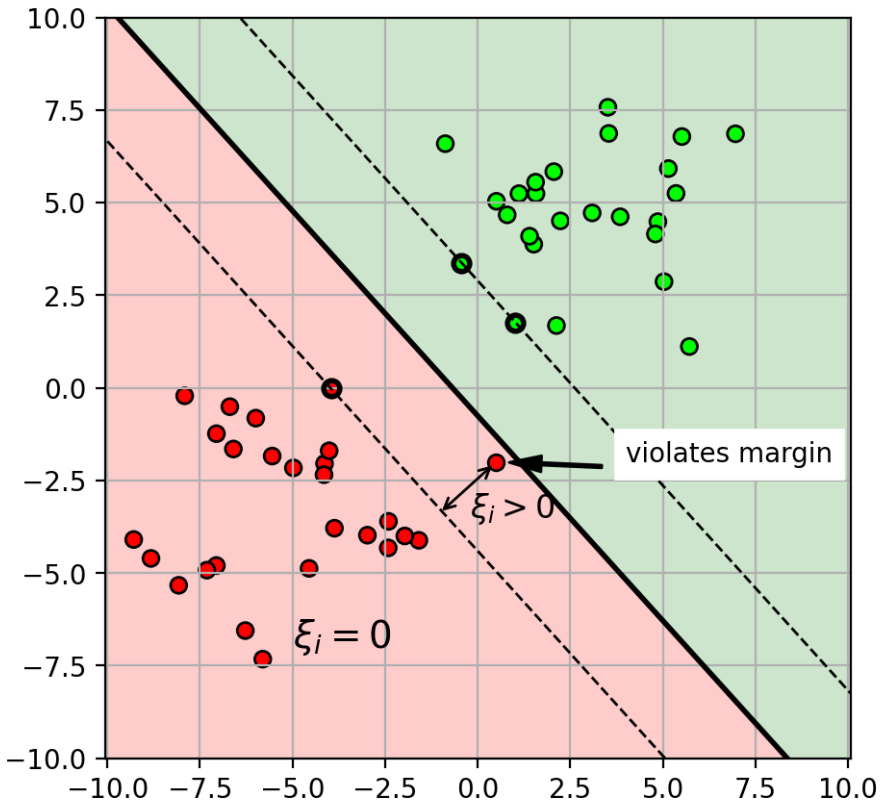
[23]:
```
slackfig = plt.figure(figsize=(5,5))
plot_svm(clf, axbox, mycmap, showleg=False)
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')

xv = array([0.5, -2])
xm = xv-clf.coef_.ravel()*5.2

plt.scatter([xv[0]], [xv[1]], c=[1], cmap=mycmap, edgecolors='k')
# slack point
plt.annotate(text='violates margin', xy=xv, xytext=(4.0,-2.0),
             backgroundcolor='white',
```

```
            arrowprops=dict(facecolor='black', width=1, shrink=0.1,␣
 ↪headwidth=6))

plt.annotate(text="", xy=xv, xytext=xm,
             arrowprops=dict(arrowstyle="<->"))
plt.text(-0.2, -3.5, "$\\xi_i>0$", fontsize=12)
plt.text(-5, -7, "$\\xi_i=0$", fontsize=14)
plt.close()
```

[24]:
```
slackfig
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[24]:



- constraint now includes slack variable
  - $y_i f(\mathbf{x}_i) \geq 1 - \xi_i, \forall i$
- Two possibilities:
  - $\xi_i = 0$, then sample $\mathbf{x}_i$ has normal margin constraint: $y_i f(\mathbf{x}_i) \geq 1$
  - $\xi_i > 0$, then sample $\mathbf{x}_i$ is inside margin: $y_i f(\mathbf{x}_i) = 1 - \xi_i$
    - Note that: $y_i f(\mathbf{x}_i) < 1$, inside margin.

20 Soft-SVM optimization problem

- Penalize each training sample that violates the margin by summing over ξ_i.
  - penalty controlled by hyperparameter C.
    - smaller value means allow more violations (less penalty)
    - larger value means don't allow violations (more penalty)

$$\begin{aligned} \underset{\mathbf{w},b}{\operatorname{argmin}} & \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}\xi_i \\ \text{s.t. } & y_i f(\mathbf{x}_i) \geq 1 - \xi_i, \quad \forall i \\ & \xi_i \geq 0 \end{aligned}$$

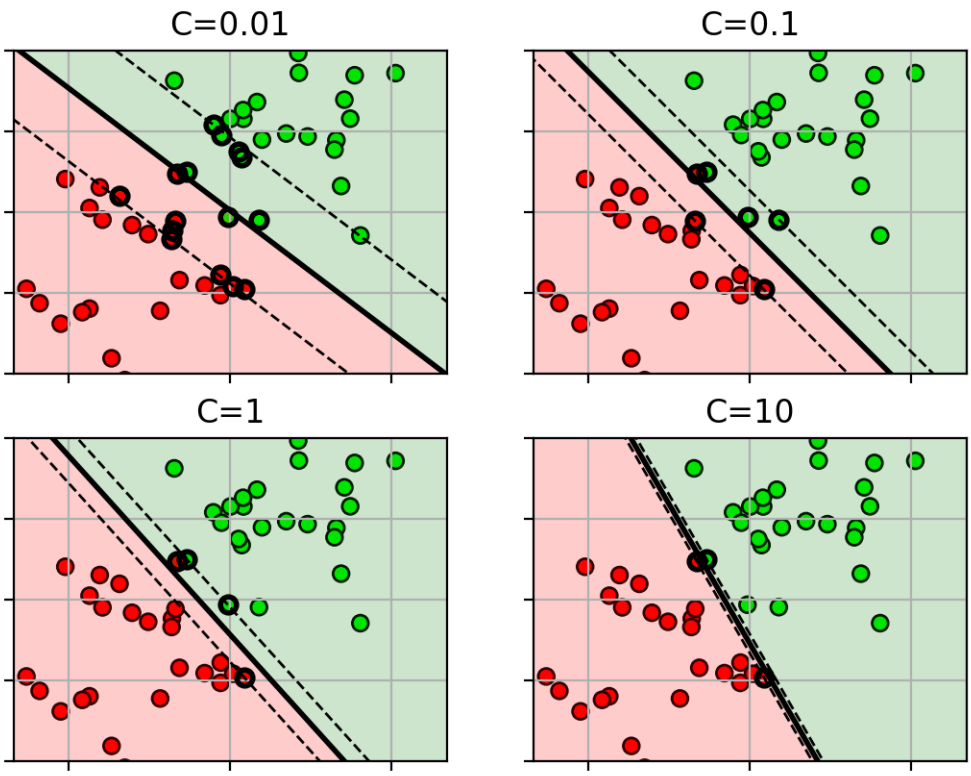
[25]:
```
# generate non-separable random data
X,Y = datasets.make_blobs(n_samples=50,
         centers=2, cluster_std=3.5, n_features=2,
         center_box=(-5,5), random_state=4487)

Cs = [0.01, 0.1, 1, 10]
Cmargfigs = plt.figure()

for i in range(len(Cs)):
    # fit SVM
    clf = svm.SVC(kernel='linear', C=Cs[i])
    clf.fit(X, Y)

    plt.subplot(2,2,i+1)
    plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
    plot_svm(clf, axbox, mycmap, showleg=False)
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    plt.title("C="+str(Cs[i]))

plt.close()
```

- Example with different C.

[26]:
```
Cmargfigs
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[26]:



21 Loss function

- After some massaging, the objective function is:

$$\underset{\mathbf{w},b}{\operatorname{argmin}} \frac{1}{C}\mathbf{w}^T\mathbf{w} + \sum_{i=1}^{N} \max(0, 1 - y_i f(\mathbf{x}_i))$$

  - hinge loss function: $L(z_i) = \max(0, 1 - z_i)$
    - Note: $\max(a, b)$ returns whichever value (a or b) is largest.

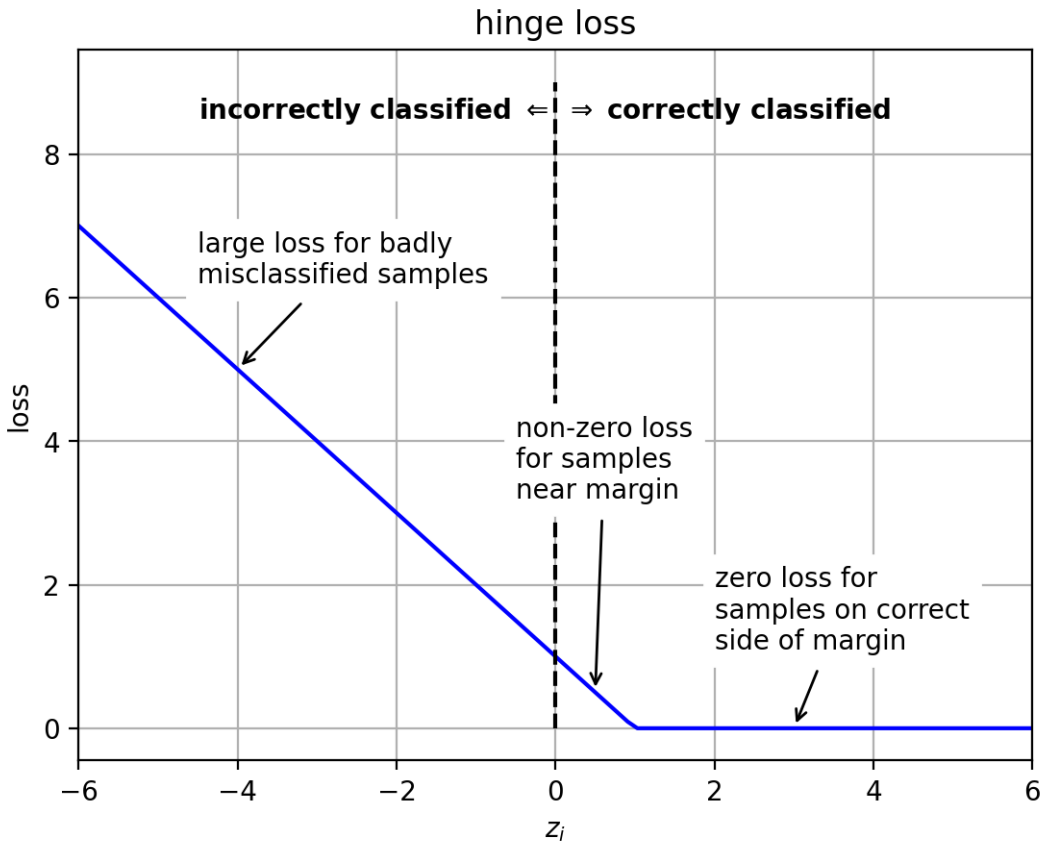
[27]:
```
z = linspace(-6,6,100)
logloss = log(1+exp(-z))
hingeloss = maximum(0, 1-z)
lossfig = plt.figure()
plt.plot(z,hingeloss, 'b-')

plt.plot([0,0], [0,9], 'k--')
plt.text(0,8.5, "incorrectly classified $\\Leftarrow$ ", ha='right',␣
 ↪weight='bold')
plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left', weight='bold')

plt.annotate(text="large loss for badly\nmisclassified samples",
```

```
             xy=(-4,5), xytext=(-4.5,6.2),backgroundcolor='white',
            arrowprops=dict(arrowstyle="->"))
plt.annotate(text="non-zero loss\nfor samples\nnear margin",
             xy=(0.5,0.5), xytext=(-0.5,3.2), backgroundcolor='white',
            arrowprops=dict(arrowstyle="->"))
plt.annotate(text="zero loss for\nsamples on correct\nside of margin",
             xy=(3,0.0), xytext=(2,1.1), backgroundcolor='white',
            arrowprops=dict(arrowstyle="->"))
plt.grid(True)
plt.xlabel('$z_i$');
plt.ylabel('loss')
plt.title('hinge loss')
plt.xlim(-6,6)
plt.close()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\R'
<>:9: SyntaxWarning: invalid escape sequence '\R'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/370200353.py:9:
SyntaxWarning: invalid escape sequence '\R'
  plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left',
weight='bold')
```

[28]:
```
lossfig
```

[28]:



22 Example: Iris Data

[29]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris2.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (versicolor=1,␣
 ↪virginica=2)

print(X.shape)
```

```
(100, 2)
```

[30]:
```
# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(50, 2)
(50, 2)
```

[31]:
```
# fit the SVM using all the data and the best C
clf = svm.SVC(kernel='linear', C=2)
clf.fit(trainX, trainY)

# get line parameters
w = clf.coef_[0]
b = clf.intercept_[0]
print(w)
print(b)
```

```
[2.87200943 0.10399865]
-13.95923965217775
```

[32]:
```
# indices of data points that are support vectors (on or inside the margin)
clf.support_
```

[32]:
```
array([ 0,  7, 12, 31, 36, 41, 13, 20, 22, 25, 33, 46], dtype=int32)
```

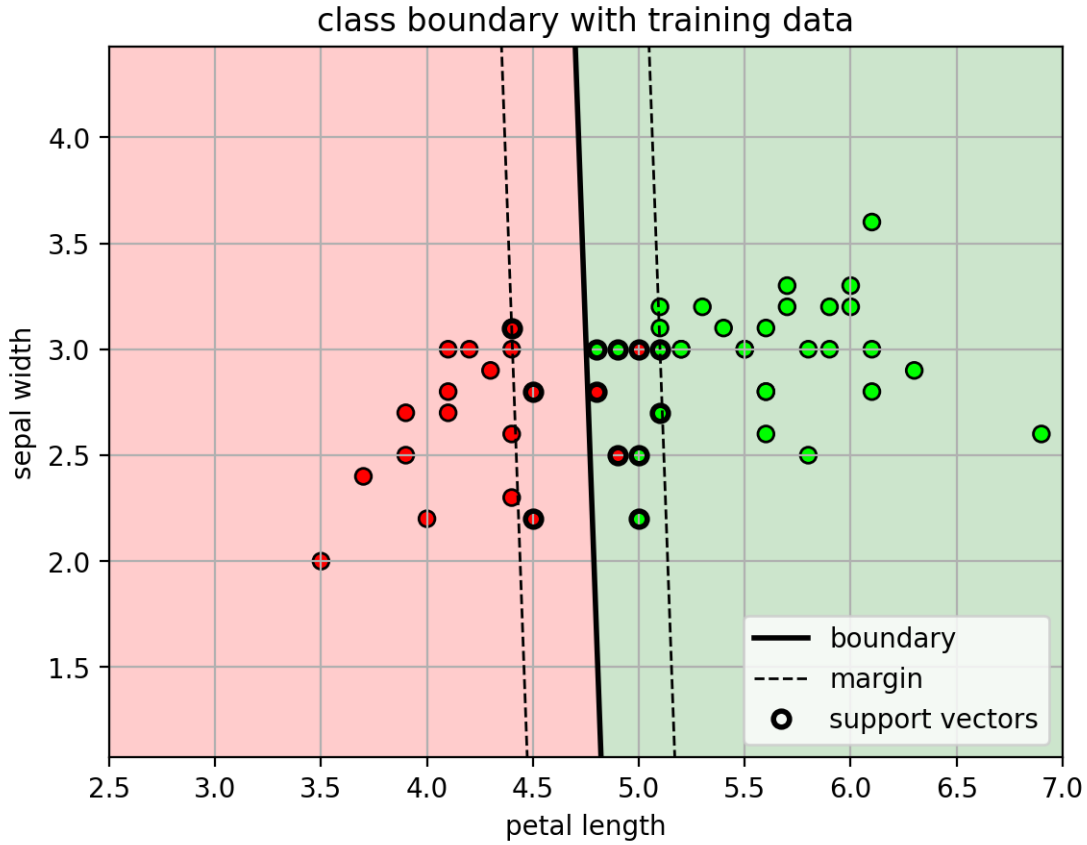
[33]:
```
axbox = [2.5, 7, 1.5, 4]
svmfig = plt.figure()
```

```
plot_svm(clf, axbox, mycmap)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.title('class boundary with training data');
plt.close()
```

[34]:
```
svmfig
```

```
Ignoring fixed y limits to fulfill fixed data aspect with adjustable data
limits.
```

[34]:



- SVM doesn't have it's own dedicated cross-validation function
- Use the `GridSearchCV` to run cross-validation for a list of parameters
  - calculate average accuracy for each parameter
  - select parameter with average highest accuracy, retrain model with all data
  - Speed up: each parameter can be trained/tested separately, specify number of parallel jobs using `n_jobs`

[35]:
```
# setup the list of parameters to try
paramgrid = {'C': logspace(-3,3,13)}
print(paramgrid)
```

```
# setup the cross-validation object
# pass the SVM object, parameter grid, and number of CV folds
# set number of parallel jobs to -1 (use all cores)
svmcv = model_selection.GridSearchCV(svm.SVC(kernel='linear'), paramgrid, cv=5,
                                     n_jobs=-1, verbose=True)

# run cross-validation (train for each split)
svmcv.fit(trainX, trainY);
```

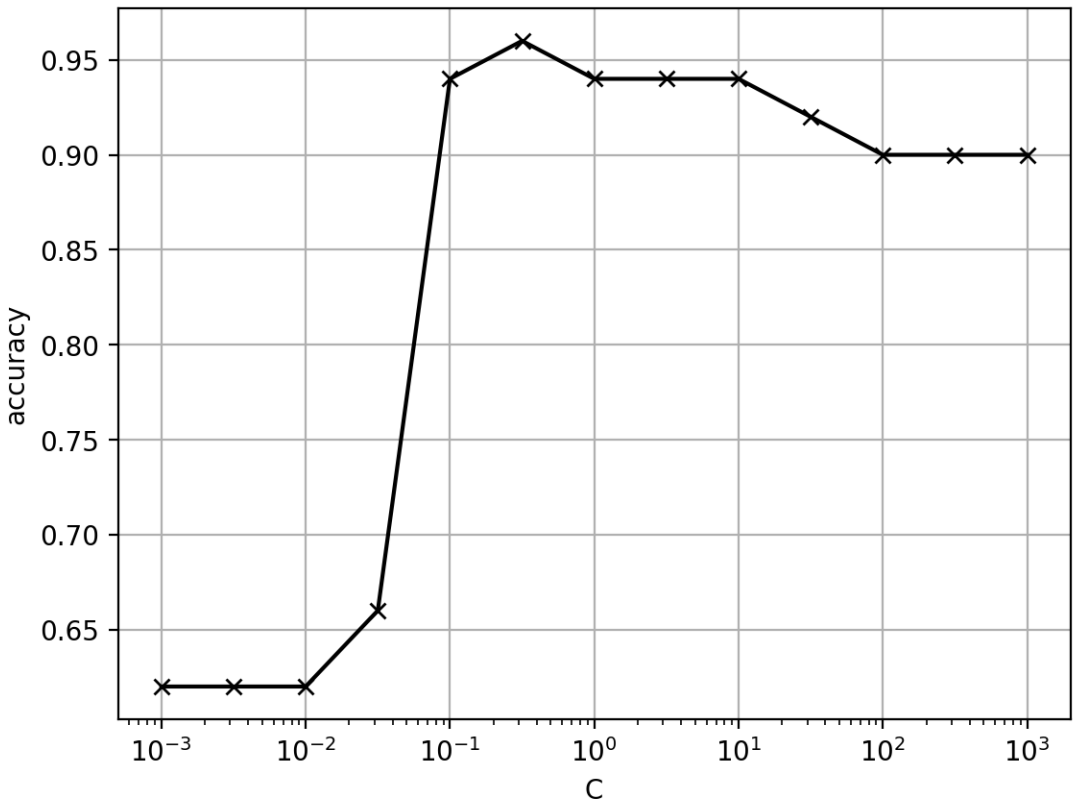
```
{'C': array([1.00000000e-03, 3.16227766e-03, 1.00000000e-02, 3.16227766e-02,
       1.00000000e-01, 3.16227766e-01, 1.00000000e+00, 3.16227766e+00,
       1.00000000e+01, 3.16227766e+01, 1.00000000e+02, 3.16227766e+02,
       1.00000000e+03])}
Fitting 5 folds for each of 13 candidates, totalling 65 fits
```

[36]:
```
# show the test error for each parameter set
for m,p in zip(svmcv.cv_results_['mean_test_score'], svmcv.
  ↪cv_results_['params']):
    print("mean={:.4f} {}".format(m,p))
```

```
mean=0.6200 {'C': np.float64(0.001)}
mean=0.6200 {'C': np.float64(0.0031622776601683794)}
mean=0.6200 {'C': np.float64(0.01)}
mean=0.6600 {'C': np.float64(0.03162277660168379)}
mean=0.9400 {'C': np.float64(0.1)}
mean=0.9600 {'C': np.float64(0.31622776601683794)}
mean=0.9400 {'C': np.float64(1.0)}
mean=0.9400 {'C': np.float64(3.1622776601683795)}
mean=0.9400 {'C': np.float64(10.0)}
mean=0.9200 {'C': np.float64(31.622776601683793)}
mean=0.9000 {'C': np.float64(100.0)}
mean=0.9000 {'C': np.float64(316.22776601683796)}
mean=0.9000 {'C': np.float64(1000.0)}
```

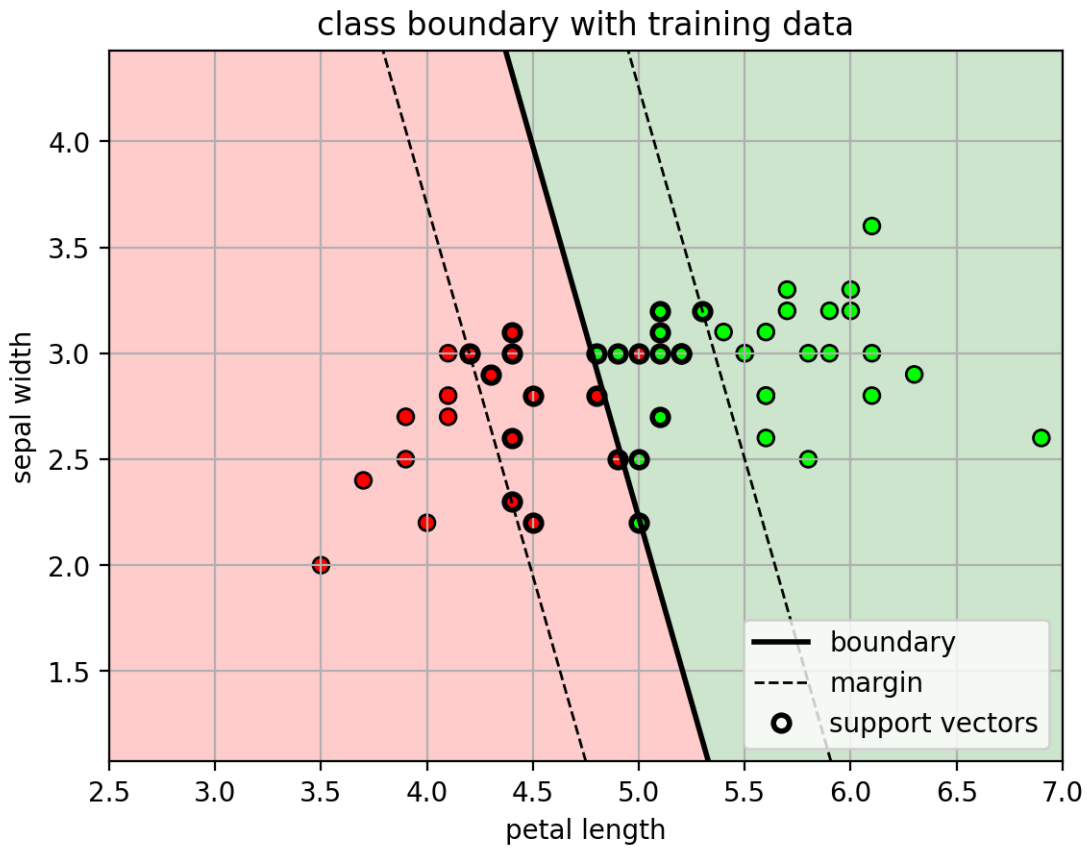
[37]:
```
# make a plot
allC      = [p['C'] for p in svmcv.cv_results_['params']]
allscores = svmcv.cv_results_['mean_test_score']

plt.figure()
plt.semilogx(allC, allscores, 'kx-')
plt.xlabel('C'); plt.ylabel('accuracy')
plt.grid(True)

# view best results and best retrained estimator
print(svmcv.best_params_)
print(svmcv.best_score_)
print(svmcv.best_estimator_)
```

```
{'C': np.float64(0.31622776601683794)}
0.96
SVC(C=np.float64(0.31622776601683794), kernel='linear')
```



[38]:
```
plt.figure()
plot_svm(svmcv.best_estimator_, axbox, mycmap)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.title('class boundary with training data');
```

```
Ignoring fixed y limits to fulfill fixed data aspect with adjustable data
limits.
```



[39]:
```
# Directly use svmcv to make predictions
predY = svmcv.predict(testX)

acc = metrics.accuracy_score(testY, predY)
print("test accuracy = " + str(acc))
```

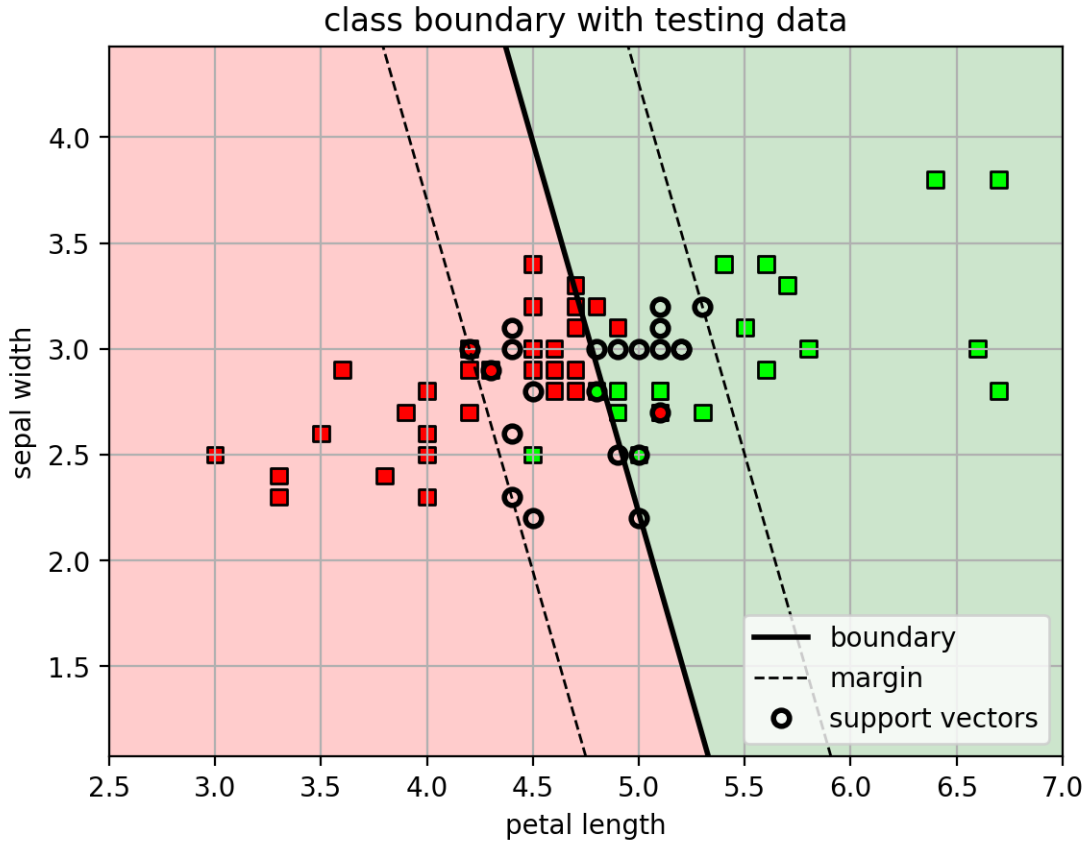
```
test accuracy = 0.88
```

[40]:
```
# Plot test data
tsvmfig = plt.figure()
plot_svm(svmcv.best_estimator_, axbox, mycmap)
plt.scatter(testX[:,0], testX[:,1], c=testY, cmap=mycmap, marker='s',␣
 ↪edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.title('class boundary with testing data');
plt.close()
```

[41]:
```
tsvmfig
```

```
Ignoring fixed y limits to fulfill fixed data aspect with adjustable data
limits.
```

[41]:



23 Multi-class SVM

- In sklearn, `svm.SVC` implements "1-vs-1" multi-class classification.
  - Train binary classifiers on all pairs of classes.
    - 3-class Example: 1 vs 2, 1 vs 3, 2 vs 3
  - To label a sample, pick the class with the most votes among the binary classifiers.
- Problem:
  - 1v1 classification is very slow when there are a large number of classes.
    - if there are C classes, need to train $C(C-1)/2$ binary classifiers!

24 1-vs-all SVM

- Use the `multiclass.OneVsRestClassifier` to build a 1-vs-all classifier from any binary classifier.
  - pass it the binary classifier as the base classifier.

- For `GridSearchCV`, the binary SVM is embedded inside the 1-vs-all wrapper class.
  - use `'estimator__C'` as the parameter label for `C` in the SVM.
  - notation `A__B` means the cross-validated parameter `B` is nested in parameter `A`.

[42]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris3.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (setosa=0,␣
 ↪versicolor=1, virginica=2)

# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(75, 2)
(75, 2)
```

[43]:
```
msvm = multiclass.OneVsRestClassifier(svm.SVC(kernel='linear'))

# setup the parameters and run CV
paramgrid = {'estimator__C': logspace(-3,3,13)}
msvmcv = model_selection.GridSearchCV(msvm, paramgrid, cv=5, n_jobs=-1,␣
 ↪verbose=True)
msvmcv.fit(trainX, trainY)
print(msvmcv.best_params_)
```

```
Fitting 5 folds for each of 13 candidates, totalling 65 fits
{'estimator__C': np.float64(10.0)}
```

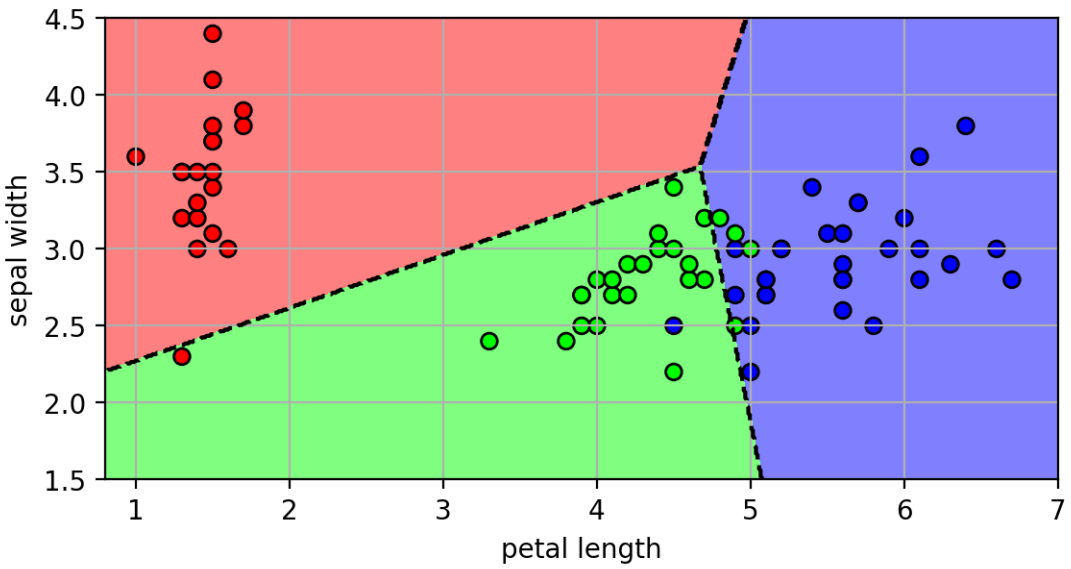
[44]:
```
def plot_posterior3(model, axbox, mycmap, showscore=False):
    xr = [ arange(0.8,7,0.01) , arange(1.5, 4.5, 0.01) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    P = model.predict(allpts).reshape(xgrid0.shape)

    # predict probabilities
    if showscore:
        Z = model.decision_function(allpts)

        ZZ = Z.reshape((len(xr[1]), len(xr[0]), 3))
        ZZZ = maximum(minimum(ZZ, 1), -1)/2 + 0.5
```

```
        plt.imshow(ZZZ, origin='lower', extent=axbox, alpha=0.50,
                   cmap=mycmap, interpolation='nearest')
    else:
        plt.imshow(P, origin='lower', extent=axbox, alpha=0.50,
               cmap=mycmap, interpolation='nearest')

    #plt.contourf(xr[0], xr[1], P, levels=[0,1,2], cmap=mycmap)
    plt.contour(xr[0], xr[1], P, levels=[0.5,1.5,2.5], linestyles='dashed',␣
 ↪colors='black')

# show training data
axbox = [0.8, 7, 1.5, 4.5]
# make a colormap for viewing classes
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#00FF00", "#0000FF"])

svm3fig = plt.figure()
plot_posterior3(msvmcv, axbox, mycmap)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.axis(axbox); plt.grid(True);
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.close()
```

25 3-class decision boundaries

[45]:
```
svm3fig
```

[45]:



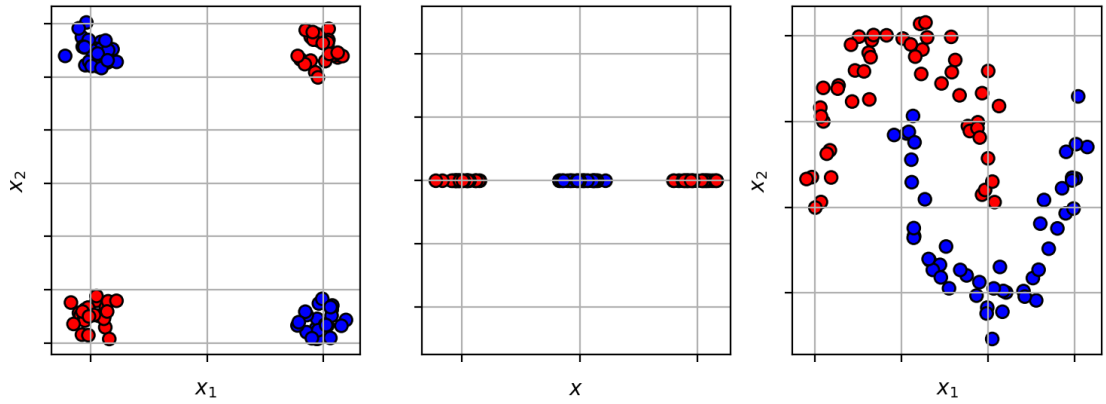
26 Non-linear decision boundary

- What if the data is separable, but not linearly separable?

[46]:
```
nlfig = plt.figure(figsize=(9,3))
plt.subplot(1,3,1)
# type one - XOR
X1,Y1 = datasets.make_blobs(n_samples=100,
         centers=array([[5,5],[-5,-5],[-5, 5], [5, -5]]), cluster_std=0.5,␣
 ↪n_features=2,
         random_state=4487)
Y1 = (Y1>=2)+0
plt.scatter(X1[:,0], X1[:,1], c=Y1, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1$'); plt.ylabel('$x_2$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])

# type two - split
X2,Y2 = datasets.make_blobs(n_samples=100,
         centers=array([[5],[-5],[0]]), cluster_std=0.5, n_features=1,
         random_state=4487)
Y2 = (Y2>1)+0
plt.subplot(1,3,2)
plt.scatter(X2[:,0], zeros(X2.shape), c=Y2, cmap=mycmap, edgecolors='k')
plt.xlabel('$x$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])


# type three - moons
X3,Y3 = datasets.make_moons(n_samples=100,
                                              noise=0.1, random_state=4487)
plt.subplot(1,3,3)
plt.scatter(X3[:,0], X3[:,1], c=Y3, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1$'); plt.ylabel('$x_2$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])
plt.close()
```

[47]:
```
nlfig
```

[47]:



27 Idea - transform the input space

- map from input space $\mathbf{x} \in \mathbb{R}^d$ to a new high-dimensional space $\mathbf{z} \in \mathbb{R}^D$.
  - $\mathbf{z} = \Phi(\mathbf{x})$, where $\Phi(\mathbf{x})$ is the transformation function.
- learn the linear classifier in the new space
  - if dimension of new space is large enough ($D > d$), then the data should be linearly separable

[48]:
```
nl2fig = plt.figure(figsize=(9,7))

# type one - XOR
plt.subplot2grid((5,3),(0,0), rowspan=2)
plt.scatter(X1[:,0], X1[:,1], c=Y1, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1$'); plt.ylabel('$x_2$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])

X1t = X1[:,0]*X1[:,1]

plt.subplot2grid((5,3),(2,0))
plt.annotate("", xy=(0, -1), xytext=(0,0.8),
             ␣
  ↪arrowprops=dict(arrowstyle='Fancy,head_width=3,tail_width=2,head_length=2'))
plt.text(0.2,0,"$\Phi(\mathbf{x}) = x_1*x_2$", fontsize=14)
plt.axis([-1, 1, -1, 1])
plt.axis('off')

plt.subplot2grid((5,3),(3,0), rowspan=2)
plt.scatter(X1t, zeros(X1t.shape), c=Y1, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1 x_2$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])
```

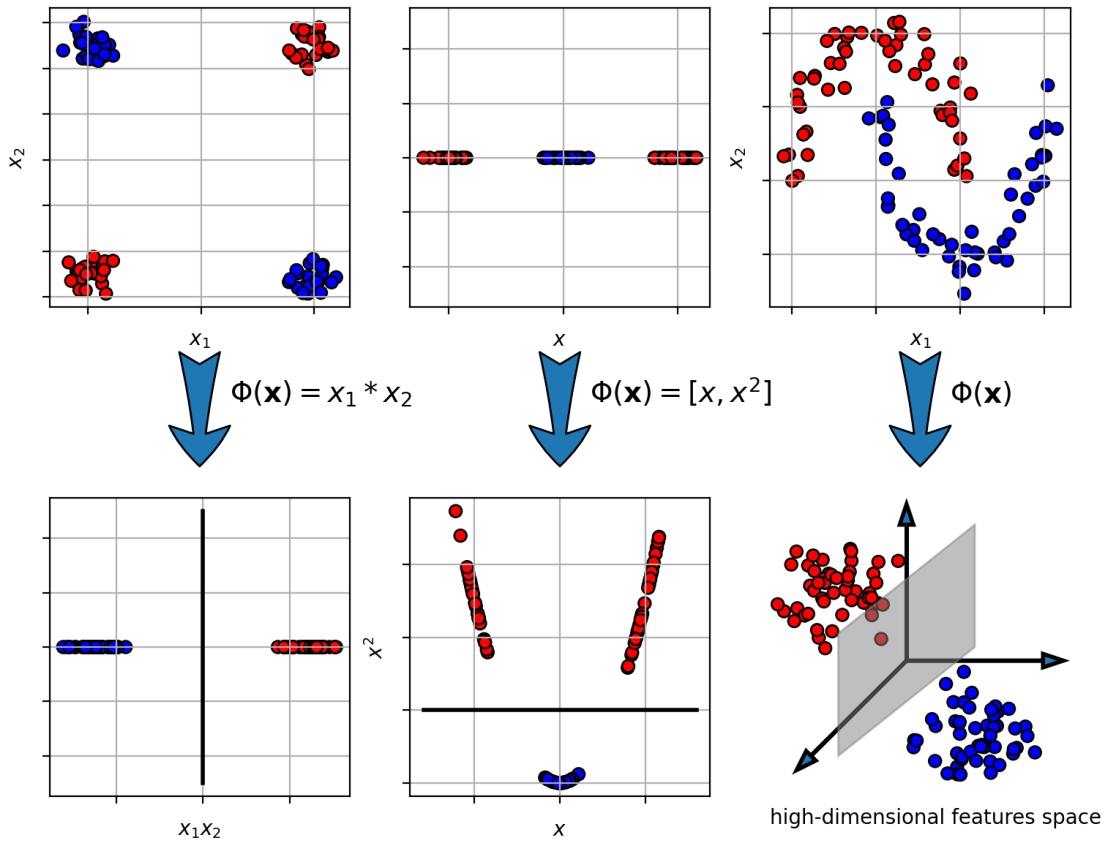
```
plt.plot([0,0],[-0.5,0.5], 'k-', lw=2)

# type two - split
plt.subplot2grid((5,3),(0,1), rowspan=2)
plt.scatter(X2[:,0], zeros(X2.shape), c=Y2, cmap=mycmap, edgecolors='k')
plt.xlabel('$x$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])


plt.subplot2grid((5,3),(2,1))
plt.annotate("", xy=(0, -1), xytext=(0,0.8),
            ␣
 ↪arrowprops=dict(arrowstyle='Fancy,head_width=3,tail_width=2,head_length=2'))
plt.text(0.2,0,"$\Phi(\mathbf{x}) = [x, x^2]$", fontsize=14)
plt.axis([-1, 1, -1, 1])
plt.axis('off')


X2t = c_[X2, X2**2]
plt.subplot2grid((5,3),(3,1), rowspan=2)
plt.scatter(X2t[:,0], X2t[:,1], c=Y2, cmap=mycmap, edgecolors='k')
plt.xlabel('$x$'); plt.ylabel('$x^2$');plt.grid(True)
plt.plot([-8,8],[10,10], 'k-', lw=2)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])

# type three - moons
plt.subplot2grid((5,3),(0,2), rowspan=2)
plt.scatter(X3[:,0], X3[:,1], c=Y3, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1$'); plt.ylabel('$x_2$'); plt.grid(True)
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])

plt.subplot2grid((5,3),(2,2))
plt.annotate("", xy=(0, -1), xytext=(0,0.8),
            ␣
 ↪arrowprops=dict(arrowstyle='Fancy,head_width=3,tail_width=2,head_length=2'))
plt.text(0.2,0,"$\Phi(\mathbf{x})$", fontsize=14)
plt.axis([-1, 1, -1, 1])
plt.axis('off')

plt.subplot2grid((5,3),(3,2), rowspan=2)
plt.arrow(0,0, 10,0, lw=2, head_width=1)
plt.arrow(0,0, 0,10, lw=2, head_width=1)
plt.arrow(0,0, -7,-7, lw=2, head_width=1)
```

```
X3t,Y3t = datasets.make_blobs(n_samples=100,
          centers=array([[-5,5],[5,-5]]), cluster_std=2, n_features=2,
          random_state=4487)
plt.scatter(X3t[:,0], X3t[:,1], c=Y3t, cmap=mycmap, edgecolors='k')
plt.fill([5, -5, -5, 5], [10, 2, -7, 1], alpha=0.5, color='gray', ls='solid')
plt.axis([-10, 12, -10, 12])
plt.text(-10,-12,"high-dimensional features space")
plt.axis('off')

plt.close()
```

```
<>:15: SyntaxWarning: invalid escape sequence '\P'
<>:37: SyntaxWarning: invalid escape sequence '\P'
<>:60: SyntaxWarning: invalid escape sequence '\P'
<>:15: SyntaxWarning: invalid escape sequence '\P'
<>:37: SyntaxWarning: invalid escape sequence '\P'
<>:60: SyntaxWarning: invalid escape sequence '\P'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1394572907.py:1
5: SyntaxWarning: invalid escape sequence '\P'
  plt.text(0.2,0,"$\Phi(\mathbf{x}) = x_1*x_2$", fontsize=14)
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1394572907.py:3
7: SyntaxWarning: invalid escape sequence '\P'
  plt.text(0.2,0,"$\Phi(\mathbf{x}) = [x, x^2]$", fontsize=14)
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1394572907.py:6
0: SyntaxWarning: invalid escape sequence '\P'
  plt.text(0.2,0,"$\Phi(\mathbf{x})$", fontsize=14)
```

[49]:
```
nl2fig
```

[49]:



28 Kernel function

- the SVM dual problem is completely written in terms of *inner product* between the high-dimensional feature vectors: $\Phi(\mathbf{x}_i)^T\Phi(\mathbf{x}_j)$
- So rather than explicitly calculate the high-dimensional vector $\Phi(\mathbf{x}_i)$,
  - we only need to calculate the inner product between two high-dim feature vectors.
- We call this a **kernel function**
  - $k(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^T\Phi(\mathbf{x}_j)$
  - calculating the kernel will be less expensive than explicitly calculating the high-dimensional feature vector and the inner product.

29 Example: Polynomial kernel

- input vector $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^d$
- kernel between two vectors is a p-th order polynomial:
  - $k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T\mathbf{x}')^p = (\sum_{i=1}^{d} x_i x_i')^p$

- For example, $p = 2$,

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}')^2 = (\sum_{i=1}^{d} x_i x_i')^2 \tag{3}$$

$$= \sum_{i=1}^{d} \sum_{j=1}^{d} (x_i x_i' x_j x_j') = \Phi(\mathbf{x})^T \Phi(\mathbf{x}') \tag{4}$$

  - transformed feature space is the quadratic terms of the input vector:

$$\Phi(\mathbf{x}) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ \vdots \\ x_2 x_1 \\ x_2 x_2 \\ \vdots \\ x_d x_1 \\ \vdots \\ x_d x_d \end{bmatrix}$$

- Comparison of number of multiplications
  - for kernel: $O(d)$
  - explicit transformation Φ: $O(d^2)$

30 Kernel trick

- Replacing the inner product with a kernel function in the optimization problem is called the **kernel trick**.
  - *turns a linear classification algorithm into a non-linear classification algorithm.*
  - the shape of the decision boundary is determined by the kernel.

31 Kernel SVM

- Replace inner product in linear SVM with kernel function:

$$\underset{\alpha}{\text{argmax}} \sum_i \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \sum_{i=1}^{N} \alpha_i y_i = 0, \quad \alpha_i \geq 0$$

- Prediction
  - $y_* = \text{sign}(\sum_{i=1}^{N} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}_*) + b)$

32 Example: Kernel SVM with polynomial kernel

- decision surface is a "cut" of a polynomial surface
- higher polynomial-order yields more complex decision boundaries.

[50]:
```
inds = (Y3==0)
X4 = X3[inds]
Y4 = Y3[inds]
```

```
tmpX,tmpY = datasets.make_blobs(n_samples=20,
         centers=1, cluster_std=0.2, n_features=2,
         center_box=(0,0), random_state=4487)
X4 = vstack((X4,tmpX))
Y4 = r_[Y4, tmpY+1]
```

[51]:
```
# fit SVM (poly kernel with different degrees)
degs = [2,3,4]

clf = {}
for d in degs:
    clf[d] = svm.SVC(kernel='poly', degree=d, C=100)
    clf[d].fit(X4, Y4)
```

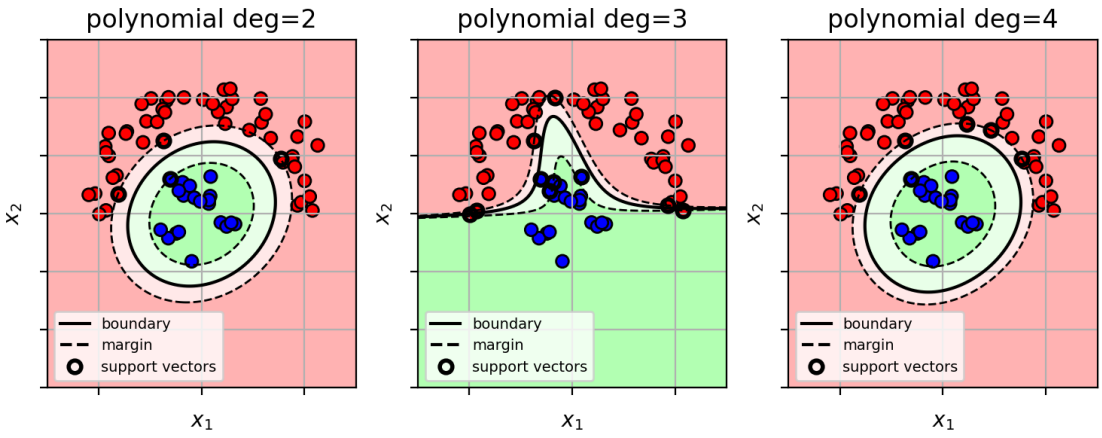
[52]:
```
def plot_posterior_svm(model, axbox, X, phi=None):
    # grid points
    xr = [ linspace(axbox[0], axbox[1], 200),
           linspace(axbox[2], axbox[3], 200) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allptsx = c_[xgrid0.ravel(), xgrid1.ravel()]

    # transform
    if phi != None:
        allpts = apply_along_axis(phi, 1, allptsx)
    else:
        allpts = allptsx

    # calculate the score
    score = model.decision_function(allpts).reshape(xgrid0.shape)

    cmap = ([1,0,0], [1,0.7,0.7], [0.7,1,0.7], [0,1,0])
    CS = plt.contourf(xr[0], xr[1], score, colors=cmap,
                      levels=[-1000, -1, 0, 1, 1000], alpha=0.3)

    plt.contour(xr[0], xr[1], score, levels=[-1, 1], linewidths=1,␣
↪linestyles='dashed', colors='k')
    plt.contour(xr[0], xr[1], score, levels=[0], linestyles='solid', colors='k')

    plt.plot([-100,-100], [-100,-100], 'k-', label='boundary')
    plt.plot([-100,-100], [-100,-100], 'k--', label='margin')

    plt.plot(X[model.support_,0], X[model.support_,1],
             'ko',fillstyle='none', markeredgewidth=2, label='support vectors')
```

```
    leg = plt.legend(loc='best', fontsize=7)
    leg.get_frame().set_facecolor('white')
    plt.axis(axbox); plt.grid(True)
```

[53]:
```
axbox = [-1.5, 1.5, -1.5, 1.5]

polysvmfig = plt.figure(figsize=(9,3))

for (i,d) in enumerate(degs):
    plt.subplot(1,3,i+1)
    plot_posterior_svm(clf[d], axbox, X4)
    plt.scatter(X4[:,0], X4[:,1], c=Y4, cmap=mycmap, edgecolors='k')
    plt.xlabel('$x_1$'); plt.ylabel('$x_2$')
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    plt.title('polynomial deg=' + str(degs[i]))
plt.close()
```

[54]:
```
polysvmfig
```

[54]:



33 RBF kernel

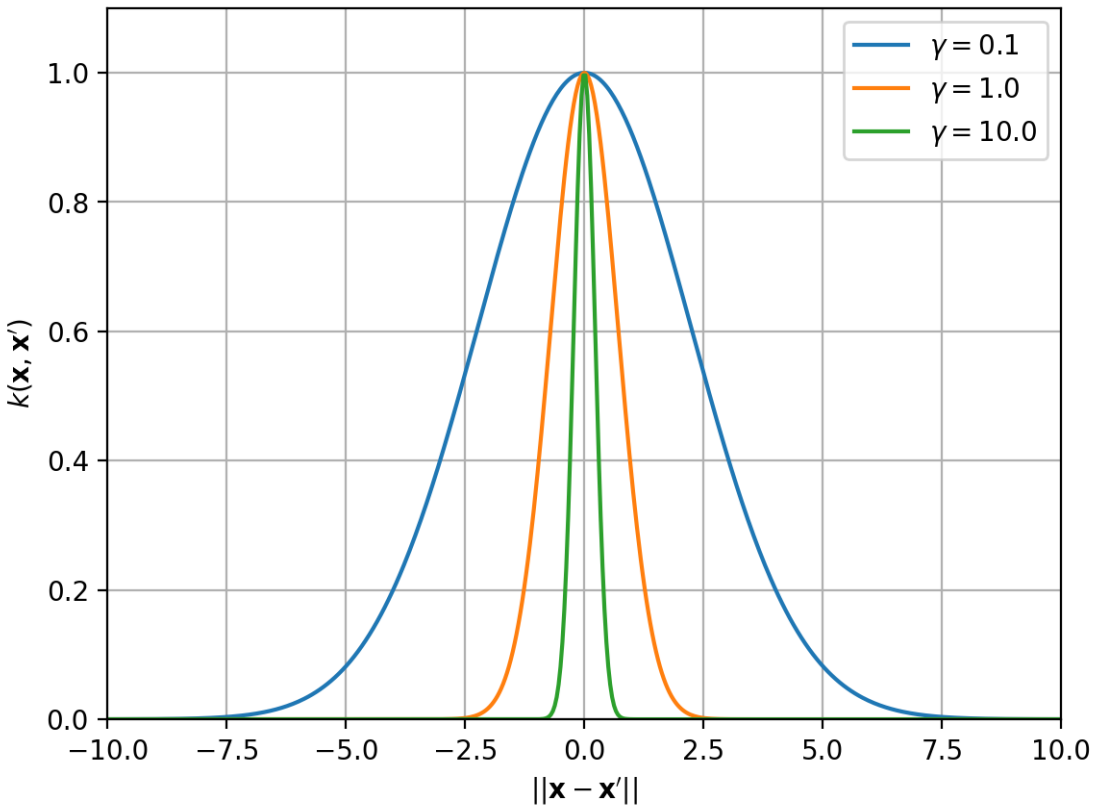
- RBF kernel (radial basis function)
  - $k(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$
  - similar to a Gaussian
- gamma $\gamma > 0$ is the inverse bandwidth parameter of the kernel
  - controls the smoothness of the function
  - small $\gamma \rightarrow$ wider RBF $\rightarrow$ smooth functions
  - large $\gamma \rightarrow$ thin RBF $\rightarrow$ wiggly function

[55]:
```
rbffig = plt.figure()
x = linspace(-10,10,500)
gammas = array([0.1, 1.0, 10.0])
for g in gammas:
    kx = exp(-g*(x**2))
    plt.plot(x, kx, label="$\gamma="+str(g)+"$")
plt.xlabel("$||\mathbf{x}-\mathbf{x}'||$")
plt.ylabel("$k(\mathbf{x},\mathbf{x}')$")
plt.axis([-10, 10, 0, 1.1]); plt.grid(True)
plt.legend()
plt.close()
```

```
<>:6: SyntaxWarning: invalid escape sequence '\g'
<>:7: SyntaxWarning: invalid escape sequence '\m'
<>:8: SyntaxWarning: invalid escape sequence '\m'
<>:6: SyntaxWarning: invalid escape sequence '\g'
<>:7: SyntaxWarning: invalid escape sequence '\m'
<>:8: SyntaxWarning: invalid escape sequence '\m'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1872335391.py:6
: SyntaxWarning: invalid escape sequence '\g'
  plt.plot(x, kx, label="$\gamma="+str(g)+"$")
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1872335391.py:7
: SyntaxWarning: invalid escape sequence '\m'
  plt.xlabel("$||\mathbf{x}-\mathbf{x}'||$")
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1872335391.py:8
: SyntaxWarning: invalid escape sequence '\m'
  plt.ylabel("$k(\mathbf{x},\mathbf{x}')$")
```

[56]:
```
rbffig
```

[56]:



34 Kernel SVM with RBF kernel

- try different γ
  - each γ yields different levels of smoothness of the decision boundary

[57]:
```
# fit SVM (RBF)
gammas = [0.1, 1, 25]

clf = {}
for i in gammas:
    clf[i] = svm.SVC(kernel='rbf', gamma=i, C=1000)
    clf[i].fit(X3, Y3)
```

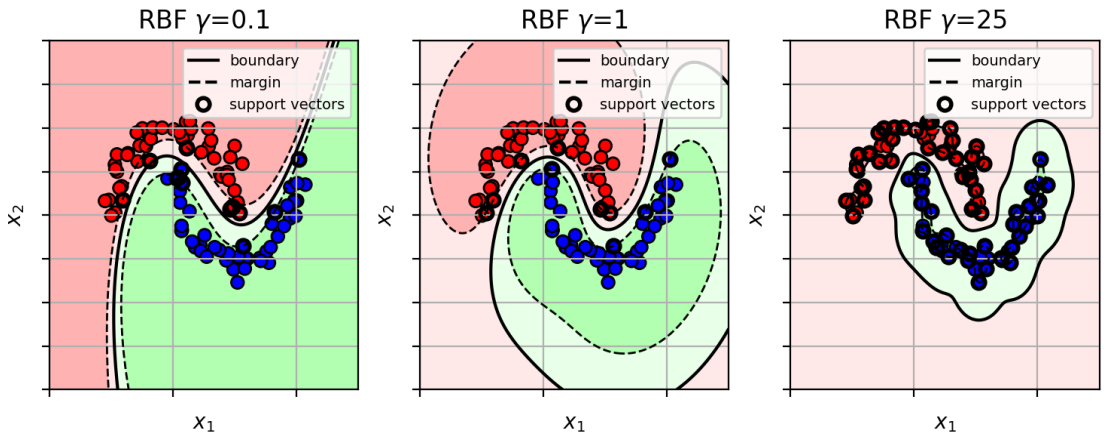
[58]:
```
axbox = [-2, 3, -2, 2]

rbfsvmfig = plt.figure(figsize=(9,3))

for i,x in enumerate(gammas):
    clfx = clf[x]
    plt.subplot(1,3,i+1)
```

```
    plot_posterior_svm(clfx, axbox, X3)
    plt.scatter(X3[:,0], X3[:,1], c=Y3, cmap=mycmap, edgecolors='k')
    plt.xlabel('$x_1$'); plt.ylabel('$x_2$')
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    plt.title('RBF $\gamma$=' + str(x))
plt.close()
```

```
<>:14: SyntaxWarning: invalid escape sequence '\g'
<>:14: SyntaxWarning: invalid escape sequence '\g'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/1549002258.py:1
4: SyntaxWarning: invalid escape sequence '\g'
  plt.title('RBF $\gamma$=' + str(x))
```

[59]:
```
rbfsvmfig
```

[59]:



35 Example on Iris data

- Large γ yields a complicated wiggly decision boundary.

[60]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris2.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (versicolor=1,␣
 ↪virginica=2)

print(X.shape)
```

```
(100, 2)
```

[61]:
```
# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

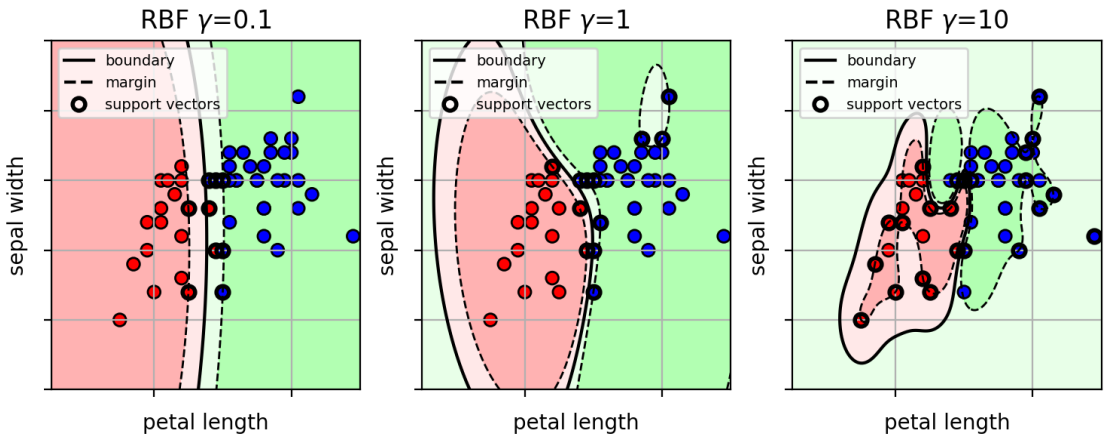
```
(50, 2)
(50, 2)
```

[62]:
```
# fit SVM (RBF)
gammas = [0.1, 1, 10]

clf = {}
for i in gammas:
    clf[i] = svm.SVC(kernel='rbf', gamma=i, C=100)
    clf[i].fit(trainX, trainY)

irbffig = plt.figure(figsize=(9,3))
axbox = [2.5, 7, 1.5, 4]
for i,x in enumerate(gammas):
    plt.subplot(1,3,i+1)
    plot_posterior_svm(clf[x], axbox, trainX)
    plt.scatter(trainX[:,0],trainX[:,1],c=trainY, cmap=mycmap, edgecolors='k')
    plt.xlabel('petal length'); plt.ylabel('sepal width')
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    plt.title('RBF $\gamma$=' + str(x))
plt.close()
```

```
<>:18: SyntaxWarning: invalid escape sequence '\g'
<>:18: SyntaxWarning: invalid escape sequence '\g'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43322/263472167.py:18
: SyntaxWarning: invalid escape sequence '\g'
  plt.title('RBF $\gamma$=' + str(x))
```

[63]:
```
irbffig
```

[63]:



36 How to select the best kernel parameters?

- use cross-validation over possible kernel parameter (γ) and SVM C parameter
  - if a lot of parameters, can be computationally expensive!

[64]:
```
# setup the list of parameters to try
paramgrid = {'C': logspace(-2,3,20),
             'gamma': logspace(-4,3,20) }

print(paramgrid)

# setup the cross-validation object
# pass the SVM object w/ rbf kernel, parameter grid, and number of CV folds
svmcv = model_selection.GridSearchCV(svm.SVC(kernel='rbf'), paramgrid, cv=5,
                                     n_jobs=-1, verbose=True)

# run cross-validation (train for each split)
svmcv.fit(trainX, trainY);

print("best params:", svmcv.best_params_)
```

```
{'C': array([1.00000000e-02, 1.83298071e-02, 3.35981829e-02, 6.15848211e-02,
       1.12883789e-01, 2.06913808e-01, 3.79269019e-01, 6.95192796e-01,
       1.27427499e+00, 2.33572147e+00, 4.28133240e+00, 7.84759970e+00,
       1.43844989e+01, 2.63665090e+01, 4.83293024e+01, 8.85866790e+01,
       1.62377674e+02, 2.97635144e+02, 5.45559478e+02, 1.00000000e+03]),
'gamma': array([1.00000000e-04, 2.33572147e-04, 5.45559478e-04, 1.27427499e-03,
       2.97635144e-03, 6.95192796e-03, 1.62377674e-02, 3.79269019e-02,
       8.85866790e-02, 2.06913808e-01, 4.83293024e-01, 1.12883789e+00,
       2.63665090e+00, 6.15848211e+00, 1.43844989e+01, 3.35981829e+01,
       7.84759970e+01, 1.83298071e+02, 4.28133240e+02, 1.00000000e+03])}
Fitting 5 folds for each of 400 candidates, totalling 2000 fits
```

```
best params: {'C': np.float64(0.37926901907322497), 'gamma':
np.float64(0.4832930238571752)}
```

[65]:
```
# show the test error for the first 25 parameter sets
N = 25
for m,p in zip(svmcv.cv_results_['mean_test_score'][0:N], svmcv.
 ↪cv_results_['params'][0:N]):
    print("mean={:.4f} {}".format(m,p))
```

```
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.0001)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.00023357214690901214)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.000545559478116852)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.0012742749857031334)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.002976351441631319)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.0069519279617756054)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.01623776739188721)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.0379269019073225)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.08858667904100823)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.2069138081114788)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(0.4832930238571752)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(1.1288378916846884)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(2.6366508987303554)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(6.1584821106602545)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(14.38449888287663)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(33.59818286283781)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(78.47599703514607)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(183.29807108324337)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(428.1332398719387)}
mean=0.6200 {'C': np.float64(0.01), 'gamma': np.float64(1000.0)}
mean=0.6200 {'C': np.float64(0.018329807108324356), 'gamma': np.float64(0.0001)}
mean=0.6200 {'C': np.float64(0.018329807108324356), 'gamma':
np.float64(0.00023357214690901214)}
mean=0.6200 {'C': np.float64(0.018329807108324356), 'gamma':
np.float64(0.000545559478116852)}
mean=0.6200 {'C': np.float64(0.018329807108324356), 'gamma':
np.float64(0.0012742749857031334)}
mean=0.6200 {'C': np.float64(0.018329807108324356), 'gamma':
np.float64(0.002976351441631319)}
```

[66]:
```
# show classifier with training data
plt.figure()
plot_posterior_svm(svmcv.best_estimator_, axbox, trainX)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.title('class boundary with training data');
plt.show()
```

[67]:
```
# predict from the model
predY = svmcv.predict(testX)

# calculate accuracy
acc      = metrics.accuracy_score(testY, predY)
print("test accuracy =", acc)
```

```
test accuracy = 0.88
```

37 Custom kernel function

- we can use any kernel function, as long as it is *positive semi-definite*:
  - 1) it can be written as an inner-product of a feature transformation: $k(\mathbf{x}_1, \mathbf{x}_2) = \langle \Phi(\mathbf{x}_1), \Phi(\mathbf{x}_2) \rangle$.
  - 2) for all possible datasets $\mathbf{X} = \{\mathbf{x}_1, \cdots \mathbf{x}_N\}$ of all possible sizes N, the kernel matrix $K = [k(\mathbf{x}_i, \mathbf{x}_j)]_{i,j}$ is a positive definite matrix.
    - $\mathbf{K}$ is a *positive semi-definite matrix* iff $\mathbf{z}^T \mathbf{K} \mathbf{z} \geq 0, \forall \mathbf{z}$
- in sklearn, pass a callable function as the kernel parameter.

37.1 Example: Laplacian kernel

- $k(\mathbf{x}_1, \mathbf{x}_2) = \exp(-\alpha||\mathbf{x}_1 - \mathbf{x}_2||)$

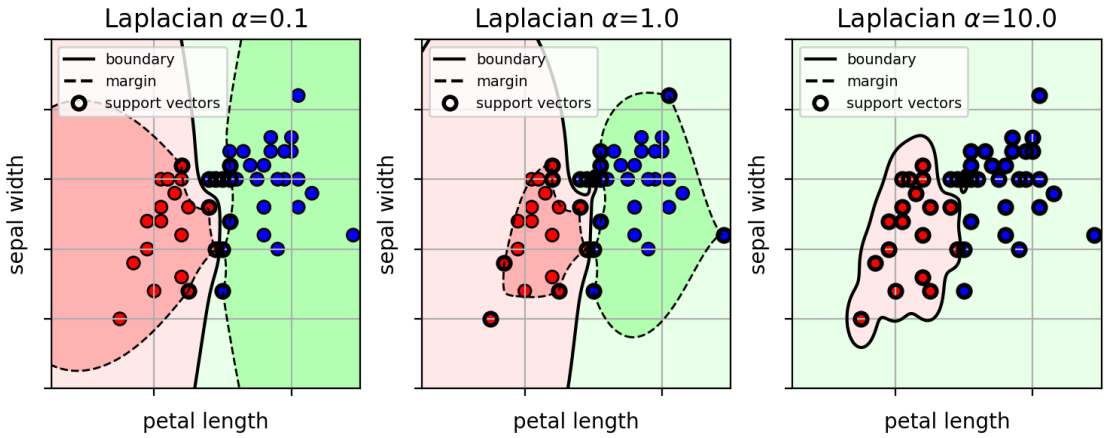
[68]:
```
from scipy import spatial

# create a custom kernel function
# Laplacian kernel: exp( -alpha*||x1-x2|| )
def mykernel(X1, X2, alpha=1.0):
    # X1,X2 are (N1 x d) and (N2 x d) matrices of N1 and N2 d-dim vectors
    # alpha is the hyperparameter

    # compute pairwise euclidean distance
    D = spatial.distance.cdist(X1, X2, metric='euclidean')

    # return the kernel matrix
    return exp(-alpha*D)
```

[69]:
```
alphas = [0.1, 1., 10.]

clf = {}
for i in alphas:
    # make a temporary kernel function with the selected alpha value
    tmpkern = lambda X1,X2,alpha=i: mykernel(X1,X2,alpha=alpha)

    # create the SVM with custom kernel function
    clf[i] = svm.SVC(kernel=tmpkern, C=100)
    clf[i].fit(trainX, trainY)
```

[70]:
```
ilapfig = plt.figure(figsize=(9,3))
axbox = [2.5, 7, 1.5, 4]
for i,x in enumerate(alphas):
    plt.subplot(1,3,i+1)
    plot_posterior_svm(clf[x], axbox, trainX)
    plt.scatter(trainX[:,0],trainX[:,1],c=trainY, cmap=mycmap, edgecolors='k')
    plt.xlabel('petal length'); plt.ylabel('sepal width')
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    plt.title('Laplacian $\\alpha$=' + str(x))
plt.close()
```

[71]:
```
ilapfig
```

[71]:



38 SVM Summary

- **Linear SVM:**
  - linear function $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$
  - given new sample $\mathbf{x}_*$, predict $y_* = \text{sign}(f(\mathbf{x}_*))$.
- **Kernel SVM:**
  - Kernel function defines the shape of the non-linear decision boundary.
    - implicitly transforms input feature into high-dimensional space.
    - uses linear classifier in high-dim space.
    - the decision boundary is non-linear in the original input space.
- **Training:**
  - Maximize the margin of the training data.
    - i.e., maximize the separation between the points and the decision boundary.
  - Use cross-validation to pick the hyperparameter C and the kernel hyperparameters.

```
X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (versicolor=1,␣
 ↪virginica=2)

print(X.shape)
```

```
(100, 2)
```

[3]:
```
# get the NB Gaussian model from sklearn
model = naive_bayes.GaussianNB()

# fit the model
model.fit(X, Y)
v = mean(model.var_.ravel())  # make it shared variance
model.var_[:] = v
```

[4]:
```
# a colormap for making the scatter plot: class 1 will be red, class 2 will be␣
 ↪green
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#FFFFFF", "#00FF00"])

axbox = [2.5, 7, 1.5, 4] # common axis range

# a function for setting a common plot
def irisaxis():
    plt.xlabel('petal length'); plt.ylabel('sepal width')
    plt.axis([2.5, 7, 1.5, 4]); plt.grid(True)
```

[5]:
```
## Visualization ␣
 ↪#############################################################################################
def plot_ellipse(ax, musigma, color="k", lw=1):
    """
    Based on
    http://stackoverflow.com/questions/17952171/
 ↪not-sure-how-to-fit-data-with-a-gaussian-python.
    """

    mu, sigma = musigma

    # Compute eigenvalues and associated eigenvectors
    vals, vecs = linalg.eigh(sigma)

    # Compute "tilt" of ellipse using first eigenvector
    x, y = vecs[:, 0]
    theta = degrees(arctan2(y, x))
```

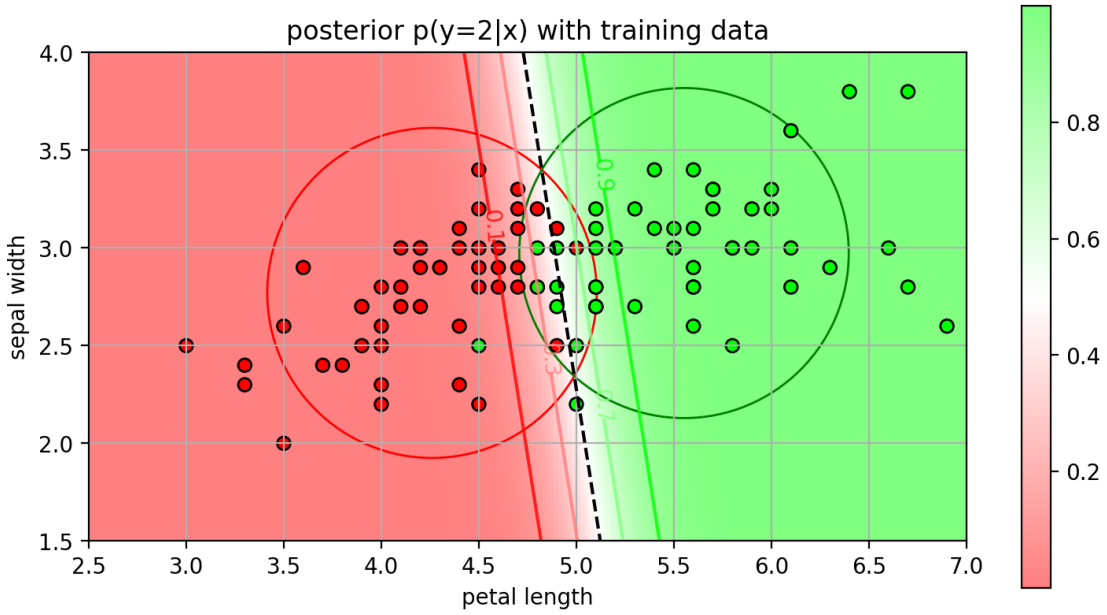
```
    # Eigenvalues give length of ellipse along each eigenvector
    # plot 2 stdevs
    w, h = 2 * sqrt(vals) * 2

    #ax.tick_params(axis='both', which='major', labelsize=20)
    ellipse = matplotlib.patches.Ellipse(mu, w, h, angle=theta, fill=False,␣
 ↪color=color, lw=lw)  # color="k")
    ellipse.set_clip_box(ax.bbox)
    ellipse.set_alpha(1.0)
    ax.add_artist(ellipse)
```

[6]:
```
def plot_posterior(model, axbox, mycmap, showlabels=True):
    xr = [ linspace(axbox[0], axbox[1], 200),
           linspace(axbox[2], axbox[3], 200) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # calculate the posterior probability
    post = model.predict_proba(allpts)
    # extract the posterior for class 2, and reshape into a grid
    post1 = post[:,1].reshape(xgrid0.shape)

    # contour plot of the posterior and decision boundary
    plt.imshow(post1, origin='lower', extent=axbox, alpha=0.50, cmap=mycmap)
    if showlabels:
        plt.colorbar(shrink=0.6)
    CS = plt.contour(xr[0], xr[1], post1, cmap=mycmap, levels=[0.1, 0.3, 0.7, 0.
 ↪9], alpha=0.8)
    if showlabels:
        #plt.colorbar(CS)
        plt.clabel(CS, inline=1, fontsize=10)
    plt.contour(xr[0], xr[1], post1, levels=[0.5], linestyles='dashed',␣
 ↪colors='black')
    irisaxis()
```

[7]:
```
pfig = plt.figure(figsize=(9,8))
plot_posterior(model, axbox, mycmap, showlabels=True)
plot_ellipse(plt.gca(), (model.theta_[0,:], diag(model.var_[0,:])), color='r')
plot_ellipse(plt.gca(), (model.theta_[1,:], diag(model.var_[1,:])), color='g')
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.title('posterior p(y=2|x) with training data');
plt.close()
```

- Example

[8]:
```
pfig
```
[8]:



- BDR in this case is a **linear** function
  - select class $y = 1$ when:

$$\sum_{i=1}^{D} \underbrace{\frac{1}{\sigma^2}(\mu_i - \nu_i)}_{w_i} x_i + \underbrace{\frac{1}{2\sigma^2}\sum_{i=1}^{D}(\nu_i^2 - \mu_i^2) + \log\frac{\pi_1}{\pi_2}}_{b} > 0$$

    - w_i is a per-feature weight
    - b is a bias term

- the BDR in this case is a **linear** classifier:
  - select class $y = 1$ when
    - $ _{i=1}^D w_i x_i + b > 0 $
    - equivalently, $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b > 0$
- Here we obtain the weights $\mathbf{w}$ by learning the CCDs
  - assuming Naive Bayes Gaussians with shared variance.
  - this is a generative model since we learn how the data is generated for each class (CCDs).
- How to learn the linear classifier in a discriminative way?
  - directly learn the posterior $p(y|\mathbf{x})$.
  - we will look at a generic linear classifier.

6 Linear Classifier

- **Setup**
  - Observation (feature vectors) $\mathbf{x} \in \mathbb{R}^d$

  - Class $y \in \{-1, +1\}$
- **Goal**: given a feature vector $\mathbf{x}$, predict its class y.
  - Calculate a *linear function* of the feature vector $\mathbf{x}$.
    - $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b$
      - $\mathbf{w} \in \mathbb{R}^d$ are the weights of the linear function.
      - multiply each feature value with a weight, and then add together.
  - Predict from the value:
    - if $f(\mathbf{x}) > 0$ then predict Class $y = 1$
    - if $f(\mathbf{x}) < 0$ then predict Class $y = -1$
    - Equivalently, $y = \text{sign}(f(\mathbf{x}))$

7 Geometric Interpretation

- The linear classifier separates the features space into 2 *half-spaces*
  - corresponding to feature values belonging to Class +1 and Class -1
  - the class boundary is normal to $\mathbf{w}$.
    - also called the *separating hyperplane*.

- Example:

$$\mathbf{w} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, b = 0$$

[9]:
```
def drawplane(w, b, wlabel=None, poscol=None, negcol=None):
    # w^Tx + b = 0
    # w0 x0 + w1 x1 + b = 0
    # x1 = -w0/w1 x0 - b / w1

    # the line
    x0 = linspace(-10,10)
    x1 = -w[0]/w[1] * x0 - b / w[1]


    # fill positive half-space or neg space
    if (poscol):
        polyx = [x0[0], x0[-1], x0[-1], x0[0]]
        polyy = [x1[0], x1[-1], x1[0], x1[0]]
        plt.fill(polyx, polyy, poscol, alpha=0.2)

    if (negcol):
        polyx = [x0[0], x0[-1], x0[0], x0[0]]
        polyy = [x1[0], x1[-1], x1[-1], x1[0]]
        plt.fill(polyx, polyy, negcol, alpha=0.2)

    # plot line
    plt.plot(x0, x1, 'k-', lw=2)

    # the w
```

```
    if (wlabel):
        xp = array([0, -b/w[1]])
        xpw = xp+w
        plt.arrow(xp[0], xp[1], w[0], w[1], width=0.01, head_width=0.3,␣
↪fc='black')
        plt.text(xpw[0]-0.5, xpw[1], wlabel)
```

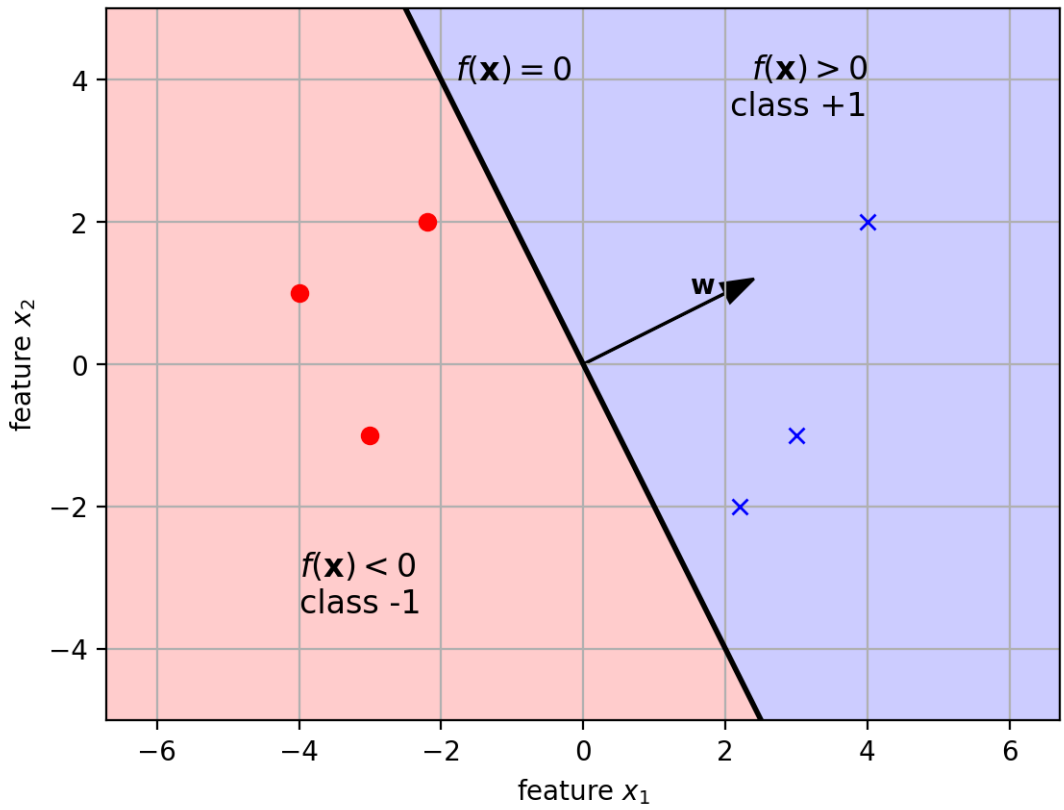
[10]:
```
linclass = plt.figure()
w = array([2, 1])
b = 0;

drawplane(w, b, '$\mathbf{w}$', 'b', 'r')

# a few points
plt.plot([2.2,4,3], [-2,2,-1], 'bx')
plt.plot([-2.2,-4,-3], [2,1,-1], 'ro')

plt.text(-1.8,4, "$f(\mathbf{x}) = 0$", fontsize=12)
plt.text(4, 3.5, "$f(\mathbf{x})>0$\nclass +1", ha="right", fontsize=12)
plt.text(-4, -3.5, "$f(\mathbf{x})<0$\nclass -1", fontsize=12)

plt.xlabel('feature $x_1$'); plt.ylabel('feature $x_2$')
plt.axis('equal')
plt.axis([-5, 5, -5, 5]); plt.grid(True)
plt.close()
```

```
<>:5: SyntaxWarning: invalid escape sequence '\m'
<>:11: SyntaxWarning: invalid escape sequence '\m'
<>:12: SyntaxWarning: invalid escape sequence '\m'
<>:13: SyntaxWarning: invalid escape sequence '\m'
<>:5: SyntaxWarning: invalid escape sequence '\m'
<>:11: SyntaxWarning: invalid escape sequence '\m'
<>:12: SyntaxWarning: invalid escape sequence '\m'
<>:13: SyntaxWarning: invalid escape sequence '\m'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/1863615804.py:5
: SyntaxWarning: invalid escape sequence '\m'
  drawplane(w, b, '$\mathbf{w}$', 'b', 'r')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/1863615804.py:1
1: SyntaxWarning: invalid escape sequence '\m'
  plt.text(-1.8,4, "$f(\mathbf{x}) = 0$", fontsize=12)
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/1863615804.py:1
2: SyntaxWarning: invalid escape sequence '\m'
  plt.text(4, 3.5, "$f(\mathbf{x})>0$\nclass +1", ha="right", fontsize=12)
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/1863615804.py:1
3: SyntaxWarning: invalid escape sequence '\m'
  plt.text(-4, -3.5, "$f(\mathbf{x})<0$\nclass -1", fontsize=12)
```

[11]:
```
linclass
```

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```

[11]:



8 Separating Hyperplane

- In a d-dimensional feature space, the parameters are $\mathbf{w} \in \mathbb{R}^d$.
- The equation $\mathbf{w}^T\mathbf{x} + b = 0$ defines a $(d-1)$-dim. linear surface:
  - for $d = 2$, $\mathbf{w}$ defines a 1-D line.
  - for $d = 3$, $\mathbf{w}$ defines a 2-D plane.
  - ...
  - in general, we call it a hyperplane.

9 Learning the classifier

- How to set the classifier parameters $(\mathbf{w}, b)$?
  - Learn them from training data!
- Classifiers differ in the objectives used to learn the parameters $(\mathbf{w}, b)$.
  - We will look at two examples:
    - *logistic regression*

* *support vector machine (SVM)*

10 Logistic regression

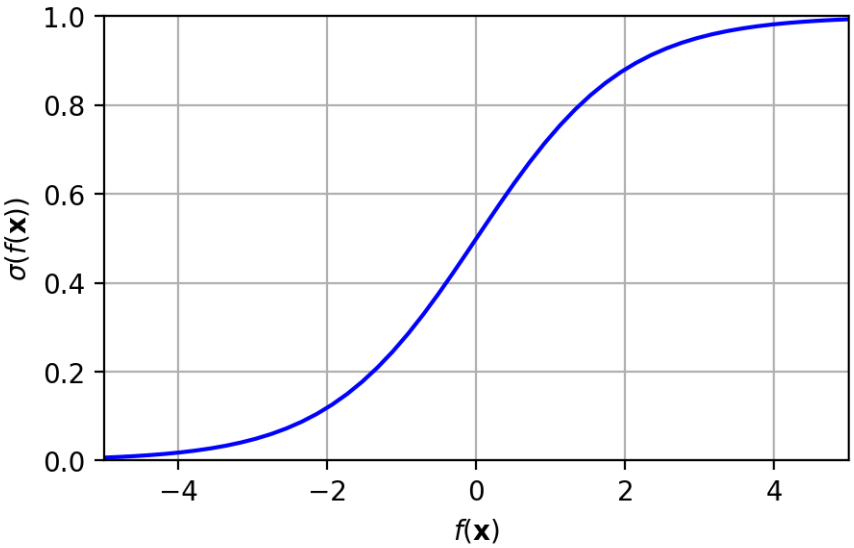
- Use a probabilistic approach
  - Map the linear function $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ to probability values between 0 and 1 using a *sigmoid* function.
  - $\sigma(z) = \frac{1}{1+e^{-z}}$

[12]:
```
z = linspace(-5,5)
sigz = 1/(1+exp(-z))

sigmoidplot = plt.figure(figsize=(5,3))

plt.plot(z,sigz, 'b-')

plt.xlabel('$f(\mathbf{x})$'); plt.ylabel('$\sigma(f(\mathbf{x}))$')
plt.axis([-5, 5, 0, 1]); plt.grid(True)
plt.close()
```

```
<>:8: SyntaxWarning: invalid escape sequence '\m'
<>:8: SyntaxWarning: invalid escape sequence '\s'
<>:8: SyntaxWarning: invalid escape sequence '\m'
<>:8: SyntaxWarning: invalid escape sequence '\s'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3647619821.py:8
: SyntaxWarning: invalid escape sequence '\m'
  plt.xlabel('$f(\mathbf{x})$'); plt.ylabel('$\sigma(f(\mathbf{x}))$')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3647619821.py:8
: SyntaxWarning: invalid escape sequence '\s'
  plt.xlabel('$f(\mathbf{x})$'); plt.ylabel('$\sigma(f(\mathbf{x}))$')
```

[13]:
```
sigmoidplot
```

[13]:



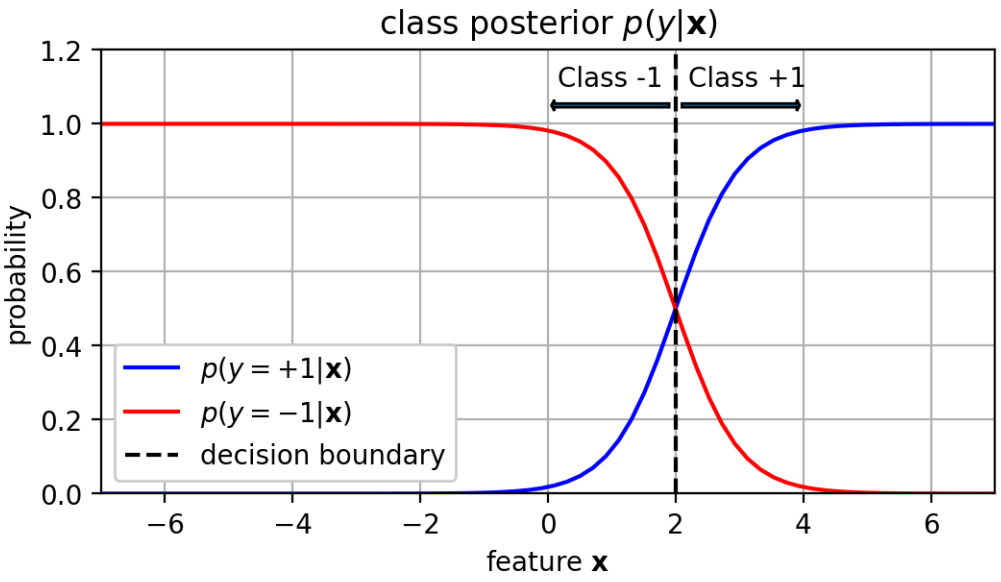
- Given a feature vector x, the probability of a class is:
  - $p(y = +1|\mathbf{x}) = \sigma(f(\mathbf{x}))$
  - $p(y = -1|\mathbf{x}) = 1 - \sigma(f(\mathbf{x}))$
- Note: here we are directly modeling the class posterior probability!
  - not the class-conditional $p(\mathbf{x}|y)$

[14]:
```
x = linspace(-10,10, 100)
w = 2.0
b = -4.0
f = w*x+b
sf = 1/(1+exp(-f))
midx = -b/w

lrexample = plt.figure(figsize=(6,3))

plt.plot(x,sf, 'b-', label="$p(y=+1|\mathbf{x})$")
plt.plot(x,1-sf, 'r-', label="$p(y=-1|\mathbf{x})$")
plt.plot([midx, midx], [0.0,1.2], 'k--', label="decision boundary")

plt.arrow(midx-0.1,1.05,-1.8,0,width=0.01)
plt.arrow(midx+0.1,1.05,1.8,0,width=0.01)
plt.text(midx+0.2,1.10,"Class +1")
plt.text(midx-0.2,1.10,"Class -1", horizontalalignment='right')
plt.legend(loc=0, framealpha=1, fontsize='medium')
plt.title('class posterior $p(y|\mathbf{x})$');
plt.xlabel('feature $\mathbf{x}$'); plt.ylabel('probability')
plt.axis([-7, 7, 0, 1.2]); plt.grid(True)
plt.close()
```

```
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:11: SyntaxWarning: invalid escape sequence '\m'
<>:19: SyntaxWarning: invalid escape sequence '\m'
<>:20: SyntaxWarning: invalid escape sequence '\m'
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:11: SyntaxWarning: invalid escape sequence '\m'
<>:19: SyntaxWarning: invalid escape sequence '\m'
<>:20: SyntaxWarning: invalid escape sequence '\m'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3055418326.py:1
0: SyntaxWarning: invalid escape sequence '\m'
  plt.plot(x,sf, 'b-', label="$p(y=+1|\mathbf{x})$")
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3055418326.py:1
1: SyntaxWarning: invalid escape sequence '\m'
  plt.plot(x,1-sf, 'r-', label="$p(y=-1|\mathbf{x})$")
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3055418326.py:1
9: SyntaxWarning: invalid escape sequence '\m'
  plt.title('class posterior $p(y|\mathbf{x})$');
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3055418326.py:2
0: SyntaxWarning: invalid escape sequence '\m'
  plt.xlabel('feature $\mathbf{x}$'); plt.ylabel('probability')
```

[15]:
```
lrexample
```

[15]:



11 Learning the parameters

- Given training data $\{\mathbf{x}_i, y_i\}_{i=1}^N$, learn the function parameters $(\mathbf{w}, b)$ using maximum likelihood estimation.

- maximize the likelihood of the data $\{\mathbf{x}_i, y_i\}$ according to the posterior:

$$(\mathbf{w}^*, b^*) = \underset{\mathbf{w},b}{\operatorname{argmax}} \sum_{i=1}^{N} \log p(y_i|\mathbf{x}_i)$$

- posterior is a Bernoulli distribution (given $\mathbf{x}$):

$$p(y|\mathbf{x}) = \begin{cases} \sigma(f(\mathbf{x})), & y = 1 \\ 1 - \sigma(f(\mathbf{x})), & y = -1 \end{cases}$$

- Note the following property:

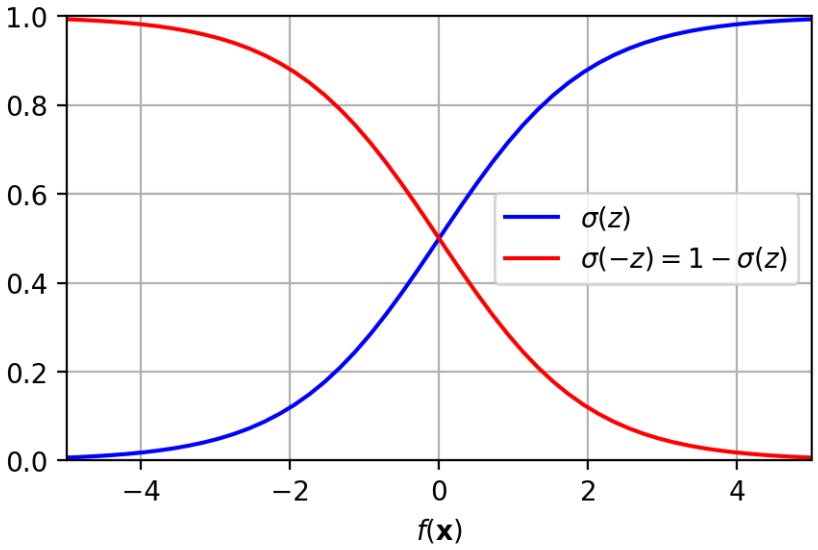
$$1 - \sigma(z) = \sigma(-z)$$

[16]:
```
z = linspace(-5,5)
sigz = 1/(1+exp(-z))
signz = 1/(1+exp(z))

sigmoidplot = plt.figure(figsize=(5,3))

plt.plot(z,sigz, 'b-', label='$\sigma(z)$')
plt.plot(z,signz, 'r-', label='$\sigma(-z) = 1-\sigma(z)$')

plt.xlabel('$f(\mathbf{x})$');
plt.axis([-5, 5, 0, 1]); plt.grid(True)
plt.legend()
plt.close()
```

```
<>:7: SyntaxWarning: invalid escape sequence '\s'
<>:8: SyntaxWarning: invalid escape sequence '\s'
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:7: SyntaxWarning: invalid escape sequence '\s'
<>:8: SyntaxWarning: invalid escape sequence '\s'
<>:10: SyntaxWarning: invalid escape sequence '\m'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3063172703.py:7
: SyntaxWarning: invalid escape sequence '\s'
  plt.plot(z,sigz, 'b-', label='$\sigma(z)$')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3063172703.py:8
: SyntaxWarning: invalid escape sequence '\s'
  plt.plot(z,signz, 'r-', label='$\sigma(-z) = 1-\sigma(z)$')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/3063172703.py:1
0: SyntaxWarning: invalid escape sequence '\m'
  plt.xlabel('$f(\mathbf{x})$');
```

[17]:
```
sigmoidplot
```

[17]:



- Thus,

$$p(y|\mathbf{x}) = \begin{cases} \sigma(f(\mathbf{x})), & y = 1 \\ \sigma(-f(\mathbf{x})), & y = -1 \end{cases}$$

- Simplifying the 2 cases into one equation,

$$p(y|\mathbf{x}) = \sigma(yf(\mathbf{x}))$$

- Taking the log,

$$\begin{aligned} \log p(y|\mathbf{x}) &= \log \sigma(yf(\mathbf{x})) \\ &= \log \frac{1}{1 + e^{-yf(\mathbf{x})}} \\ &= -\log(1 + e^{-yf(\mathbf{x})}) \end{aligned}$$

- Substituting into the MLE formulation:

$$\begin{aligned} (\mathbf{w}^*, b^*) &= \underset{\mathbf{w},b}{\operatorname{argmax}} \sum_{i=1}^{N} \log p(y_i|\mathbf{x}_i) \\ &= \underset{\mathbf{w},b}{\operatorname{argmin}} \sum_{i=1}^{N} \log(1 + e^{-y_i f(\mathbf{x}_i)}) \end{aligned}$$

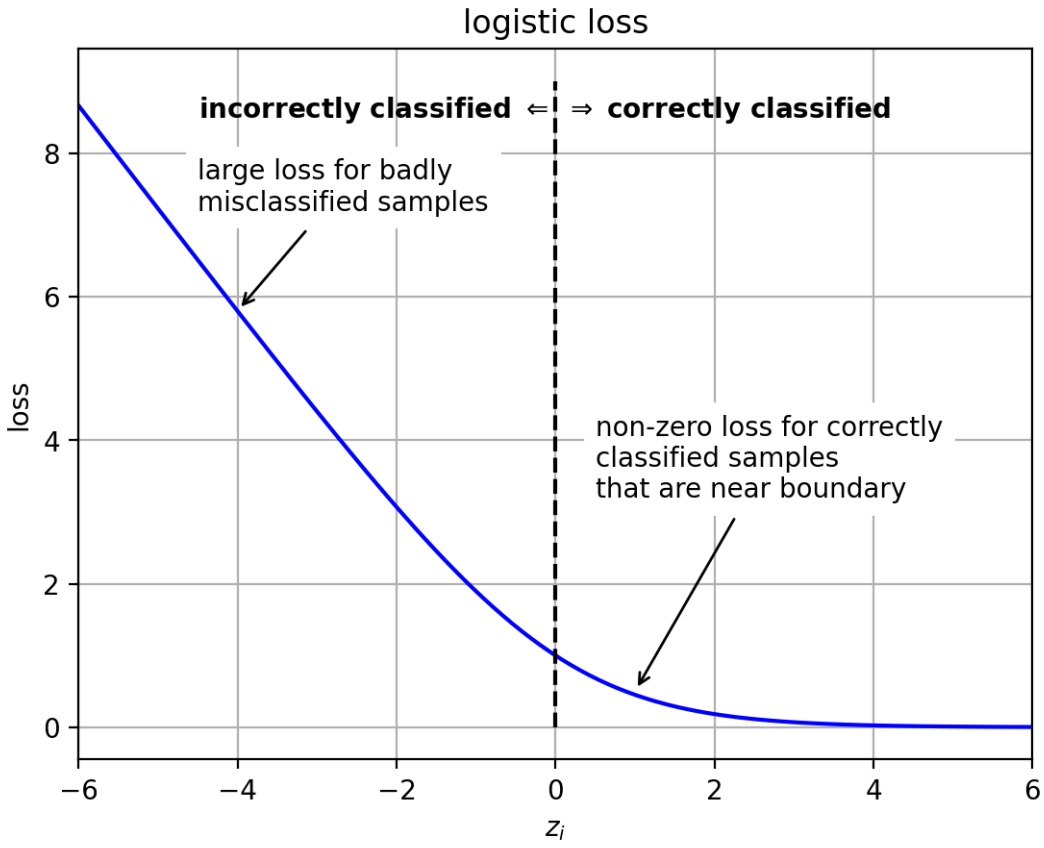
- the term on the right is a *data-fit term*
  - wants to make the parameters $(\mathbf{w}, b)$ to well fit the data.
  - Define $z_i = y_i f(\mathbf{x}_i)$
    - Interesting observation:
      - $z_i > 0$ when sample $\mathbf{x}_i$ is classified correctly
      - $z_i < 0$ when sample $\mathbf{x}_i$ is classified incorrectly
      - $z_i = 0$ when sample is on classifier boundary
  - logistic loss function: $L(z_i) = \log(1 + \exp(-z_i))$

[18]:
```
z = linspace(-6,6,100)
logloss = log(1+exp(-z)) / log(2)
lossfig = plt.figure()
plt.plot(z,logloss, 'b-')
plt.plot([0,0], [0,9], 'k--')
plt.text(0,8.5, "incorrectly classified $\\Leftarrow$ ", ha='right',␣
 ↪weight='bold')
plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left', weight='bold')
plt.annotate(text="large loss for badly\nmisclassified samples",
             xy=(-4,5.8), xytext=(-4.5,7.2),backgroundcolor='white',
            arrowprops=dict(arrowstyle="->"))
plt.annotate(text="non-zero loss for correctly\nclassified samples\nthat are␣
 ↪near boundary",
             xy=(1,0.5), xytext=(0.5,3.2), backgroundcolor='white',
            arrowprops=dict(arrowstyle="->"))
plt.xlabel('$z_i$'); plt.ylabel('loss')
plt.title('logistic loss');  plt.grid(True)
plt.xlim(-6,6)
plt.close()
```

```
<>:7: SyntaxWarning: invalid escape sequence '\R'
<>:7: SyntaxWarning: invalid escape sequence '\R'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/703837359.py:7:
SyntaxWarning: invalid escape sequence '\R'
  plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left',
weight='bold')
```

[19]:
```
lossfig
```

[19]:



12 Regularization

- to prevent *overfitting*, add a prior distribution on $\mathbf{w}$.
  - prefer solutions that are likely under the prior.

$$(\mathbf{w}^*, b^*) = \underset{\mathbf{w},b}{\operatorname{argmax}} \log p(\mathbf{w}) + \sum_{i=1}^{N} \log p(y_i|\mathbf{x}_i)$$

- assume Gaussian distribution on $\mathbf{w}$ with variance $C/2$
  - $p(\mathbf{w}) = N(\mathbf{w}|0, \frac{C}{2}\mathbf{I})$
    - small values of C keep $\mathbf{w}$ close to 0.
    - large values of C allow larger values of $\mathbf{w}$.
  - $\log p(\mathbf{w}) = -\frac{1}{C}\mathbf{w}^T\mathbf{w} + \text{constant}$

- Substituting,

$$(\mathbf{w}^*, b^*) = \underset{\mathbf{w},b}{\operatorname{argmin}} \frac{1}{C}\mathbf{w}^T\mathbf{w} + \sum_{i=1}^{N} \log(1 + \exp(-y_i f(\mathbf{x}_i))$$

- the first term is the *regularization term*
  - Note: $\mathbf{w}^T\mathbf{w} = \sum_{j=1}^{d} w_j^2$
  - penalty term that keeps entries in $\mathbf{w}$ from getting too large.

  - C is the regularization *hyperparameter*
    - larger C values apply less penalty on large $\mathbf{w} \rightarrow$ allow large values in $\mathbf{w}$.
    - smaller C values apply more penalty on large $\mathbf{w} \rightarrow$ discourage large values in $\mathbf{w}$.
- the second term is the *data fit term* - same as before.

13 Optimization

- **no closed-form solution**
  - use an iterative optimization algorithm to find the optimal solution
  - e.g., *gradient descent* - step downhill in each iteration.
    - $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{dE}{d\mathbf{w}}$
    - where E is the objective function
    - η is the *learning rate* (how far to step in each iteration).

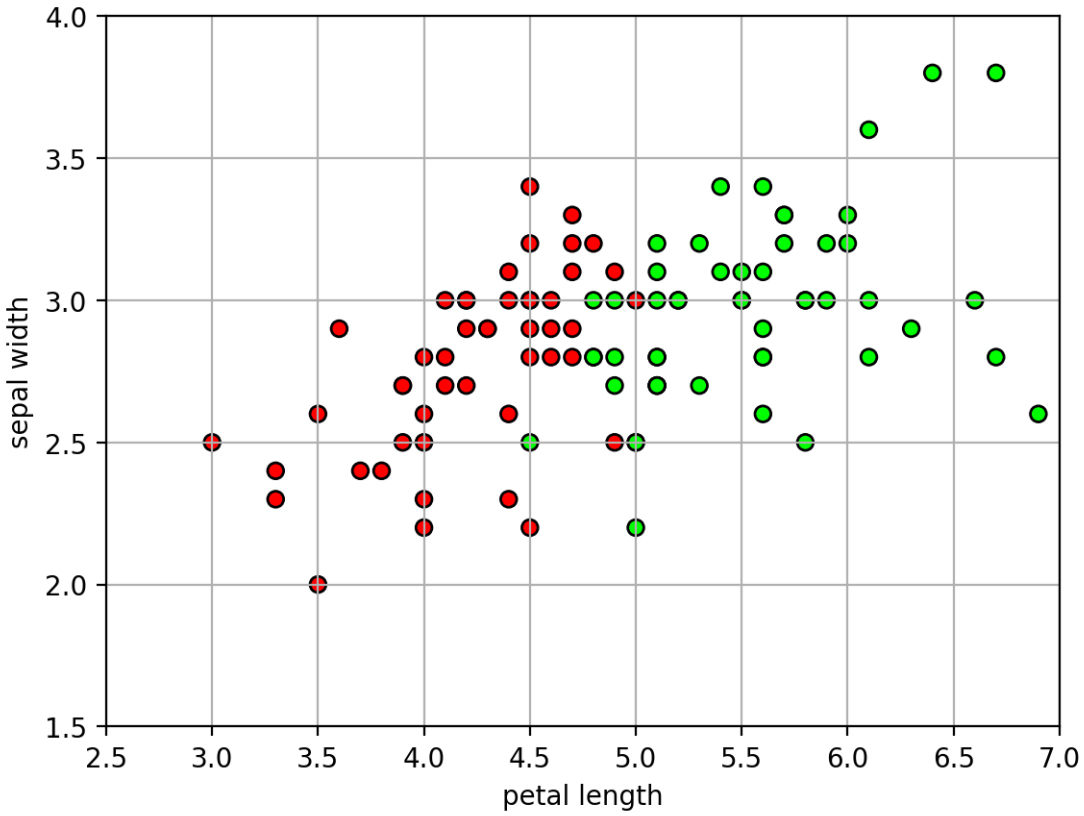
14 Example: Iris Data

[20]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris2.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (versicolor=1,␣
 ↪virginica=2)
                     #  --> automaticaly mapped to (-1, +1) when training␣
 ↪classifier

print(X.shape)
```

```
(100, 2)
```

[21]:
```
# a colormap for making the scatter plot: class -1 will be red, class +1 will␣
 ↪be green
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#FFFFFF", "#00FF00"])

axbox = [2.5, 7, 1.5, 4] # common axis range

# a function for setting a common plot
def irisaxis(axbox):
    plt.xlabel('petal length'); plt.ylabel('sepal width')
    plt.axis(axbox); plt.grid(True)
```

[22]:
```
# show the data
plt.figure()
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
irisaxis(axbox)
```



[23]:
```
# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(50, 2)
(50, 2)
```

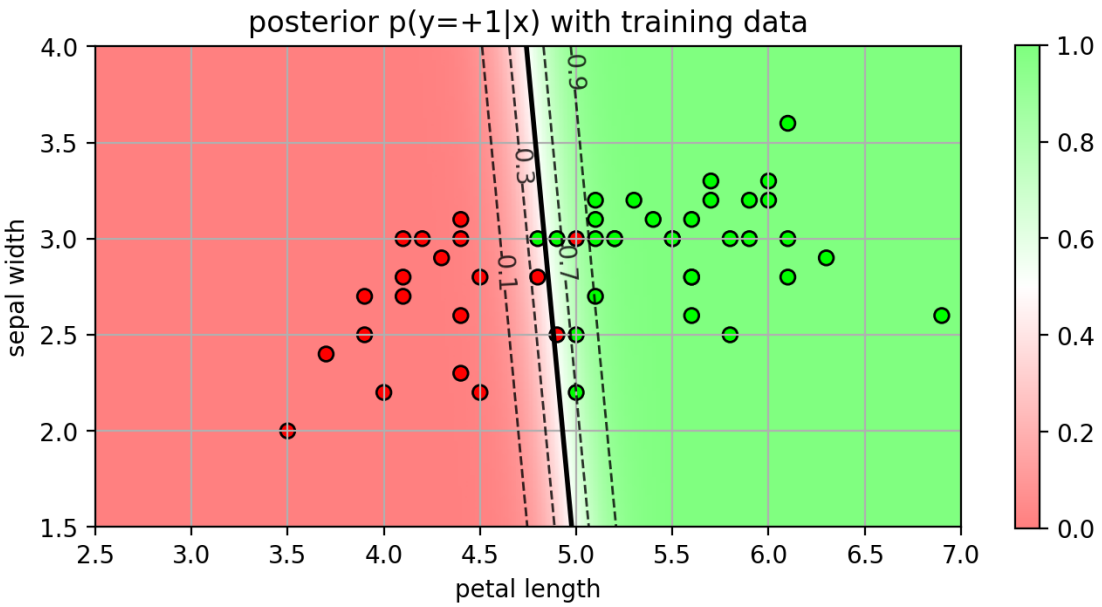
[24]:
```
# learn logistic regression classifier
# (C is a regularization hyperparameter)
logreg = linear_model.LogisticRegression(C=100)
logreg.fit(trainX, trainY)

print("w =", logreg.coef_)
print("b =", logreg.intercept_)
```

```
w = [[9.53947455 0.8988902 ]]
b = [-48.82195461]
```

- Equation:
  - $f(\mathbf{x}) = (9.51 * \text{petal\_length}) + (0.895 * \text{sepal\_width}) - 48.68$
- Interpretation:
  - large petal length makes $f(\mathbf{x})$ positive, so large petal length is associated with class +1.

[25]:
```
def plot_posterior(model, axbox, mycmap, showlabels=True):
    xr = [ linspace(axbox[0], axbox[1], 200),
           linspace(axbox[2], axbox[3], 200) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # calculate the posterior probability
    post = model.predict_proba(allpts)
    # extract the posterior for class 2, and reshape into a grid
    post1 = post[:,1].reshape(xgrid0.shape)

    # contour plot of the posterior and decision boundary
    plt.imshow(post1, origin='lower', extent=axbox, alpha=0.50, cmap=mycmap,␣
 ↪vmin=0.0, vmax=1.0)
    if showlabels:
        plt.colorbar(shrink=0.6)
    CS = plt.contour(xr[0], xr[1], post1, colors='k', linestyles='dashed',␣
 ↪levels=[0.1, 0.3, 0.7, 0.9], alpha=0.8, linewidths=1)
    if showlabels:
        #plt.colorbar(CS)
        plt.clabel(CS, inline=1, fontsize=10)
    plt.contour(xr[0], xr[1], post1, levels=[0.5], linewidths=2, colors='black')
    irisaxis(axbox)
```

[26]:
```
# show the posterior and training data
plt.figure(figsize=(8,6))
plot_posterior(logreg, axbox, mycmap)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.title('posterior p(y=+1|x) with training data');
```



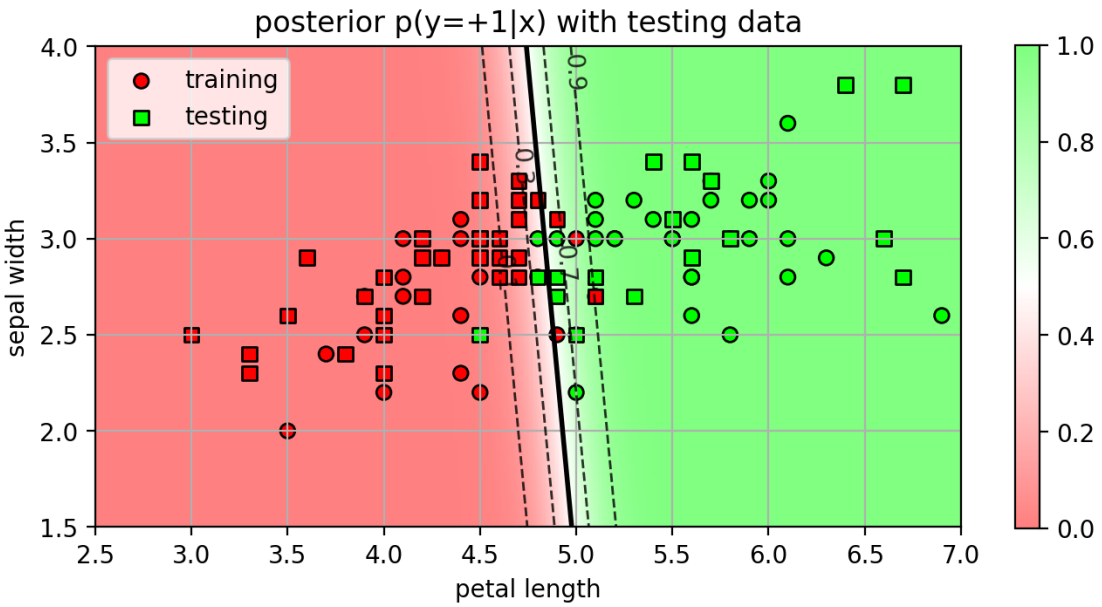
[27]:
```
# predict from the model
predY = logreg.predict(testX)

# calculate accuracy
acc = metrics.accuracy_score(testY, predY)
print("test accuracy =", acc)
```

```
test accuracy = 0.92
```

[28]:
```
# show the posterior and training data
postfig = plt.figure(figsize=(8,6))
plot_posterior(logreg, axbox, mycmap)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, marker="o",␣
 ↪label="training", edgecolors='k')
plt.scatter(testX[:,0], testX[:,1], c=testY, cmap=mycmap, marker="s",␣
 ↪label="testing", edgecolors='k')
plt.title('posterior p(y=+1|x) with testing data');
plt.legend(loc=0);
plt.close()
```

[29]:
```
postfig
```

[29]:



15 Selecting the regularization hyperparameter

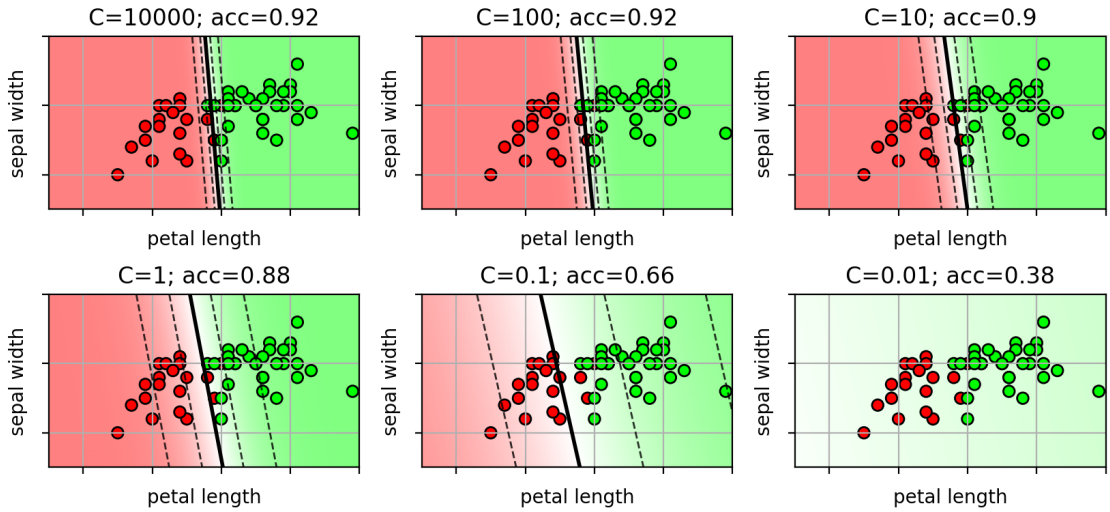
- the regularization hyperparameter C has a large effect on the decision boundary and the accuracy.
  - larger C makes the classifier more confident (posterior probabilities saturate to 0 and 1)
    - more likely to overfit
  - smaller C makes the classifer less confident (wider range of posterior probabilities).
    - less likely to overfit
- How to set the value of C?

[30]:
```
lrC = plt.figure(figsize=(10,4.5))

allC = [10000,100, 10, 1, 0.1, 0.01]
for (myCind,myC) in enumerate(allC):
    logreg = linear_model.LogisticRegression(C=myC)
    logreg.fit(trainX, trainY)

    # predict from the model
    predY = logreg.predict(testX)

    # calculate accuracy
    acc = metrics.accuracy_score(testY, predY)

    plt.subplot(2,3,myCind+1)
    plot_posterior(logreg, axbox, mycmap, showlabels=False)
    plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
```

```
    plt.title('C='+str(myC)+"; acc="+str(acc));
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
plt.close()
```

[31]:
```
lrC
```

[31]:



16 Cross-validation

- Use *cross-validation* on the training set to select the best value of C.
  - Run many experiments on the training set to see which parameters work on different versions of the data.
    - Split the data into batches of training and validation data.
    - Try a range of C values on each split.
    - Pick the value that works best over all splits.
- **Procedure**
  1. select a range of C values to try
  2. Repeat K times
  3. Split the training set into training data and validation data
  4. Learn a classifier for each value of C
  5. Record the accuracy on the validation data for each C
  6. Select the value of C that has the highest average accuracy over all K folds.
  7. Retrain the classifier using all data and the selected C.
- scikit-learn already has built-in `cross_validation` module (more later).
- for logistic regression, use *LogisticRegressionCV* class

[32]:
```
# learn logistic regression classifier using CV
#  Cs is an array of possible C values
```

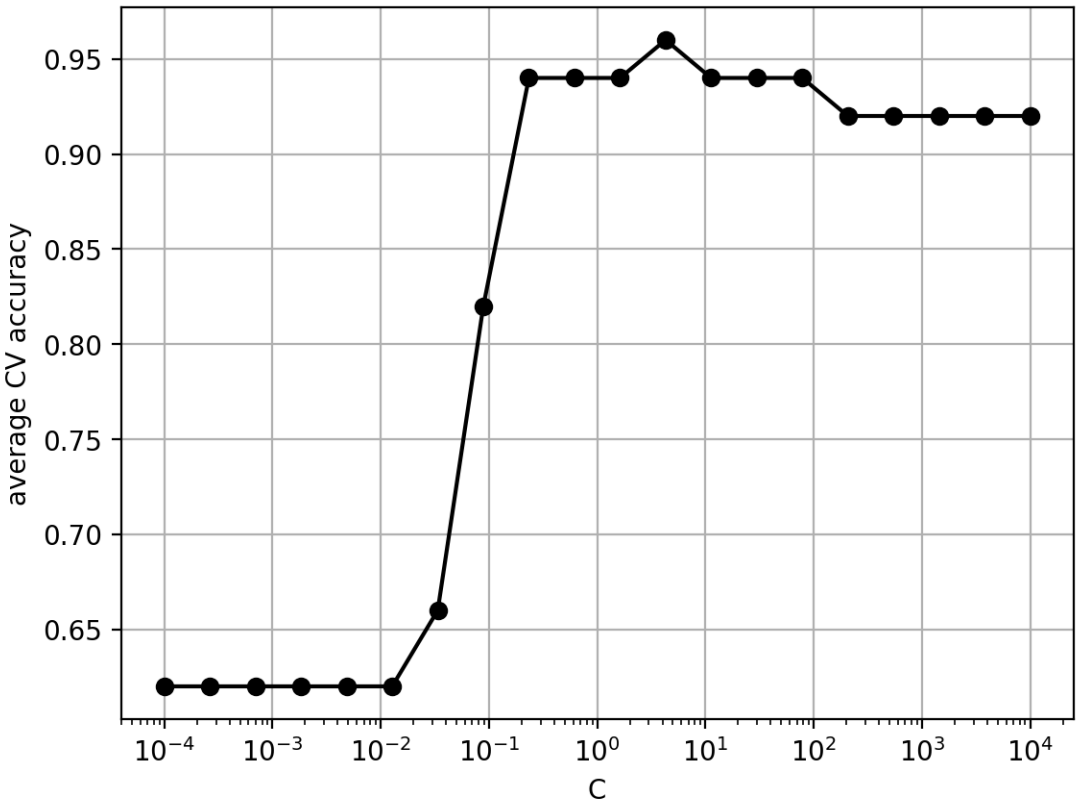
```
#  cv is the number of folds
#  n_jobs=-1 means run in parallel with all cores
logreg = linear_model.LogisticRegressionCV(Cs=logspace(-4,4,20), cv=5,␣
 ↪n_jobs=-1)
logreg.fit(trainX, trainY)

print("w=", logreg.coef_)
print("b=", logreg.intercept_)

# predict from the model
predY = logreg.predict(testX)

# calculate accuracy
acc = metrics.accuracy_score(testY, predY)
print("test accuracy=", acc)
```

```
w= [[4.62056477 0.72461222]]
b= [-24.25601763]
test accuracy= 0.9
```

17 Which C was selected?

[33]:
```
print("C =", logreg.C_)
# calculate the average score for each C
avgscores = mean(logreg.scores_[2],0)  # 2 is the class label
plt.semilogx(logreg.Cs_, avgscores, 'ko-')
plt.xlabel('C'); plt.ylabel('average CV accuracy'); plt.grid(True);
```

```
C = [4.2813324]
```



18 Multi-class classification

- So far, we have only learned a classifier for 2 classes (+1, -1)
  - called a **binary classifier**
- For more than 2 classes, split the problem up into several binary classifier problems.
  - **1-vs-rest**
    - *Training:* for each class, train a classifier for that class versus the other classes.
      - For example, if there are 3 classes, then train 3 binary classifiers: 1 vs {2,3}; 2 vs {1,3}; 3 vs {1,2}
    - *Prediction:* calculate probability for each binary classifier. Select the class with highest probability.

19 Example on 3-class Iris data

[34]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris3.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
```

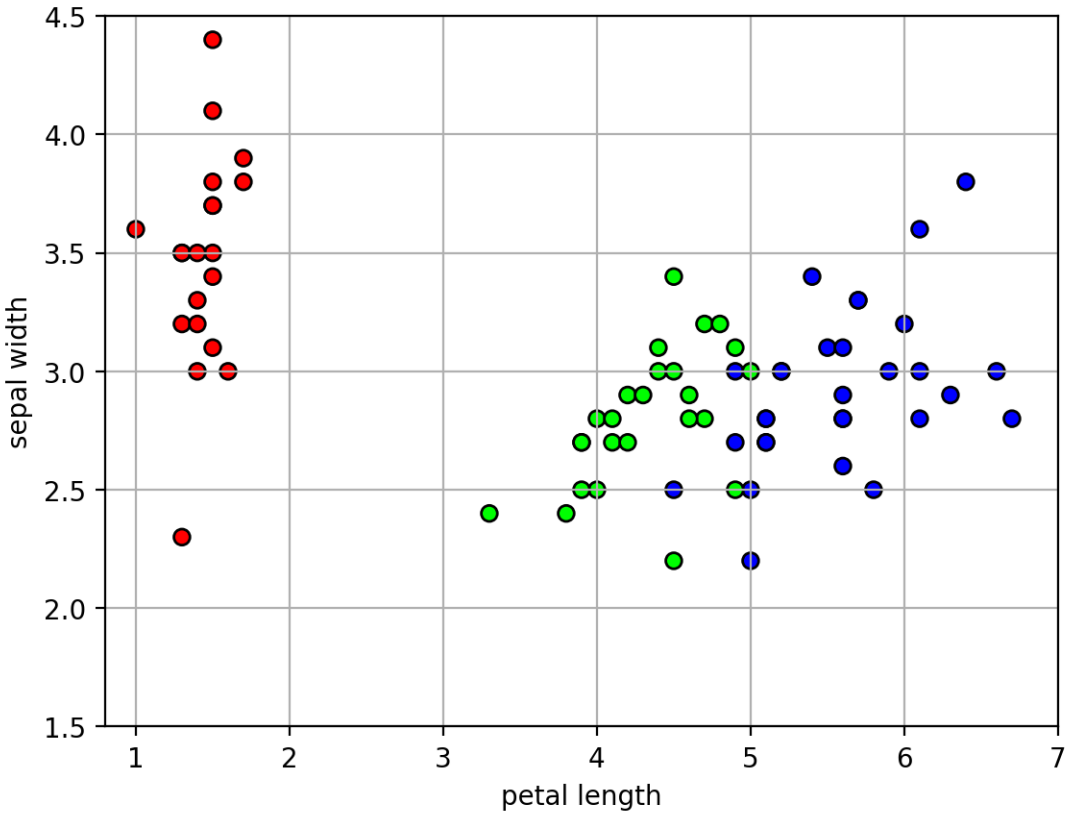
```
Y = irisdata[:,2]    # the third column is the class label (setosa=0,␣
 ↪versicolor=1, virginica=2)

print(X.shape)
```

```
(150, 2)
```

[35]:
```
# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(75, 2)
(75, 2)
```

[36]:
```
# look at training data

axbox3 = [0.8, 7, 1.5, 4.5]
# make a colormap for viewing 3 classes
mycmap3 = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#00FF00", "#0000FF"])

plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap3, edgecolors='k')
plt.axis(axbox3); plt.grid(True);
plt.xlabel('petal length'); plt.ylabel('sepal width');
```



[37]:
```
# learn logistic regression classifier (one-vs-all)
mlogreg = linear_model.LogisticRegression(C=10, multi_class='ovr')
mlogreg.fit(trainX, trainY)

# now contains 3 hyperplanes and 3 bias terms (one for each class)
print("w=", mlogreg.coef_)
print("b=", mlogreg.intercept_)

# predict from the model
predY = mlogreg.predict(testX)

# calculate accuracy
acc = metrics.accuracy_score(testY, predY)
print("test accuracy=", acc)
```

```
w= [[-3.56993402  1.08723834]
 [-0.03184394 -2.23108274]
 [ 5.42003067 -1.68985998]]
b= [  6.41058825   6.12262666 -21.81518434]
test accuracy= 0.9733333333333334
```

```
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
```

```
packages/sklearn/linear_model/_logistic.py:1281: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. Use
OneVsRestClassifier(LogisticRegression(..)) instead. Leave it to its default
value to avoid this warning.
  warnings.warn(
```

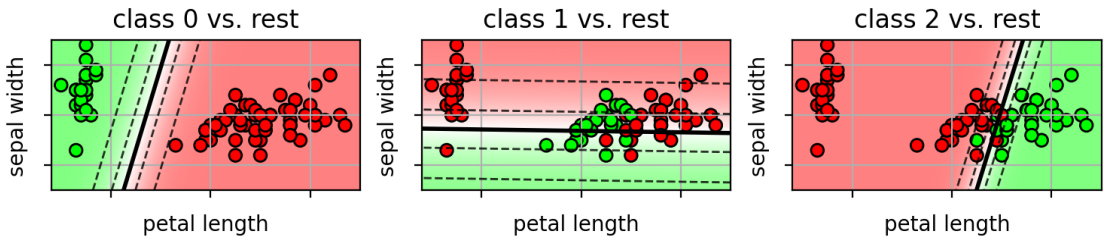
[38]:
```
def plot_1vr_classifiers(logreg, mlogreg, axbox, mycmap, trainX, trainY,␣
 ↪titstr):
    # plot each classifier (assume 3)
    for i in range(3):
        plt.subplot(1,3,i+1)
        # make binary model
        ilogreg = linear_model.LogisticRegression(logreg)
        ilogreg.coef_ = mlogreg.coef_[i,:].reshape(1,2)
        ilogreg.intercept_ = mlogreg.intercept_[i].reshape(1,)
        ilogreg.classes_ = array([1, 2])
        itrainY = (trainY == i)

        plot_posterior(ilogreg, axbox, mycmap, showlabels=False)
        plt.scatter(trainX[:,0], trainX[:,1], c=itrainY, cmap=mycmap,␣
 ↪edgecolors='k')

        plt.gca().xaxis.set_ticklabels([])
        plt.gca().yaxis.set_ticklabels([])
        plt.title(titstr.format(i))

mlrfig = plt.figure(figsize=(9,6))
plot_1vr_classifiers(logreg, mlogreg, axbox3, mycmap, trainX, trainY, "class {}␣
 ↪vs. rest")
plt.close()
```

- the individual 1-vs-rest binary classifiers

[39]:
```
print("w=", mlogreg.coef_)
print("b=", mlogreg.intercept_)

mlrfig
```

```
w= [[-3.56993402  1.08723834]
 [-0.03184394 -2.23108274]
 [ 5.42003067 -1.68985998]]
b= [  6.41058825   6.12262666 -21.81518434]
```

[39]:

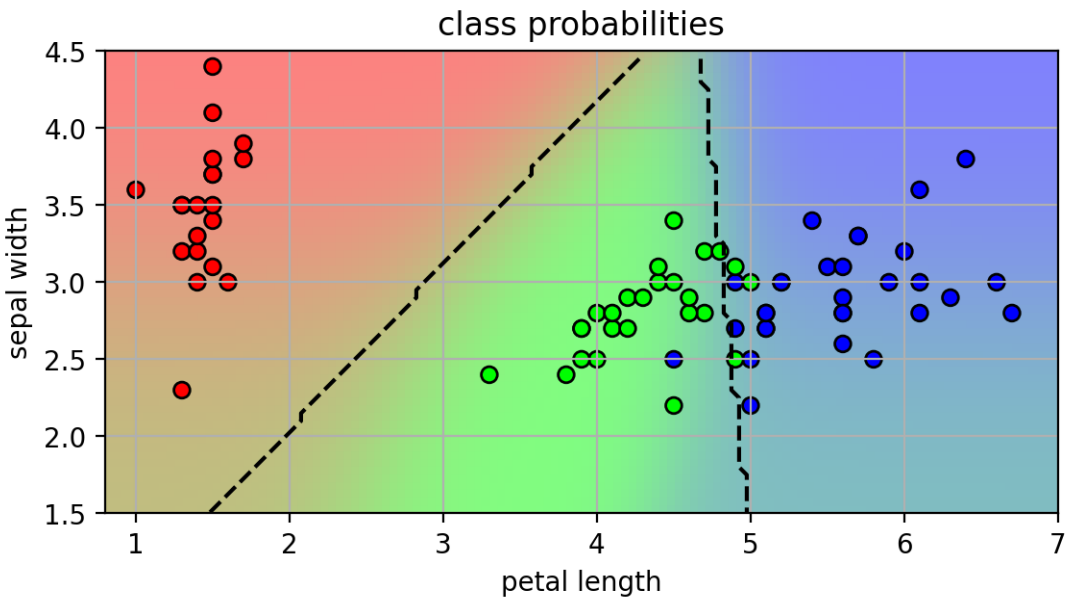


[40]:
```
def plot_posterior3(model, axbox, mycmap):
    xr = [ arange(0.8,7,0.05) , arange(1.5, 4.5, 0.05) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # predict probabilities
    Z = model.predict_proba(allpts)
    P = model.predict(allpts)

    # use probabilities as RGB color
    ZZ = Z.reshape((len(xr[1]), len(xr[0]), 3))

    plt.imshow(ZZ, origin='lower', extent=axbox, alpha=0.50)
    plt.contour(xr[0], xr[1], P.reshape(xgrid0.shape), levels=[0.5,1.5,2.5],␣
 ↪linestyles='dashed', colors='black')
    irisaxis(axbox)

lr3class = plt.figure()
plot_posterior3(mlogreg, axbox3, mycmap3)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap3, edgecolors='k')
plt.title('class probabilities');
plt.close()
```

- the final classifier, combining all 1 vs rest classifiers

[41]:
```
lr3class
```

[41]:



20 Multiclass logistic regression

- Another way to get a multi-class classifier is to define a multi-class objective.
  - One weight vector $\mathbf{w}_c$ for each class c.
  - linear function for each class, $f_c(\mathbf{x}) = \mathbf{w}_c^T \mathbf{x}$.
- Define probabilities with **softmax** function
  - analogous to sigmoid function for binary logistic regression.

$$p(y = c|\mathbf{x}) = \frac{e^{f_c(\mathbf{x})}}{e^{f_1(\mathbf{x})} + \cdots + e^{f_K(\mathbf{x})}}$$

  - The class with largest response of $f_c(\mathbf{x})$ will have the highest probability.

[42]:
```
f1 = linspace(-5,5,101)
f2 = linspace(-5,5,101)

sfmfig = plt.figure(figsize=(10,4))
for i,f3 in enumerate([-1,3]):
    xgrid0, xgrid1 = meshgrid(f1, f2)
    plt.subplot(1,2,i+1)
    p = exp(xgrid0) / (exp(xgrid0) + exp(xgrid1)+exp(f3))

    plt.imshow(p, origin='lower', extent=[-5,5,-5,5], vmin=0, vmax=1)
    plt.colorbar(shrink=0.8)
    CS = plt.contour(f1, f2, p, colors='k', linestyles='dashed', levels=[0.1, 0.
↪3, 0.5, 0.7, 0.9], alpha=0.8, linewidths=1)

    plt.clabel(CS, inline=1, fontsize=10)
```

```
    plt.xlabel('$f_1$')
    plt.ylabel('$f_2$')
    plt.title('$p(y=1|\mathbf{x})$, $f_3=' + str(f3) + '$')
plt.close()
```

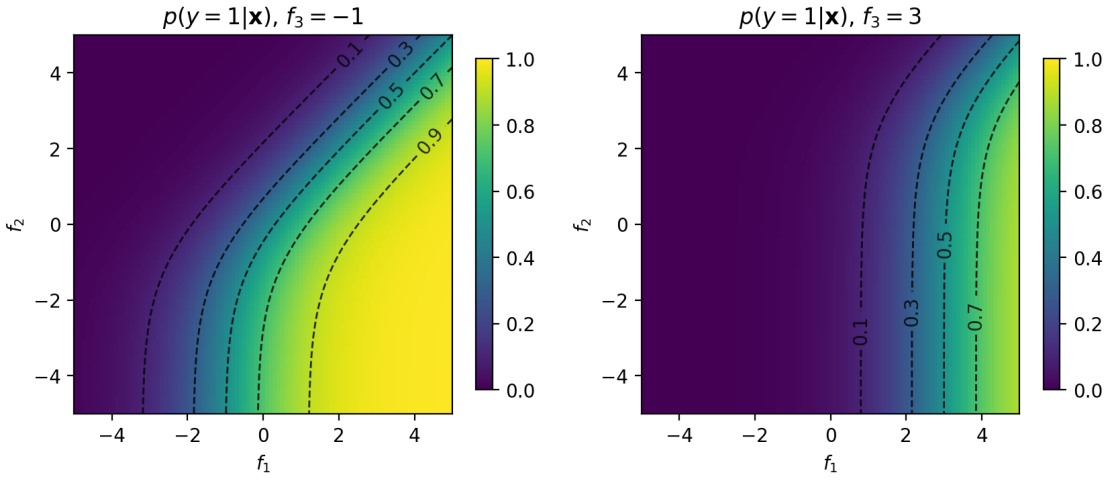
```
<>:18: SyntaxWarning: invalid escape sequence '\m'
<>:18: SyntaxWarning: invalid escape sequence '\m'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43280/696709212.py:18
: SyntaxWarning: invalid escape sequence '\m'
  plt.title('$p(y=1|\mathbf{x})$, $f_3=' + str(f3) + '$')
```

- Example with $K = 3$.

$$p(y = 1|\mathbf{x}) = \frac{e^{f_1(\mathbf{x})}}{e^{f_1(\mathbf{x})} + e^{f_2(\mathbf{x})} + e^{f_3(\mathbf{x})}}$$

[43]:
```
sfmfig
```

[43]:



21 Parameter estimation

- Estimate the $\{\mathbf{w}_j\}$ parameters using MLE.
- Let $(\mathbf{x}, \mathbf{y})$ be a data sample pair:
  - $\mathbf{x}$ feature vector.
  - $\mathbf{y} = [y_1, \cdots, y_K]$ is a one-hot vector, where $y_c = 1$ when class c, and 0 otherwise.

- Data likelihood of $(\mathbf{x}, \mathbf{y})$.

$$\text{likelihood:} \qquad p(\mathbf{y}|\mathbf{x}) = \prod_{j=1}^{K} p(y=j|\mathbf{x})^{y_j}$$

$$\text{log-likelihood:} \qquad \log p(\mathbf{y}|\mathbf{x}) = \sum_{j=1}^{K} y_j \log p(y=j|\mathbf{x})$$

$$\text{negative log-likelihood:} \qquad -\log p(\mathbf{y}|\mathbf{x}) = -\sum_{j=1}^{K} y_j \log p(y=j|\mathbf{x})$$

  - equivalent to the *cross-entropy loss*

- Given dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N}$
  - maximize the data log-likelihood:

$$\max_{\{\mathbf{w}_j\}} \sum_{i=1}^{N} \log p(\mathbf{y}_i|\mathbf{x}_i) = \max_{\{\mathbf{w}_j\}} \sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \log p(y=j|\mathbf{x}_i)$$

  - i.e., minimize the cross-entropy loss

[44]:
```
# learn logistic regression classifier
mlogreg = linear_model.LogisticRegression(C=10,
            multi_class='multinomial')
            # use multi-class and corresponding solver
mlogreg.fit(trainX, trainY)

# now contains 3 hyperplanes and 3 bias terms (one for each class)
print("w=", mlogreg.coef_)
print("b=", mlogreg.intercept_)

# predict from the model
predY = mlogreg.predict(testX)

# calculate accuracy
acc = metrics.accuracy_score(testY, predY)
print("test accuracy=", acc)
```

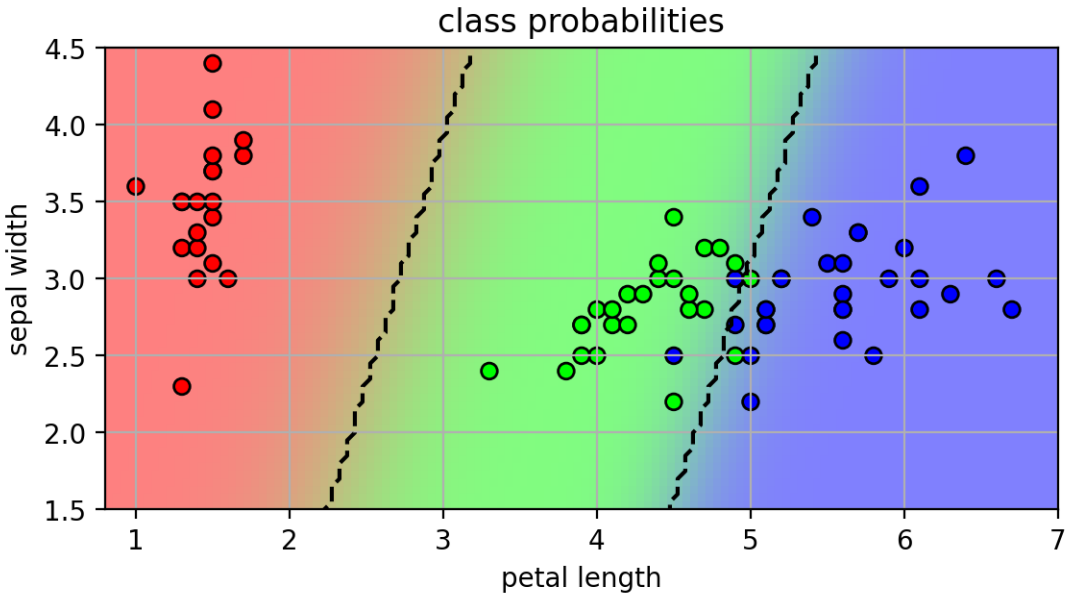
```
w= [[-4.13160888  1.30538269]
 [-0.71650483  0.23668666]
 [ 4.84811371 -1.54206935]]
b= [ 11.46619303   5.40363007 -16.8698231 ]
test accuracy= 0.9733333333333334
```

```
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1272: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

[45]:
```
lr3classm = plt.figure()
plot_posterior3(mlogreg, axbox3, mycmap3)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap3, edgecolors='k')
plt.title('class probabilities');
plt.close()
```

[46]:
```
lr3classm
```

[46]:



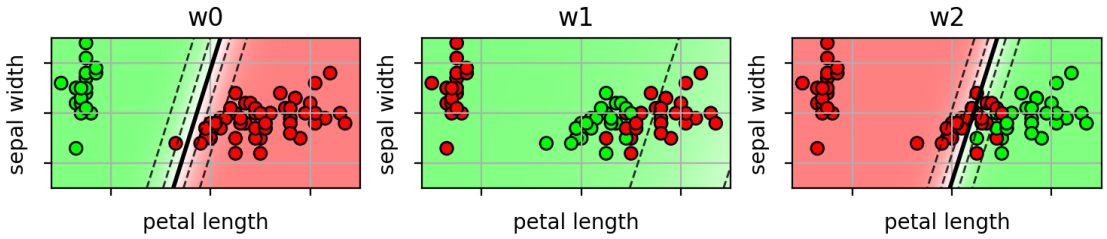
[47]:
```
lr31vr = plt.figure(figsize=(9,6))
plot_1vr_classifiers(logreg, mlogreg, axbox3, mycmap, trainX, trainY, "w{}")
plt.close()
```

- individual weight vectors work together to partition the space

[48]:
```
print("w=", mlogreg.coef_)
print("b=", mlogreg.intercept_)

lr31vr
```

```
w= [[-4.13160888  1.30538269]
 [-0.71650483  0.23668666]
 [ 4.84811371 -1.54206935]]
b= [ 11.46619303   5.40363007 -16.8698231 ]
```

[48]:



22 Logistic Regression Summary

- **Classifier:**
  - linear function $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$
  - Given a feature vector $\mathbf{x}$, the probability of a class is:
    - $p(y = +1|\mathbf{x}) = \sigma(f(\mathbf{x}))$
    - $p(y = -1|\mathbf{x}) = 1 - \sigma(f(\mathbf{x}))$
    - *sigmoid* function: $\sigma(z) = \frac{1}{1+e^{-z}}$
  - logistic loss function: $L(z) = \log(1 + \exp(-z))$
- **Training:**
  - Maximize the likelihood of the training data.
  - Use regularization to prevent overfitting.
    - Use cross-validation to pick the regularization hyperparameter C.
- **Classification:**
  - Given a new sample $\mathbf{x}^*$:
    - pick class with highest probability $p(y|\mathbf{x}^*)$:

$$y^* = \begin{cases} +1, p(y = +1|\mathbf{x}^*) > p(y = -1|\mathbf{x}^*) \\ -1, \text{otherwise} \end{cases}$$

    - alternatively, just use $f(\mathbf{x}^*)$

$$y^* = \begin{cases} +1, f(\mathbf{x}^*) > 0 \\ -1, \text{otherwise} \end{cases} = \text{sign}(f(\mathbf{x}_*))$$

- **Extend to multi-class:**
  - K linear functions, one for each class.
  - compute probability using softmax function
  - MLE equivalent to cross-entropy loss

Lecture3c

September 14, 2025

1 CS5489 - Machine Learning

2 Lecture 3c - Classification Summary

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

1. Discriminative linear classifiers
2. Logistic regression
3. SVM & Kernel SVM
4. Summary

[1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
```

[2]:
```
def drawstump(fdim, fthresh, fdir='gt', poscol=None, negcol=None, lw=2,␣
 ↪ls='k-'):
    # fdim = dimension
    # fthresh = threshold
    # fdir = direction (gt, lt)

    if fdir == 'lt':
        # swap colors
        tmp = poscol
        poscol = negcol
        negcol = tmp

    # assume fdim=0
    polyxn = [fthresh, fthresh, -30, -30]
```

```
    polyyn = [30, -30, -30, 30]

    polyxp = [fthresh, fthresh, 30, 30]
    polyyp = [30, -30, -30, 30]

    # fill positive half-space or neg space
    if (poscol):
        if fdim==0:
            plt.fill(polyxp, polyyp, poscol, alpha=0.2)
        else:
            plt.fill(polyyp, polyxp, poscol, alpha=0.2)

    if (negcol):
        if fdim==0:
            plt.fill(polyxn, polyyn, negcol, alpha=0.2)
        else:
            plt.fill(polyyn, polyxn, negcol, alpha=0.2)

    # plot line
    if fdim==0:
        plt.plot(polyxp[0:2], polyyp[0:2], ls, lw=lw)
    else:
        plt.plot(polyyp[0:2], polyxp[0:2], ls, lw=lw)

def drawplane(w, b=None, c=None, wlabel=None, poscol=None, negcol=None, lw=2,␣
 ↪ls='k-'):
    # w^Tx + b = 0
    # w0 x0 + w1 x1 + b = 0
    # x1 = -w0/w1 x0 - b / w1

    # OR
    # w^T (x-c) = 0 = w^Tx - w^Tc  --> b = -w^Tc
    if c != None:
        b = -sum(w*c)

    # the line
    if (abs(w[0])>abs(w[1])):   # vertical line
        x0 = array([-30,30])
        x1 = -w[0]/w[1] * x0 - b / w[1]
    else:                       # horizontal line
        x1 = array([-30,30])
        x0 = -w[1]/w[0] * x1 - b / w[0]

    # fill positive half-space or neg space
    if (poscol):
        polyx = [x0[0], x0[-1], x0[-1], x0[0]]
        polyy = [x1[0], x1[-1], x1[0], x1[0]]
```

```
        plt.fill(polyx, polyy, poscol, alpha=0.2)

    if (negcol):
        polyx = [x0[0], x0[-1], x0[0], x0[0]]
        polyy = [x1[0], x1[-1], x1[-1], x1[0]]
        plt.fill(polyx, polyy, negcol, alpha=0.2)

    # plot line
    lineplt, = plt.plot(x0, x1, ls, lw=lw)

    # the w
    if (wlabel):
        xp = array([0, -b/w[1]])
        xpw = xp+w
        plt.arrow(xp[0], xp[1], w[0], w[1], width=0.01)
        plt.text(xpw[0]-0.5, xpw[1], wlabel)
    return lineplt
```

[3]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris2.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (versicolor=1,␣
 ↪virginica=2)

print(X.shape)

# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(100, 2)
(50, 2)
(50, 2)
```

[4]:
```
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#FFFFFF", "#00FF00"])

axbox = [2.5, 7, 1.5, 4]
```

4 Feature Pre-processing

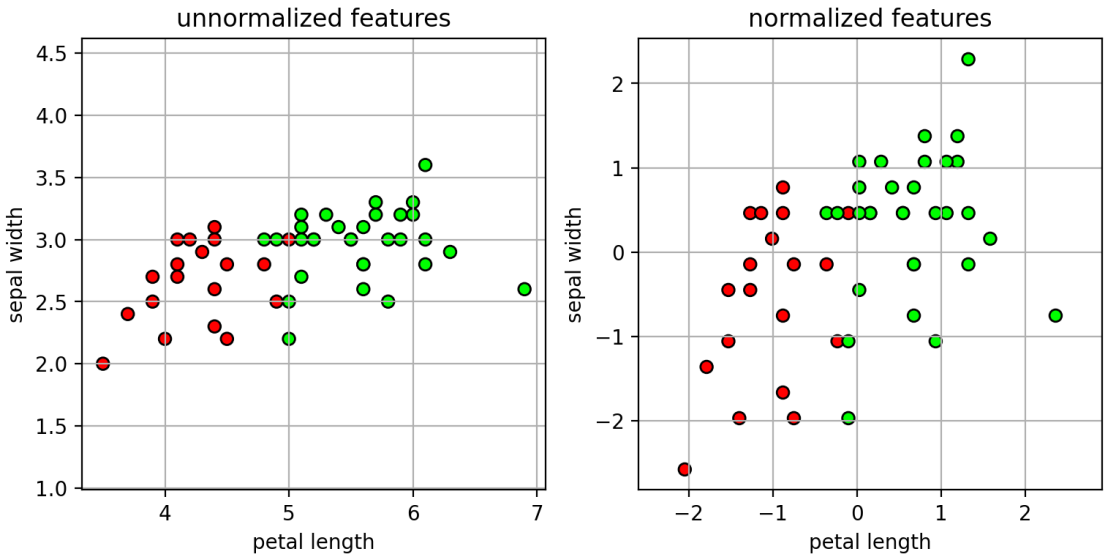
- Some classifiers, such as SVM and LR, are sensitive to the scale of the feature values.
  - feature dimensions with larger values may dominate the objective function.
- Common practice is to *standardize* or *normalize* each feature dimension before learning the classifier.
  - Two Methods...
- **Method 1:** scale each feature dimension so the mean is 0 and variance is 1.
  - $\tilde{x}_d = \frac{1}{s}(x_d - m)$
  - s is the standard deviation of feature values.
  - m is the mean of the feature values.
- **NOTE:** the parameters for scaling the features should be estimated from the training set!
  - same scaling is applied to the test set.

[5]:
```
# using the iris data
scaler = preprocessing.StandardScaler()  # make scaling object
trainXn = scaler.fit_transform(trainX)   # use training data to fit scaling␣
 ↪parameters
testXn  = scaler.transform(testX)        # apply scaling to test data
```

[6]:
```
nfig1 = plt.figure(figsize=(9,4))
axbox2 = [-3, 3, -3, 3]

plt.subplot(1,2,1)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.axis(axbox); plt.grid(True)
plt.axis('equal')
plt.title("unnormalized features")

plt.subplot(1,2,2)
plt.scatter(trainXn[:,0], trainXn[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.axis(axbox2); plt.grid(True)
plt.axis('equal')
plt.title("normalized features")
plt.close()
```

[7]:
```
nfig1
```

[7]:



- **Method 2:** scale features to a fixed range, -1 to 1.
  - $\tilde{x}_d = 2 * (x_d - min)/(max - min) - 1$
  - max and min are the maximum and minimum features values.

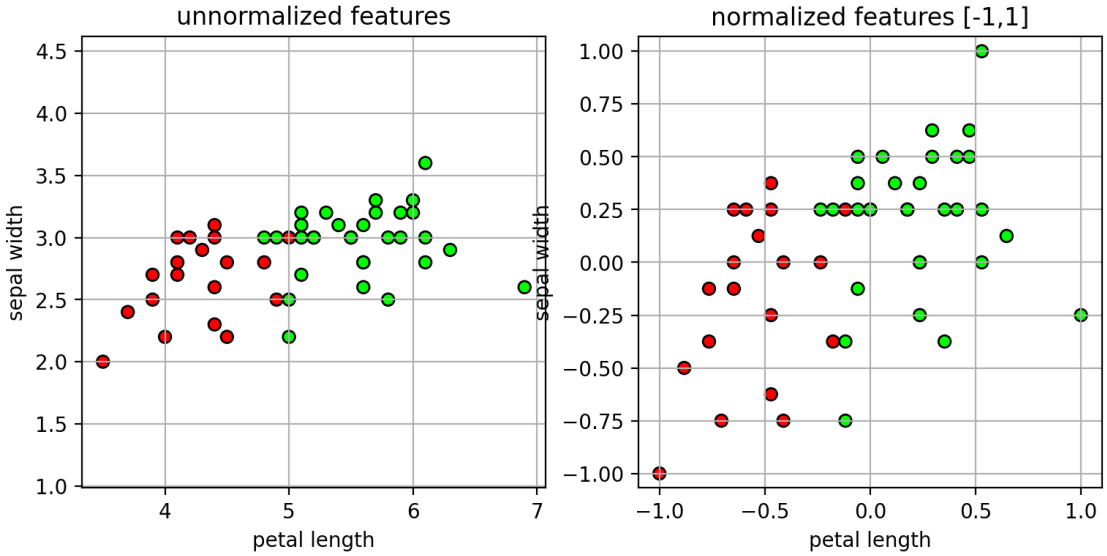
[8]:
```
# using the iris data
scaler = preprocessing.MinMaxScaler(feature_range=(-1,1))     # make scaling␣
 ↪object
trainXn = scaler.fit_transform(trainX)    # use training data to fit scaling␣
 ↪parameters
testXn  = scaler.transform(testX)         # apply scaling to test data
```

[9]:
```
nfig2 = plt.figure(figsize=(9,4))
axbox2 = [-1, 1, -1, 1]

plt.subplot(1,2,1)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')

plt.axis(axbox); plt.grid(True)
plt.axis('equal')
plt.title("unnormalized features")

plt.subplot(1,2,2)
plt.scatter(trainXn[:,0], trainXn[:,1], c=trainY, cmap=mycmap, edgecolors='k')
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.axis(axbox2); plt.grid(True)
plt.axis('equal')
plt.title("normalized features [-1,1]")
```

```
plt.close()
```

[10]:
```
nfig2
```

[10]:



5 Data Representation and Feature Engineering

- How to represent data as a vector of numbers?
  - the encoding of the data into a feature vector should make sense
  - inner-products or distances calculated between feature vectors should be meaningful in terms of the data.
- Categorical variables
  - Example: x has 3 possible category labels: cat, dog, horse
  - We could encode this as: $x = 0$, $x = 1$, and $x = 2$.
    - Suppose we have two data points: $x = cat$, $x' = horse$.
    - What is the meaning of $x * x' = 2$?

6 One-hot encoding

- encode a categorical variable as a vector of ones and zeros
  - if there are K categories, then the vector is K dimensions.
- Example:
  - x=cat → x=[1 0 0]
  - x=dog → x=[0 1 0]
  - x=horse → x=[0 0 1]

[11]:
```
# one-hot encoding example
X = [['cat'], ['dog'], ['cat'], ['bird'], ['dog']]  # each row is a sample
```

```
ohe = preprocessing.OneHotEncoder(sparse_output=False)
ohe.fit(X)          # map the categories to one-hot vectors
print(ohe.categories_)
ohe.transform(X)    # transform to one-hot-encoding
```

```
[array(['bird', 'cat', 'dog'], dtype=object)]
```

[11]:
```
array([[0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.]])
```

7 Binning

- encode a real value as a vector of ones and zeros
  - assign each feature value to a bin, and then use one-hot-encoding

[12]:
```
# example
X = [[-3], [0.5], [1.5], [2.5]]  # the data
bins = [-2,-1,0,1,2]       # define the bin edges

# map from value to bin number
Xbins = digitize(X, bins=bins)

# map from bin number (0..5) to 0-1 vector
ohe = preprocessing.OneHotEncoder(categories=[arange(6)], sparse_output=False)
ohe.fit(Xbins)
ohe.transform(Xbins)
```

[12]:
```
array([[1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 1.]])
```

8 Data transformations - polynomials

- Represent interactions between features using polynomials
- Example:
  - 2nd-degree polynomial models pair-wise interactions
    - $[x_1, x_2] \rightarrow [x_1^2, x_1x_2, x_2^2]$
  - Combine with other degrees:
    - $[x_1, x_2] \rightarrow [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$

[13]:
```
X = [[0,1], [1,2], [3,4]]
pf = preprocessing.PolynomialFeatures(degree=2)
pf.fit(X)
```

```
pf.transform(X)
```

[13]:
```
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  1.,  2.,  1.,  2.,  4.],
       [ 1.,  3.,  4.,  9., 12., 16.]])
```

9 Data transformations - univariate

- Apply a non-linear transformation to the feature
  - e.g., x $\rightarrow$ log(x)
  - useful if the dynamic range of x is very large

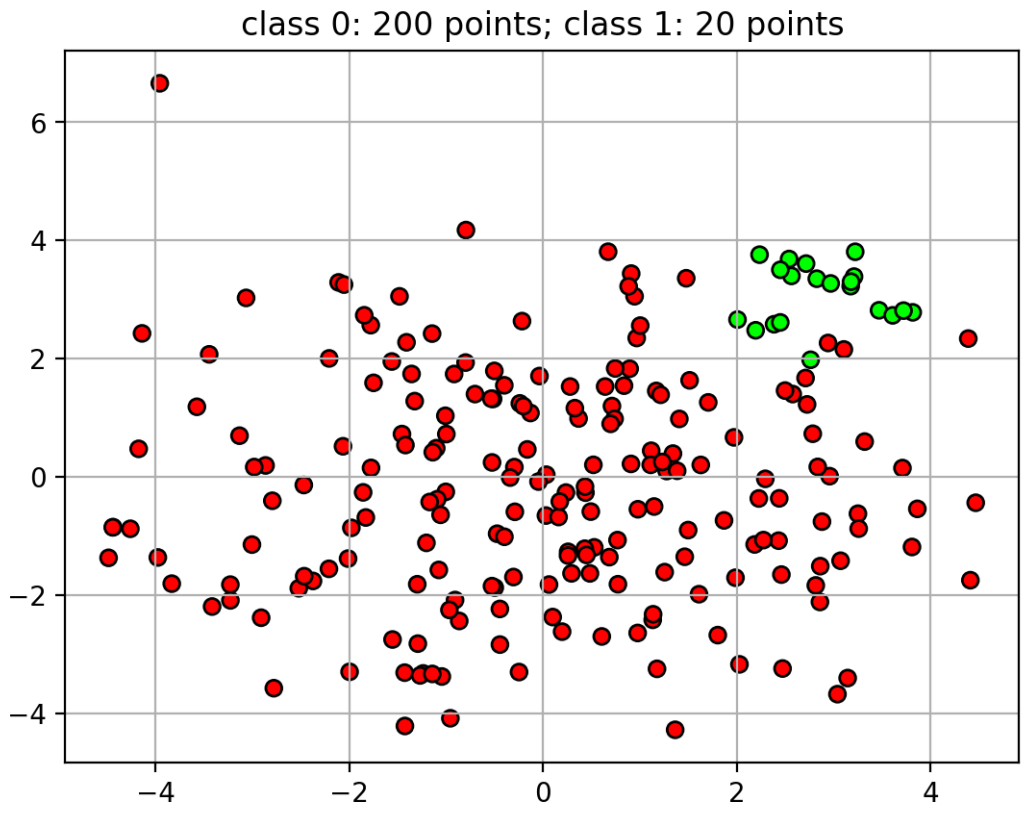
10 Unbalanced Data

- For some classification tasks that data will be unbalanced
  - many more examples in one class than the other.
- **Example:** detecting credit card fraud
  - credit card fraud is rare
    - 50 examples of fraud, 5000 examples of legitimate transactions.

[14]:
```
# generate random data
X,Y = datasets.make_blobs(n_samples=200,
         centers=[[0,0]], cluster_std=2, n_features=2, random_state=4487)
X2,Y2 = datasets.make_blobs(n_samples=20,
         centers=[[3,3]], cluster_std=0.5, n_features=2, random_state=4487)

X = r_[X,X2]
Y = r_[Y,Y2+1]

udatafig = plt.figure()
plt.scatter(X[:,0],X[:,1],c=Y,cmap=mycmap, edgecolors='k')
plt.grid(True)
plt.title('class 0: 200 points; class 1: 20 points')
plt.close()
```

[15]:
```
udatafig
```

[15]:

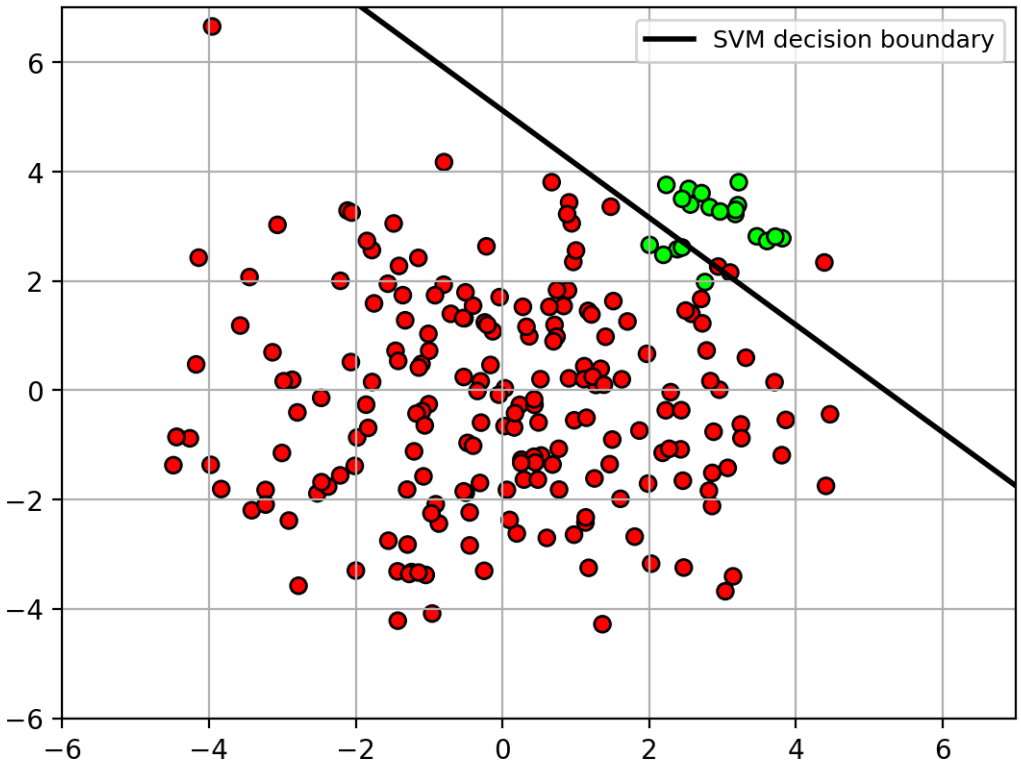


- Unbalanced data can cause problems when training the classifier
  - classifier will focus more on the class with more points.
  - decision boundary is pushed away from class with more points

[16]:
```
clf = svm.SVC(kernel='linear', C=10)
clf.fit(X, Y)

udatafig1 = plt.figure()
plt.scatter(X[:,0],X[:,1],c=Y,cmap=mycmap, edgecolors='k')

w = clf.coef_[0]
b = clf.intercept_[0]
l1 = drawplane(w, b, lw=2, ls='k-')
plt.legend((l1,), ('SVM decision boundary',), fontsize=9)
plt.axis([-6, 7, -6, 7])
plt.grid(True)
plt.close()
```

[17]:
```
udatafig1
```

[17]:



- **Solution:** apply weights on the classes during training.
  - weights are inversely proportional to the class size.

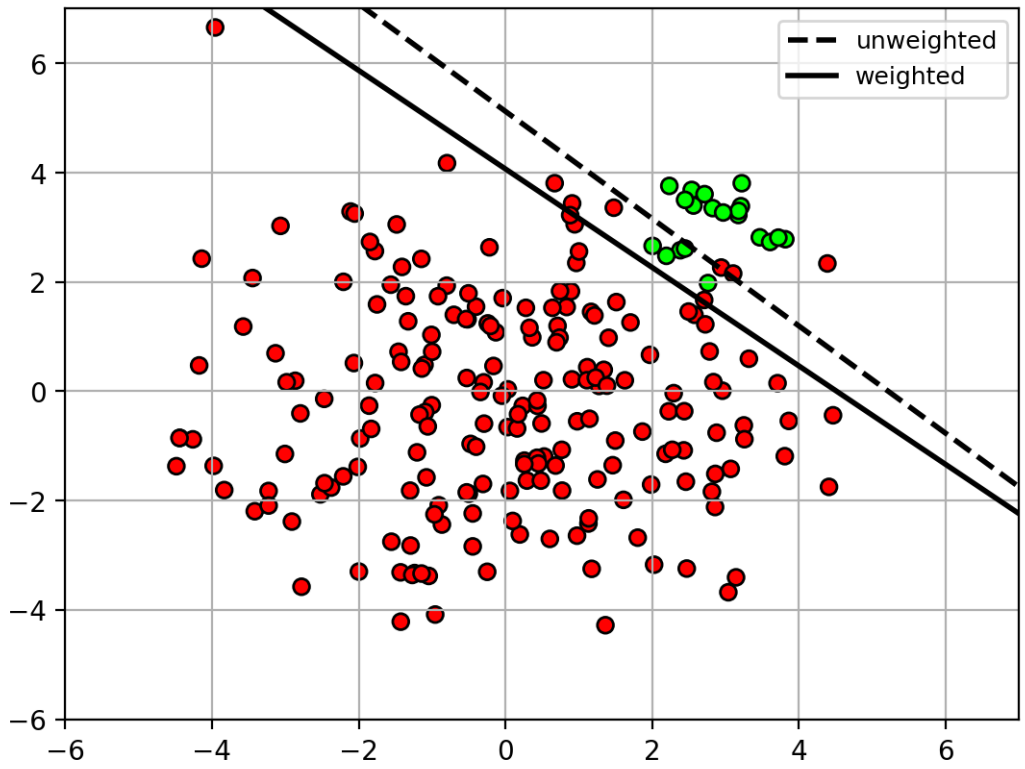
[18]:
```
clfw = svm.SVC(kernel='linear', C=10,  class_weight='balanced')
clfw.fit(X, Y)

print("class weights =", clfw.class_weight_)
```

```
class weights = [0.55 5.5 ]
```

[19]:
```
udatafig2 = plt.figure()
plt.scatter(X[:,0],X[:,1],c=Y,cmap=mycmap, edgecolors='k')

w = clf.coef_[0]
b = clf.intercept_[0]
ww = clfw.coef_[0]
bw = clfw.intercept_[0]
l1 = drawplane(w, b, lw=2, ls='k--')
l2 = drawplane(ww, bw, lw=2, ls='k-')
plt.legend((l1,l2), ('unweighted', 'weighted'), fontsize=9)
plt.axis([-6, 7, -6, 7])
plt.grid(True)
plt.close()
```

[20]:
```
udatafig2
```

[20]:



11 Classifier Imbalance

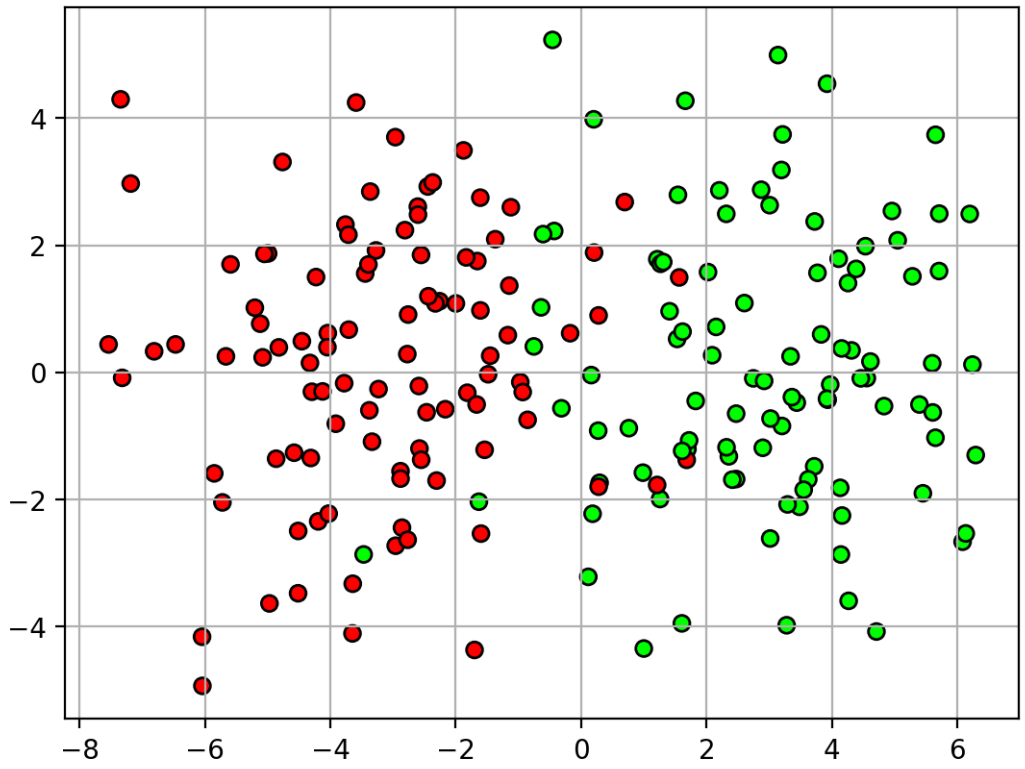
- In some tasks, errors on certain classes cannot be tolerated.
- **Example:** detecting spam vs non-spam
  - non-spam should *definitely not* be marked as spam
    - okay to mark some spam as non-spam

[21]:
```
X,Y = datasets.make_blobs(n_samples=200,
         centers=[[-3,0],[3,0]], cluster_std=2, n_features=2, random_state=447)
udatafig3 = plt.figure()
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.grid(True)
plt.close()

clf = svm.SVC(kernel='linear', C=10)
clf.fit(X, Y)
```

[21]:
```
SVC(C=10, kernel='linear')
```

[22]:
```
udatafig3
```

[22]:



...

- Class weighting can be used to make the classifier focus on certain classes
  - e.g., weight non-spam class higher than spam class
    - classifier will try to correctly classify all non-spam samples, at the expense of making errors on spam samples.

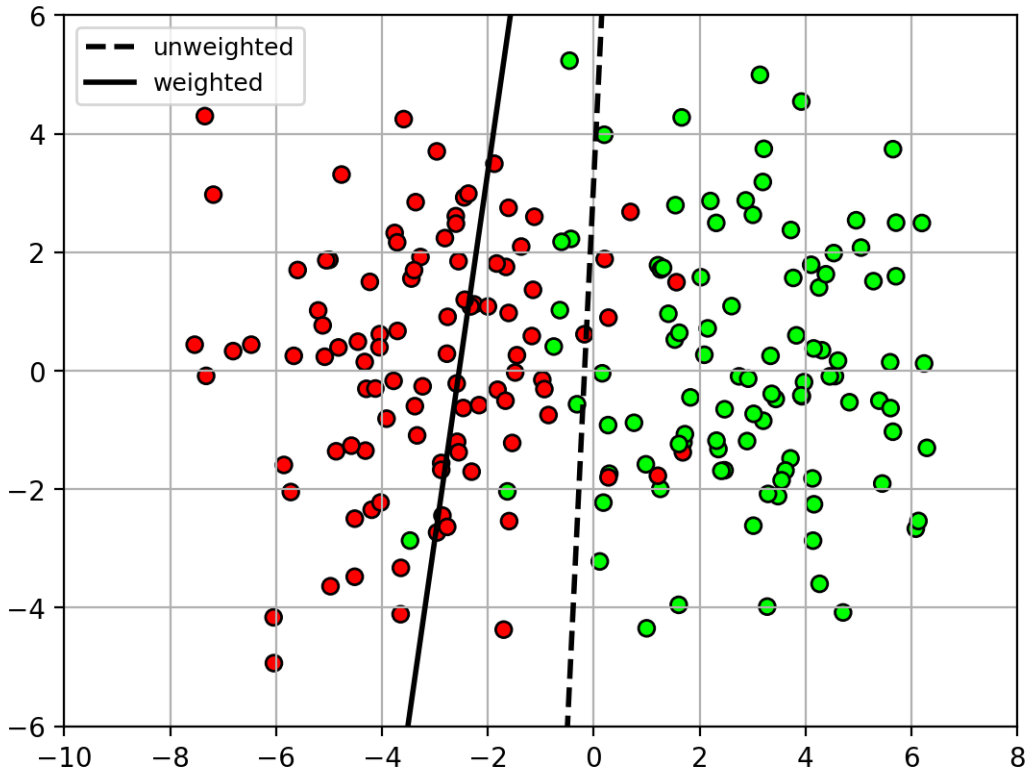
[23]:
```
# dictionary (key,value) = (class name, class weight)
cw = {0: 0.2,
      1:  5}  # class 1 is 25 times more important!

clfw = svm.SVC(kernel='linear', C=10,  class_weight=cw)
clfw.fit(X, Y);
```

[24]:
```
udatafig4 = plt.figure()
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.grid(True)

w = clf.coef_[0]
b = clf.intercept_[0]
ww = clfw.coef_[0]
bw = clfw.intercept_[0]
```

```
l1 = drawplane(w, b, lw=2, ls='k--')
l2 = drawplane(ww, bw, lw=2, ls='k-')
plt.legend((l1,l2), ('unweighted', 'weighted'), fontsize=9)
plt.axis([-10, 8, -6, 6])

plt.close()
```

[25]:
```
udatafig4
```

[25]:



12 Classification Summary

- **Classification task**
  - Observation $\mathbf{x}$: typically a real vector of feature values, $\mathbf{x} \in \mathbb{R}^d$.
  - Class y: from a set of possible classes, e.g., $Y = \{0, 1\}$
  - **Goal:** given an observation $\mathbf{x}$, predict its class y.

13 Loss functions

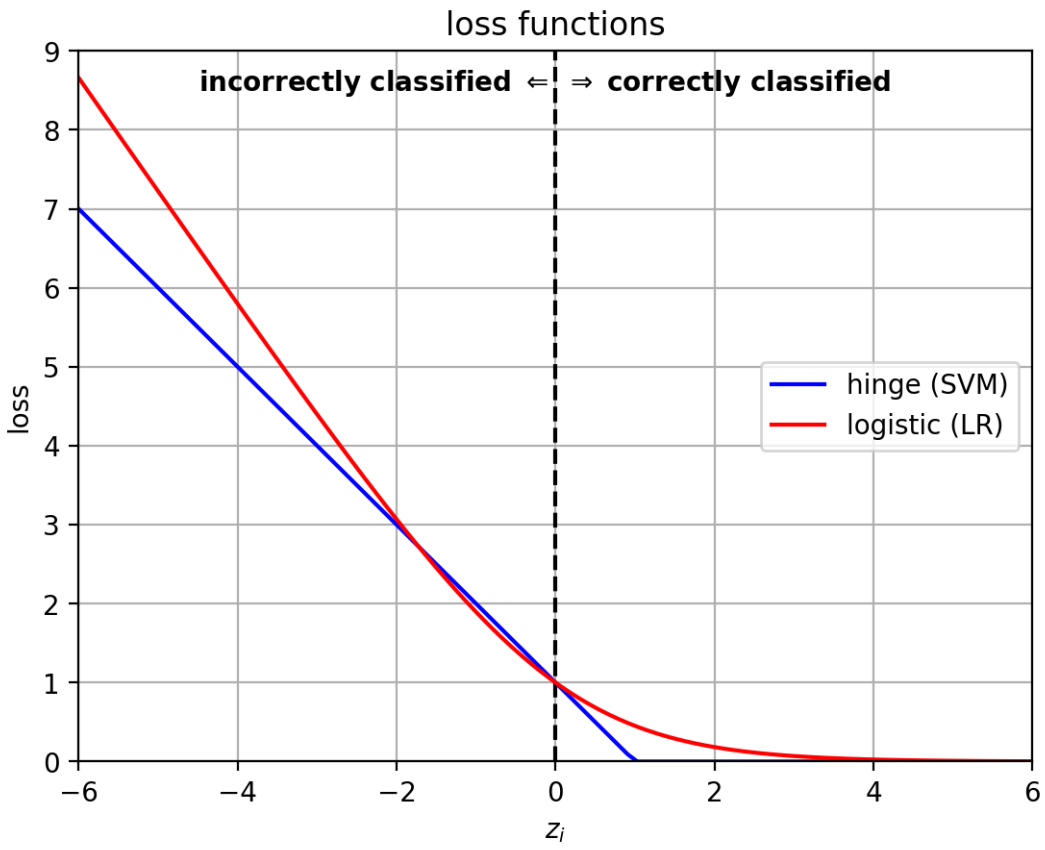
- The classifiers differ in their loss functions, which influence how they work.
  - $z_i = y_i f(\mathbf{x}_i)$

[28]:
```
z = linspace(-6,6,100)
logloss = log(1+exp(-z)) / log(2)
hingeloss = maximum(0, 1-z)
exploss = exp(-z)
lossfig = plt.figure()

plt.plot([0,0], [0,9], 'k--')
plt.text(0,8.5, "incorrectly classified $\\Leftarrow$ ", ha='right',␣
 ↪weight='bold')
plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left', weight='bold')

plt.plot(z,hingeloss, 'b-', label='hinge (SVM)')
plt.plot(z,logloss, 'r-', label='logistic (LR)')
# plt.plot(z,exploss, 'g-', label='exponential (AdaBoost)')
plt.axis([-6,6,0,9]); plt.grid(True)
plt.xlabel('$z_i$');
plt.ylabel('loss')
plt.legend(loc='right', fontsize=10)
plt.title('loss functions')
plt.close()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\R'
<>:9: SyntaxWarning: invalid escape sequence '\R'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_43365/493515450.py:9:
SyntaxWarning: invalid escape sequence '\R'
  plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left',
weight='bold')
```

[29]:
```
lossfig
```

[29]:



14 Regularization and Overfitting

- Some models have terms to prevent overfitting the training data.
  - this can improve *generalization* to new data.
- There is a parameter to control the regularization effect.
  - select this parameter using cross-validation on the training set.

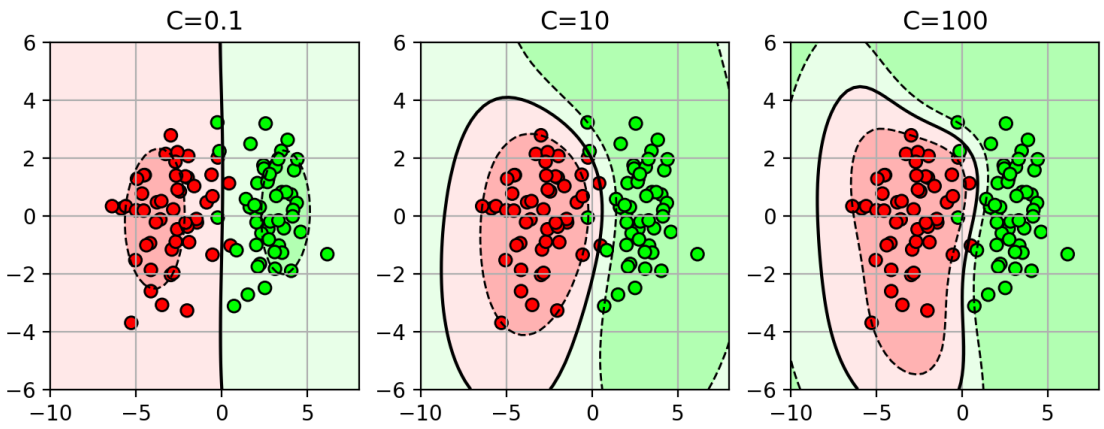
[30]:
```
X,Y = datasets.make_blobs(n_samples=100,
         centers=[[-3,0],[3,0]], cluster_std=1.5, n_features=2,␣
 ↪random_state=447)
udatafig3 = plt.figure()
axbox = [-10,8,-6,6]
xr = [linspace(axbox[0], axbox[1], 50), linspace(axbox[2], axbox[3], 50)]

Cs = [0.1, 10, 100]
clf={}

ofig = plt.figure(figsize=(9,3))
for i,C in enumerate(Cs):
```

```
    clf[C] = svm.SVC(kernel='rbf', C=C, gamma=0.05)
    clf[C].fit(X, Y)

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    score = clf[C].decision_function(allpts).reshape(xgrid0.shape)

    cmap = ([1,0,0], [1,0.7,0.7], [0.7,1,0.7], [0,1,0])

    plt.subplot(1,len(Cs),i+1)
    plt.contourf(xr[0], xr[1], score, colors=cmap,
             levels=[-1000, -1, 0, 1, 1000], alpha=0.3)
    plt.contour(xr[0], xr[1], score, levels=[-1, 1], linewidths=1,␣
 ↪linestyles='dashed', colors='k')
    plt.contour(xr[0], xr[1], score, levels=[0], linestyles='solid', colors='k')

    plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')

    #plt.plot(clf.support_vectors_[:,0], clf.support_vectors_[:,1],
    #         'ko',fillstyle='none', markeredgewidth=2)
    plt.axis(axbox); plt.grid(True)
    plt.title('C='+str(C))
plt.close()
```

```
<Figure size 640x480 with 0 Axes>
```

[31]:
```
ofig
```

[31]:



15 Structural Risk Minimization

- A general framework for balancing data fit and model complexity.
- Many learning problems can be written as a combination of data-fit and regularization term:

$$f^* = \operatorname*{argmin}_f \sum_i L(y_i, f(\mathbf{x}_i)) + \lambda\Omega(f)$$

  - assume f within some class of funcitions, e.g., linear functions $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$.
  - L is the loss function, e.g., logistic loss.
  - Ω is the regularization function on f, e.g., $||\mathbf{w}||^2$
  - λ is the tradeoff parameter, e.g., $1/C$.

16 Other things

- *Multiclass classification*
  - can use binary classifiers to do multi-class using *1-vs-rest* formulation.
- *Feature normalization*
  - normalize each feature dimension so that some feature dimensions with larger ranges do not dominate the optimization process.
- *Unbalanced data*
  - if more data in one class, then apply weights to each class to balance objectives.
- *Class imbalance*
  - mistakes on some classes are more critical.
  - reweight class to focus classifier on correctly predicting one class at the expense of others.

17 Applications

- Web document classification, spam classification
- Face gender recognition, face detection, digit classification

18 Features

- Choice of features is important!
  - using uninformative features may confuse the classifier.
  - use domain knowledge to pick the best features to extract from the data.

19 Which classifier is best?

- **"No Free Lunch" Theorem** (Wolpert and Macready)

  "If an algorithm performs well on a certain class of problems then it necessarily pays for that with degraded performance on the set of all remaining problems."

- In other words, there is no *best* classifier for all tasks. The best classifier depends on the particular problem.